CE310-04-class-SOLUTIONS

February 6, 2026

1 CE310 Class 04

Real Valued Genetic Algorithms and Travelling Sales-Person Problem (TSP)

Traveling Salesperson Problem: Given `N` cities with mutual distance `d[i,j]`, find the order in which to visit the cities and get back to the city you started from, *as to minimise the distance travelled.*

While solutions to the TSP are *tours*, they can easily be represented as *permutations*. For instance, `[4, 3, 1, 2]` means "visit city 4, then 3, then 1, then 2 and then go back to 4 to complete the tour". So, the total distance travelled would be: `d[4,3] + d[3,1] + d[1,2] + d[2,4]`

TASKS:

1. Understand and test the following code that generates a random set of `N` cities and precomputes an array of distances `d[i,j]` for all pairs of cities.

```
import numpy as np

# Set the number of cities
N = 20

# Generate an array random x and y coordinates for the cities
# in a region of 100 by 100 miles
xy = 100 * np.random.rand(N,2)

# Create an numpy array with N rows and N columns for storing
# the distances between every pair of city

d = np.empty((N,N))
for i in range(N):
    for j in range(N):
        # Calculate Euclidean distance
        dx = xy[i,0] - xy[j,0]
        dy = xy[i,1] - xy[j,1]
        d[i,j] = np.sqrt(dx**2 + dy**2)
```

You can use the following code to plot the cities and visualise the distance matrix:

```
# Visualising the distance matrix as a table
import pandas as pd
print(pd.DataFrame(d))
```

```python
# Visualising the distance matrix as a heatmap
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
sns.heatmap(d, annot=False, cmap='viridis')
plt.title("Distance Matrix as Heatmap")
plt.ylabel("From City")
plt.xlabel("To City")
plt.show()
```

2. Study and test the following TSP simulator, that uses the array `d` to calculate the total distance travelled by a salesperson if visiting the cities in the order specified by a particular `permutation`:

```python
def tsp_simulator(permutation):
    dist = 0
    for city in range(N-1):
        dist += d[permutation[city],permutation[city+1]]
    dist += d[permutation[0],permutation[city+1]] # Adding the return leg to the starting city
    return dist
```

3. Like with did with the bin-packing problem, let us use *random search* to solve the TSP. Specifically, generate 1,000,000 random permutations (i.e., random tours), running the TSP simulator on each and ensuring you save the best permutation found.

We have already seen a couple of ways of generating random permutations. Here is another one:

```python
permutation = np.random.permutation(range(N))
```

- What's the minimal tour length you can get by random search?
- Visualise your best tour using the following code

```python
# Plotting tours
def plot_tour(perm):
    for city in range(N-1):
        plt.plot([xy[perm[city]][0],xy[perm[city+1]][0]],
                 [xy[perm[city]][1],xy[perm[city+1]][1]],'b-')
    plt.plot([xy[perm[0]][0],xy[perm[city+1]][0]],
             [xy[perm[0]][1],xy[perm[city+1]][1]],'b-')
    plt.plot(xy[:,0],xy[:,1],'r*')
    plt.show()
```

- See how the distance matrix changes if you reordered the cities based on your best permutation. The calculation is trivial: `permuted_d = d[best_perm][:, best_perm]`. Visualise `permuted_d` and compare with the original. Can you explain what you see?

4. Now let us transition to a *new way of representing and generating random permutations which will make it easy for us to use a GA to solve TSPs.*

- Imagine that we *assign to each city a numerical value representing the* ***priority*** with which it should be visited (and assume there are no ties).

- It is then easy to turn these priorities into a permutation/tour: the city with highest priority should be visited first, the city with the second highest priority should be second, and so on.
- So, an *array of floats in an indirect representation for permutations.*
- How do we transform a `priorities` array into a `permutation`? Super easy:

```
permutation = np.argsort(priorities)[::-1]
```

Test what happens if you apply `argsort` to the list `[0.4, 0.2, 1.1, 0.1]`.

5. Test that this new representation works OK, by modifying your random search code developed in Task 3 above so that it generates permutations as follows:

```
priorities = np.random.rand(N) # An array of random numbers between 0 and 1
permutation = np.argsort(priorities)[::-1]
```

Run the new random search code, and see if you more or less get the same result as in Task 3.

6. The plan is to use a *Real-Valued GA* to solve the TSP problem. To do this we need to **copy over and adapt the binary GA we defined in last week's lab** so it works on real-valued chromosomes. If you didn't complete the implementation of the binary GA, see my solutions now available in Moodle here [https://moodle.essex.ac.uk/mod/folder/view.php?id=1418183].

To get started, let us just do minimal changes to it so we get something running as soon as possible:

- Change the *initisation* so that we create a population of *arrays of floats* (priorities) (using `np.random.rand(N)`) instead of 0s and 1s
- Redefine the *fitness function* as follows:

```
def fitness(priorities):
    permutation = np.argsort(priorities)[::-1]
    distance = tsp_simulator(permutation)
    fitness = 1.0 / distance # So the higher the distance the worse the fitness
    return fitness
```

- For now, let us *switch off mutation* by either commenting out any use of `mutation()` in the code or redefining it as

```
# A fake mutation which just CLONES a parent
def mutate():
    idx = tournament()
    parent = population[idx]
    return parent
```

Later we will redefine it properly.

- Also, for now, leave `crossover()` as it is. We know that one-point crossover is not very suitable for floats, but we will fix this later.

7. Run this GA for 1,000,000 iterations (fitness evaluations) to see if the results we obtain are comparable with those obtained by random search.

8. Let us now *reintroduce mutation.* One simple way to mutate is to add small random uniform displacements. For instance:

```
def mutate():
    idx = tournament()
    parent = population[idx]
    offset = 0.05 * (np.random.rand(N) - 0.5)
    offspring = parent + offset
    return offspring
```

- Note that 0.05 is not necessarily optimal, so some tweaking may be necesary.
- A more sophisticated method would be to use normal deviates with 0 mean and standard deviation 0.05. See Unit 3 lecture slides 32 and 33.
- *Re-run the algorithm with this mutation included.* Do you get better results (numerically, plotting the tours and looking at distance matrix heat-map)?

9. It is now time to *improve crossover*. Use either *line crossover* or *box crossover*, see Unit 3 lecture slides 29 to 31. Implement, test and re-run the simulation to see if results are better.
10. It is clear that ideal tours should not present edges that cross each other. Yet, for large enough `N` you will probably find that there are lots of crossings in the end result. Implement and test a new mutation operator that does the following:
    - Copies the parental array into the offspring array
    - Selects two random positions in the offspring array and inverts the order of the floats stored between those positions.
    - Returns the offspring Does this operator help?

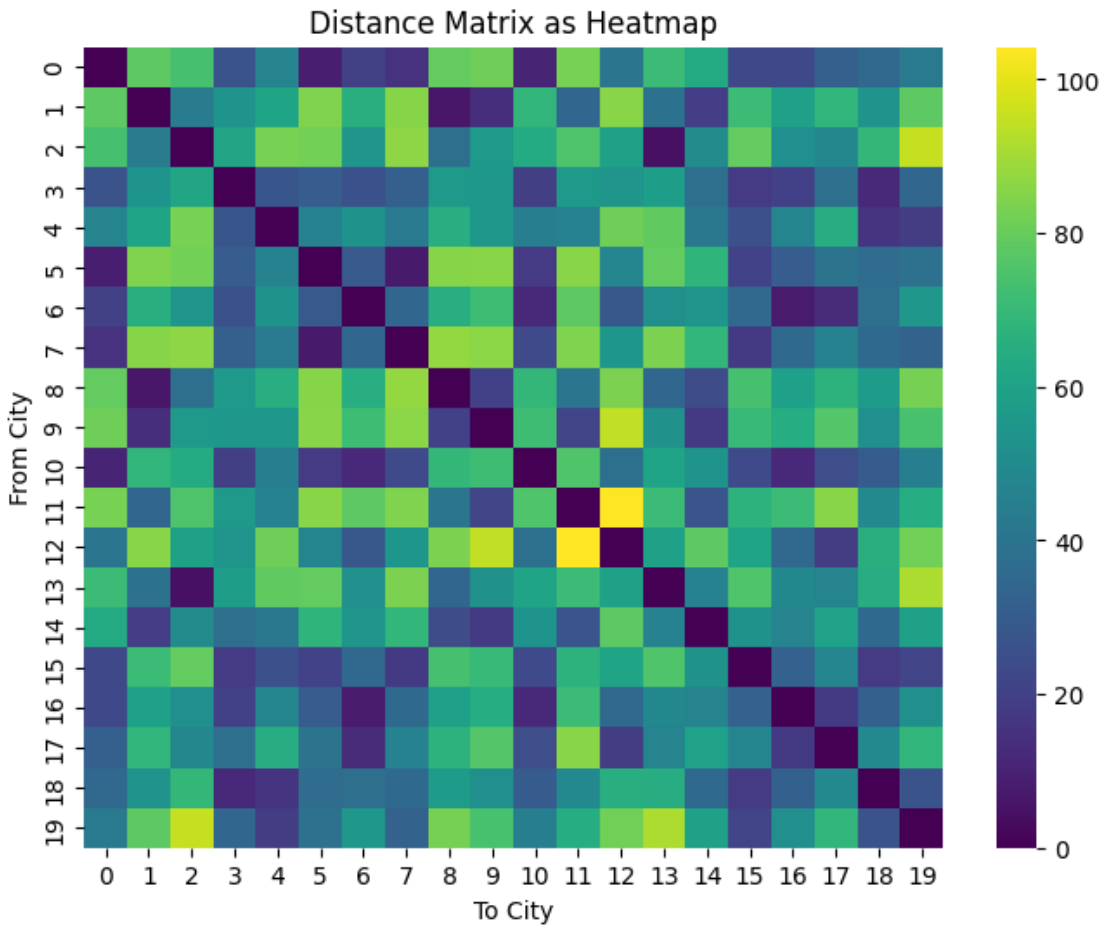
1.1 Task 1

[1]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Creating the cities
N = 20
xy = 100 * np.random.rand(N,2)

# Computing distance matrix
d = np.empty((N,N))
for i in range(N):
    for j in range(N):
        # Calculate Euclidean distance
        dx = xy[i,0] - xy[j,0]
        dy = xy[i,1] - xy[j,1]
        d[i,j] = np.sqrt(dx**2 + dy**2)

def visualise(d):
    # Visualising the distance matrix both numerically and as heat map
```

```
    pd.set_option('display.precision', 1) # Reducing precision for printing␣
↪bigger matrices
    print(pd.DataFrame(d))

    plt.figure(figsize=(8, 6))
    sns.heatmap(d, annot=False, cmap='viridis')
    plt.title("Distance Matrix as Heatmap")
    plt.ylabel("From City")
    plt.xlabel("To City")
    plt.show()

visualise(d)
```

```
      0     1     2     3     4     5     6     7     8     9    10    11  \
0    0.0  78.2  73.7  26.5  47.0   8.6  20.1  15.0  79.2  81.1  10.5  82.9
1   78.2   0.0  42.9  54.1  60.7  83.9  65.8  85.4   6.3  13.7  68.2  34.0
2   73.7  42.9   0.0  61.4  82.8  81.9  54.3  86.6  37.6  56.3  63.6  75.0
3   26.5  54.1  61.4   0.0  27.7  30.5  26.0  31.3  56.2  55.2  19.3  56.5
4   47.0  60.7  82.8  27.7   0.0  46.1  53.2  42.4  65.2  55.3  44.2  46.1
5    8.6  83.9  81.9  30.5  46.1   0.0  28.6   7.1  85.4  85.7  18.3  85.6
6   20.1  65.8  54.3  26.0  53.2  28.6   0.0  34.6  65.5  71.6  12.0  78.3
7   15.0  85.4  86.6  31.3  42.4   7.1  34.6   0.0  87.4  86.0  23.3  84.2
8   79.2   6.3  37.6  56.2  65.2  85.4  65.5  87.4   0.0  19.8  69.0  40.3
9   81.1  13.7  56.3  55.2  55.3  85.7  71.6  86.0  19.8   0.0  71.9  20.8
10  10.5  68.2  63.6  19.3  44.2  18.3  12.0  23.3  69.0  71.9   0.0  75.3
11  82.9  34.0  75.0  56.5  46.1  85.6  78.3  84.2  40.3  20.8  75.3   0.0
12  40.3  85.8  59.6  54.3  81.4  47.5  28.3  54.6  83.7  94.3  37.9 104.2
13  71.0  39.2   4.3  57.8  78.7  79.1  52.0  83.6  34.1  52.4  60.8  70.8
14  63.2  19.0  50.3  37.5  41.7  68.0  54.4  68.5  23.6  17.8  54.1  26.9
15  22.5  70.9  79.2  18.3  25.4  20.7  34.8  17.5  73.6  70.0  23.3  66.9
16  22.5  59.0  51.4  19.8  47.5  30.5   7.5  35.3  59.0  64.3  12.2  70.8
17  31.2  68.4  48.3  37.5  65.1  39.6  12.7  46.1  66.9  76.3  24.6  85.7
18  34.9  53.7  68.8  11.7  16.2  36.6  37.6  35.2  57.1  51.9  29.8  49.1
19  42.5  78.3  95.2  34.5  18.7  38.4  55.5  32.5  82.3  73.7  44.1  64.5

       12    13    14    15    16    17    18    19
0    40.3  71.0  63.2  22.5  22.5  31.2  34.9  42.5
1    85.8  39.2  19.0  70.9  59.0  68.4  53.7  78.3
2    59.6   4.3  50.3  79.2  51.4  48.3  68.8  95.2
3    54.3  57.8  37.5  18.3  19.8  37.5  11.7  34.5
4    81.4  78.7  41.7  25.4  47.5  65.1  16.2  18.7
5    47.5  79.1  68.0  20.7  30.5  39.6  36.6  38.4
6    28.3  52.0  54.4  34.8   7.5  12.7  37.6  55.5
7    54.6  83.6  68.5  17.5  35.3  46.1  35.2  32.5
8    83.7  34.1  23.6  73.6  59.0  66.9  57.1  82.3
9    94.3  52.4  17.8  70.0  64.3  76.3  51.9  73.7
10   37.9  60.8  54.1  23.3  12.2  24.6  29.8  44.1
11  104.2  70.8  26.9  66.9  70.8  85.7  49.1  64.5
```

12	0.0	59.3	78.6	60.9	35.0	18.5	65.9	81.7
13	59.3	0.0	46.0	75.7	48.6	46.9	64.8	91.3
14	78.6	46.0	0.0	53.0	47.0	60.2	35.3	59.2
15	60.9	75.7	53.0	0.0	31.8	47.5	18.2	20.9
16	35.0	48.6	47.0	31.8	0.0	17.7	31.5	51.7
17	18.5	46.9	60.2	47.5	17.7	0.0	49.1	68.2
18	65.9	64.8	35.3	18.2	31.5	49.1	0.0	26.7
19	81.7	91.3	59.2	20.9	51.7	68.2	26.7	0.0



1.2 Task 2

[2]:
```
import random

def tsp_simulator(permutation):
    dist = 0
    for city in range(N-1):
        dist += d[permutation[city],permutation[city+1]]
```

```
    dist += d[permutation[0],permutation[city+1]] # Adding the return leg to␣
 ↪the starting city
    return dist

perm = list(range(N))
for _ in range(5):
    print(perm, tsp_simulator(perm))
    random.shuffle(perm) # shuffling in place
```

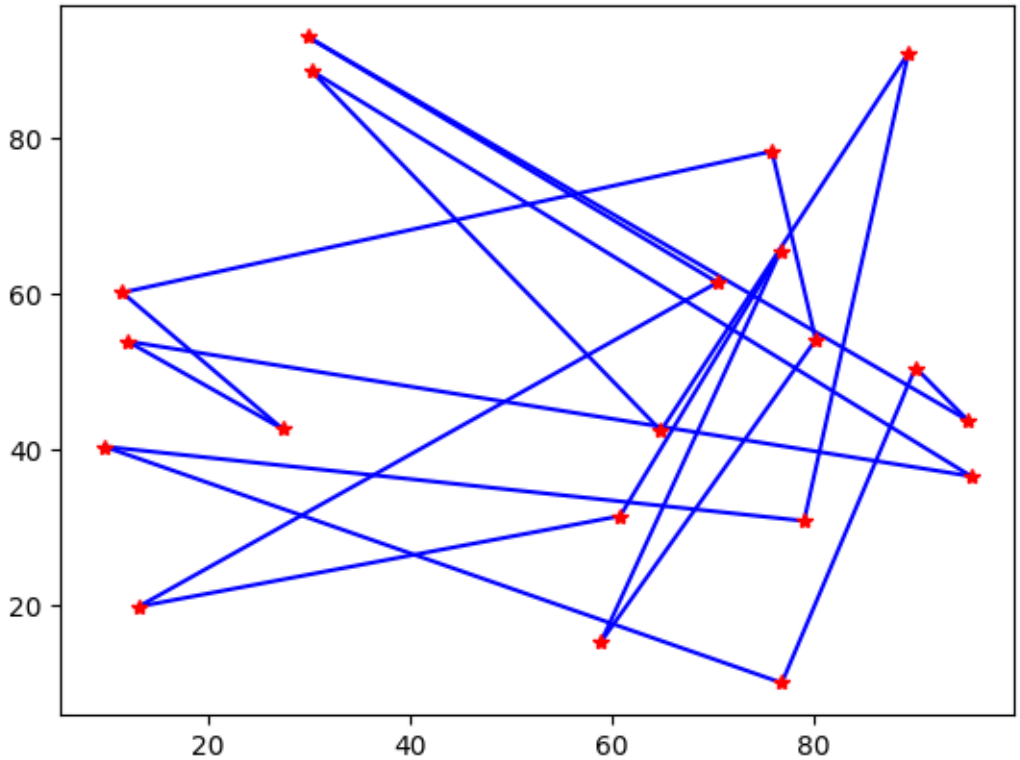
```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
1004.347873419609
[16, 2, 10, 19, 6, 17, 15, 7, 4, 3, 0, 14, 1, 9, 5, 12, 8, 18, 11, 13]
927.4060012700907
[2, 5, 6, 17, 10, 11, 16, 7, 18, 1, 4, 12, 15, 8, 3, 9, 0, 13, 19, 14]
1159.320912684732
[9, 19, 11, 12, 1, 4, 14, 15, 6, 3, 8, 5, 2, 10, 18, 16, 0, 13, 7, 17]
1192.2038689330307
[3, 13, 17, 14, 8, 19, 16, 11, 1, 15, 0, 10, 9, 2, 4, 6, 18, 5, 12, 7]
1003.141211560172
```

1.3 Task 3

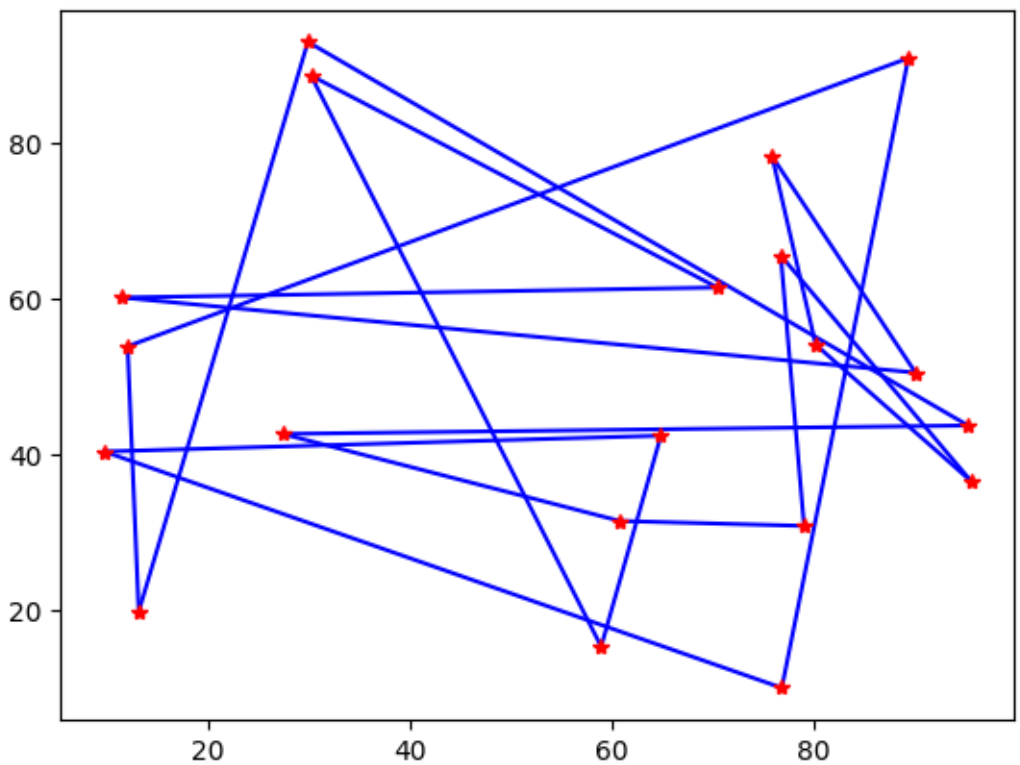
[3]:
```
from math import inf

# Plotting tours
def plot_tour(perm):
    for city in range(N-1):
        plt.plot([xy[perm[city]][0],xy[perm[city+1]][0]],
                 [xy[perm[city]][1],xy[perm[city+1]][1]],'b-')
    plt.plot([xy[perm[0]][0],xy[perm[city+1]][0]],
             [xy[perm[0]][1],xy[perm[city+1]][1]],'b-')
    plt.plot(xy[:,0],xy[:,1],'r*')
    plt.show()

best_perm = None
best_distance = inf

for _ in range(1_000_000):
    perm = np.random.permutation(range(N))
    distance = tsp_simulator(perm)
    if distance < best_distance:
        best_distance = distance
        best_perm = perm
        print(best_distance)
        plot_tour(best_perm)

permuted_d = d[best_perm][:,best_perm]
```

```
visualise(permuted_d)
```

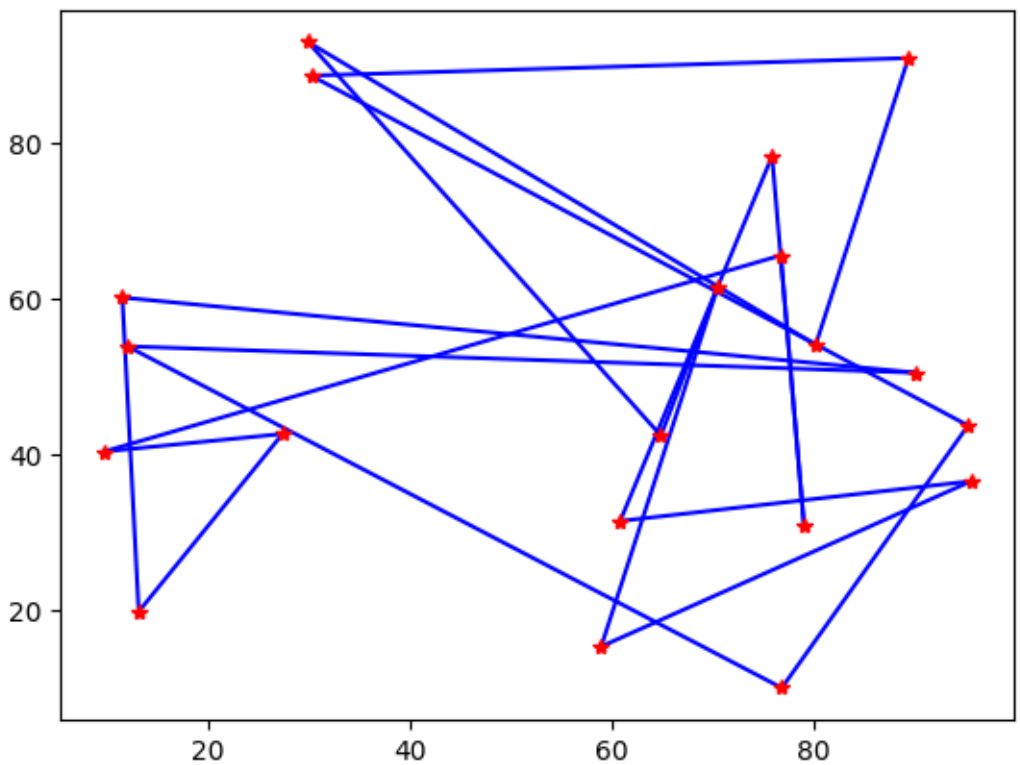
1059.0434488585126



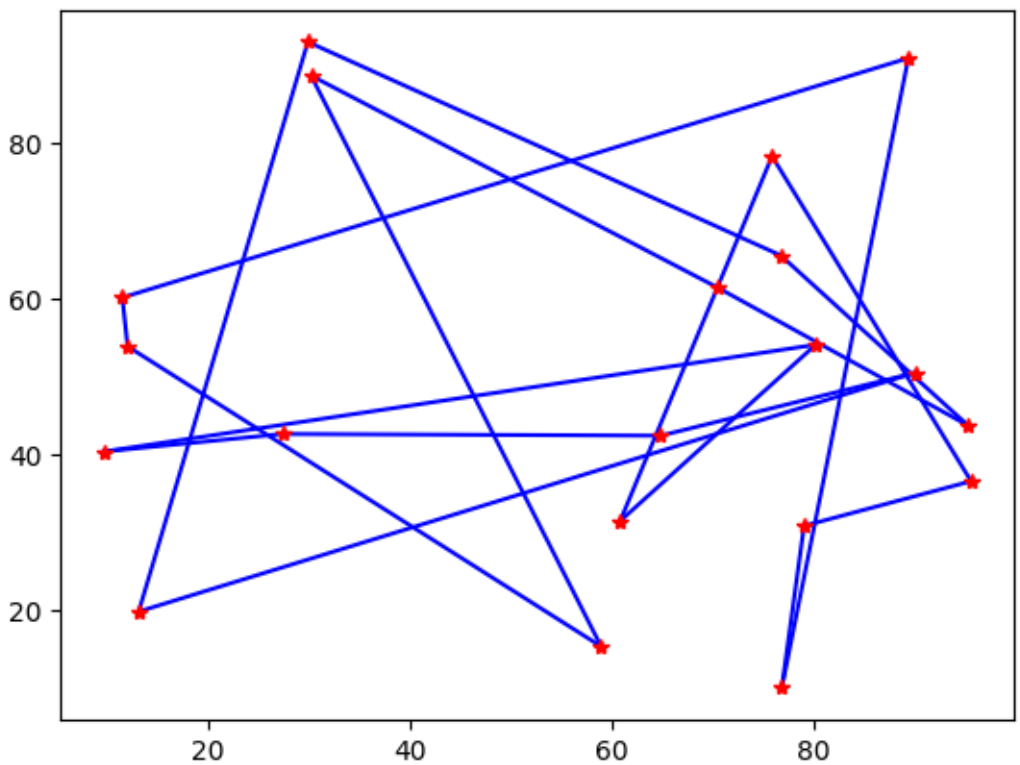
1050.356314964852



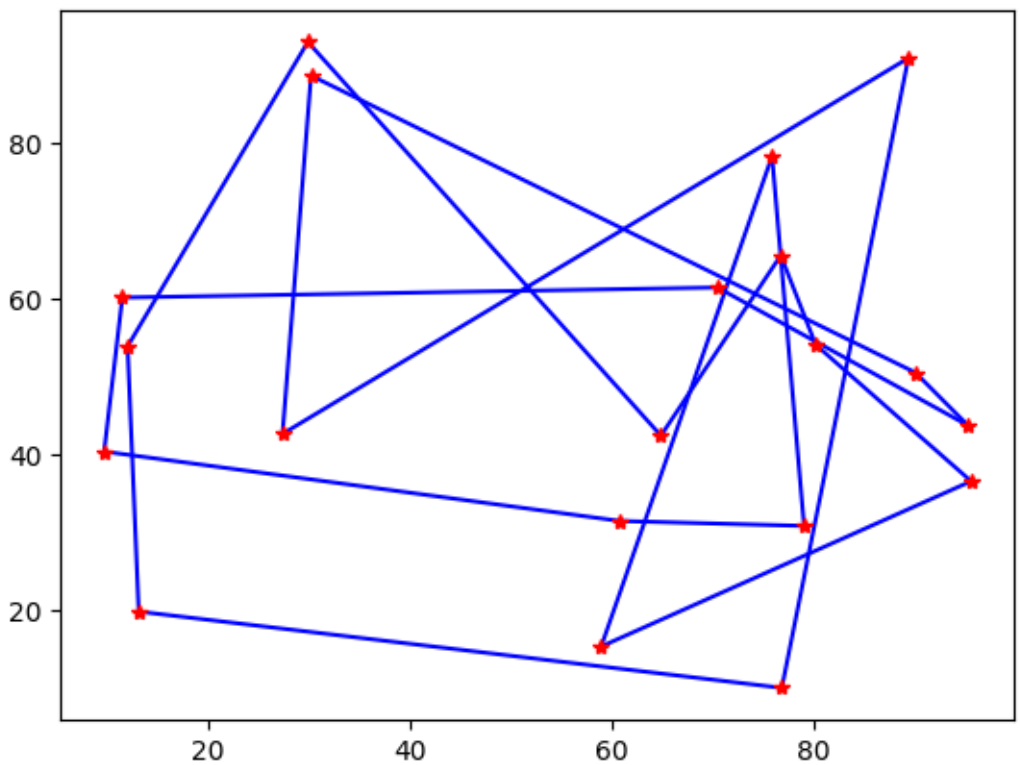
1008.1463991619813



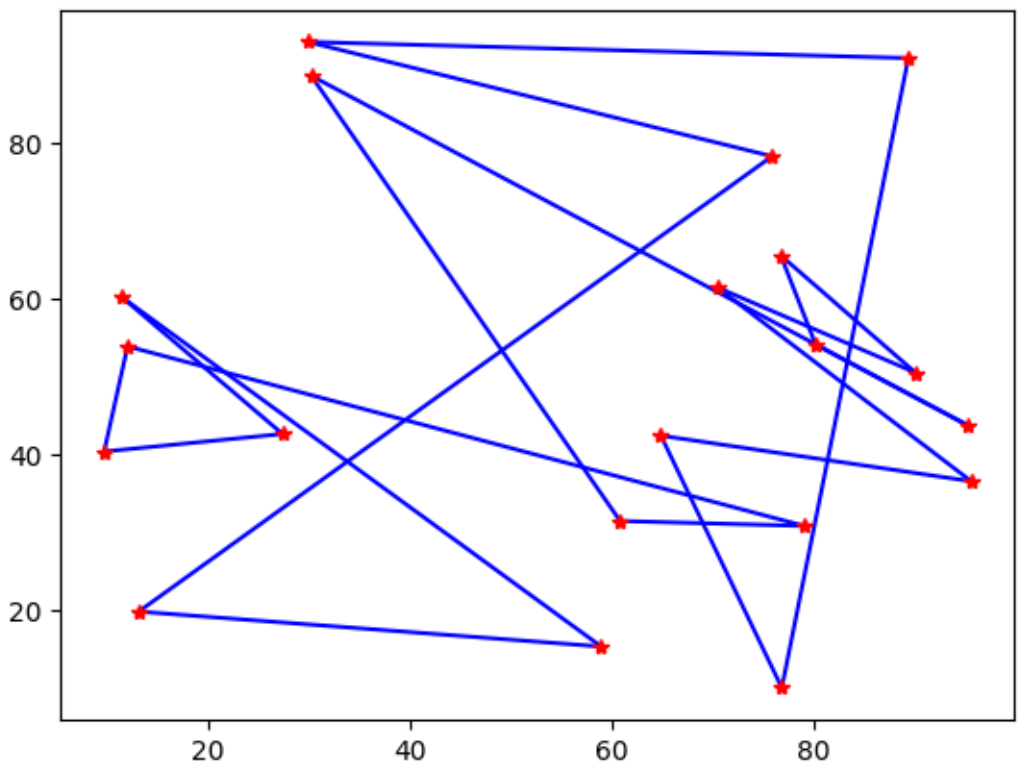
948.1612756342629



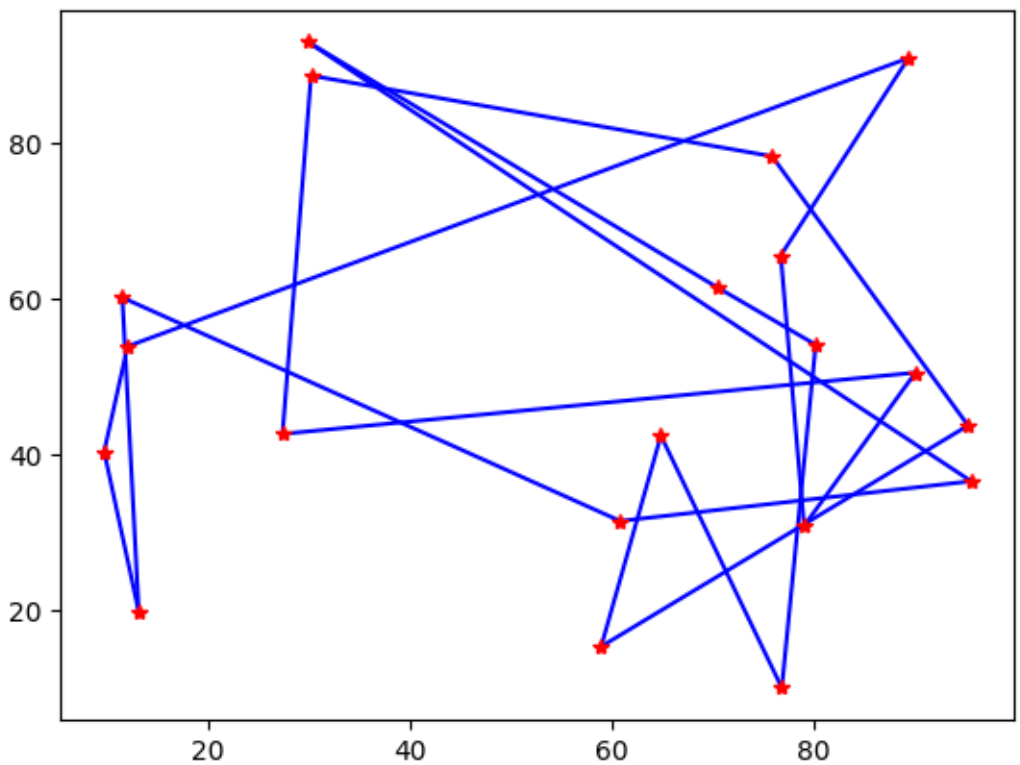
884.585824053633



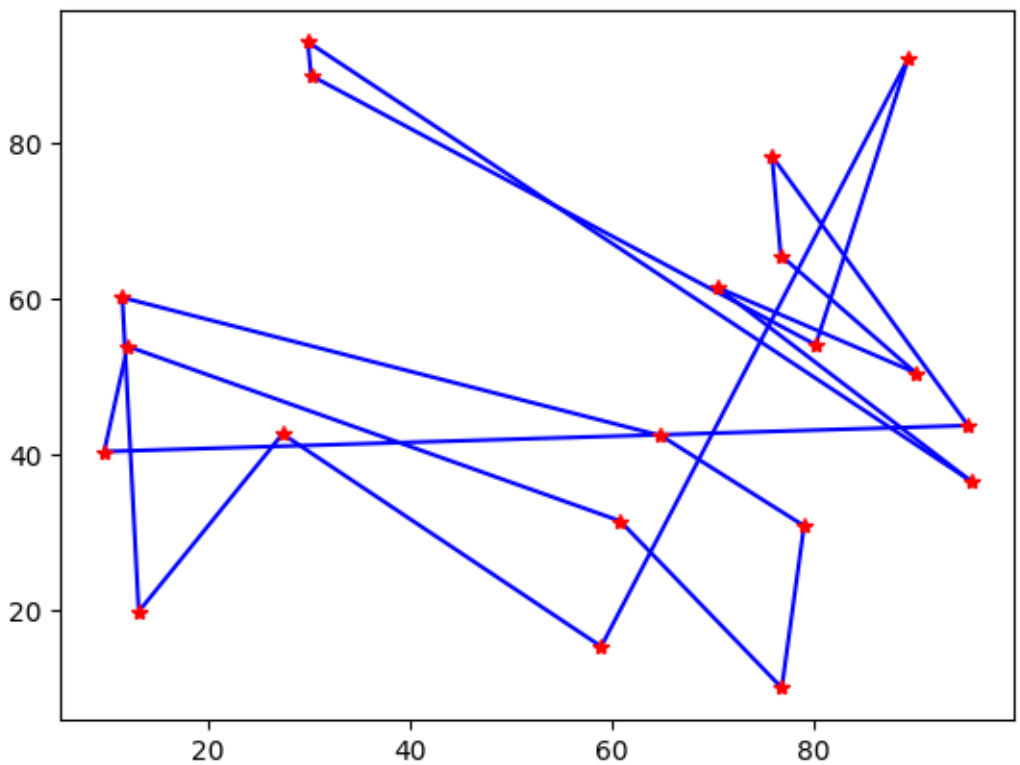
848.8250438652996



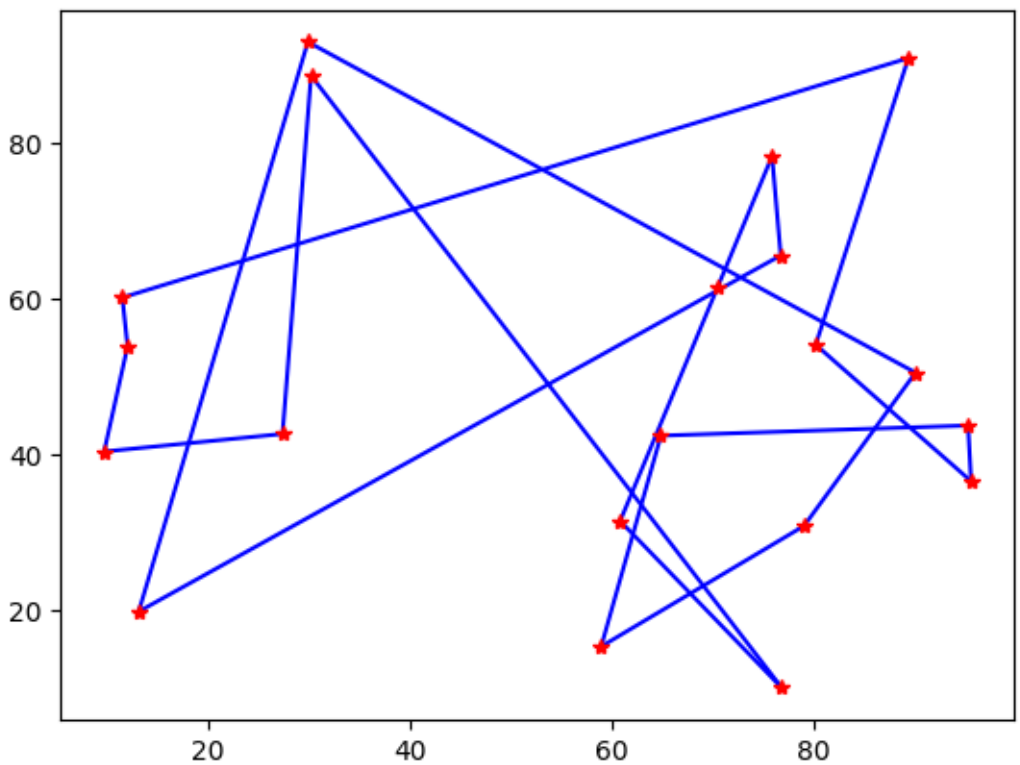
836.7694647894587



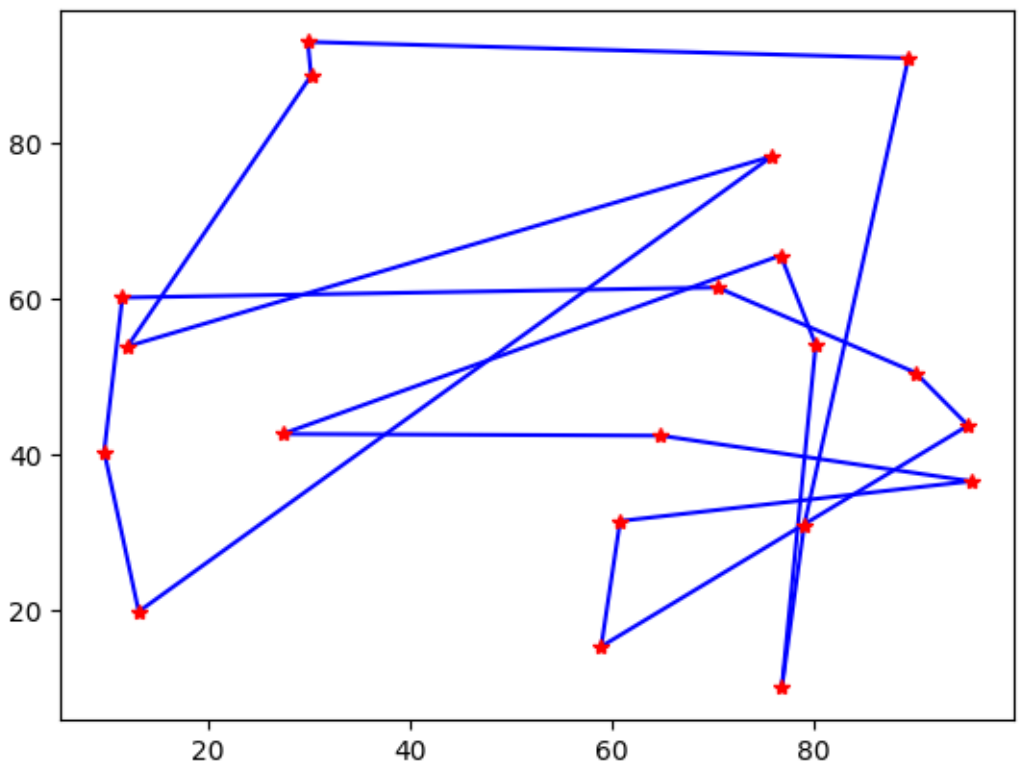
785.470208946648



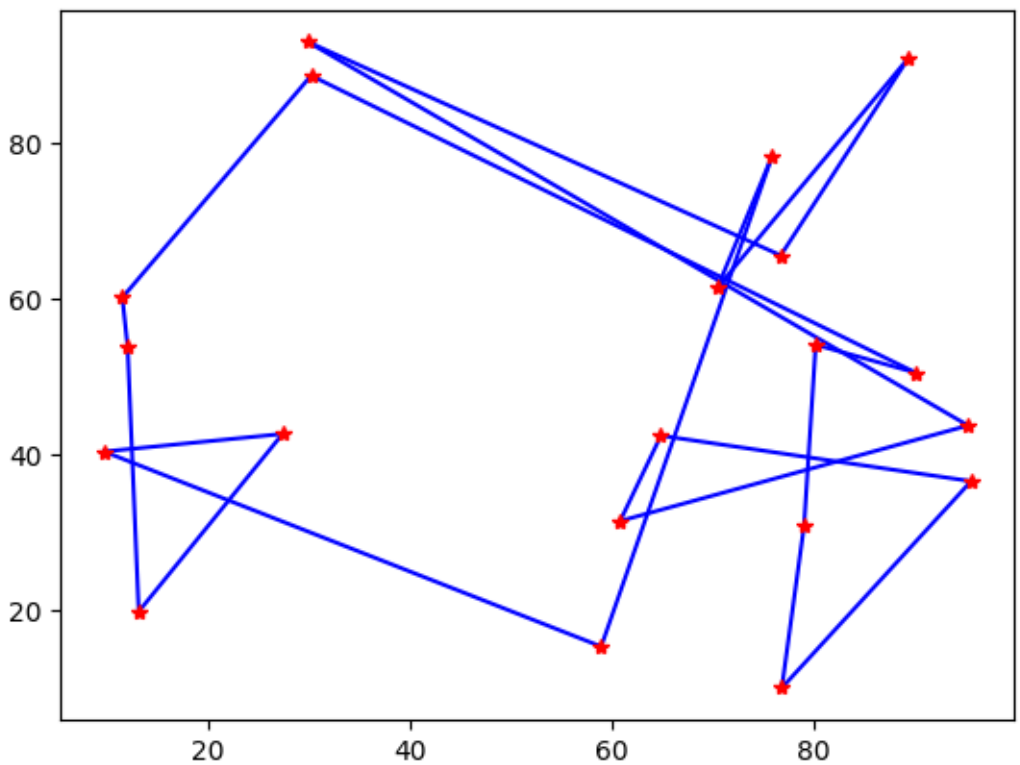

```
748.9765170537432
```



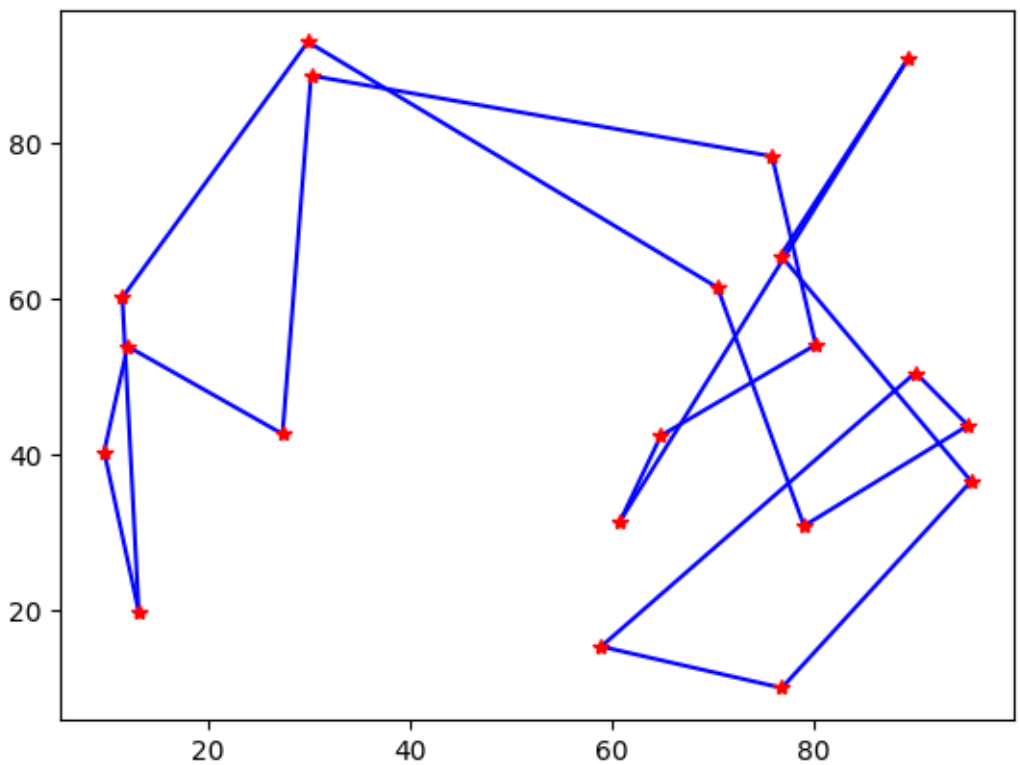
746.4845421969745



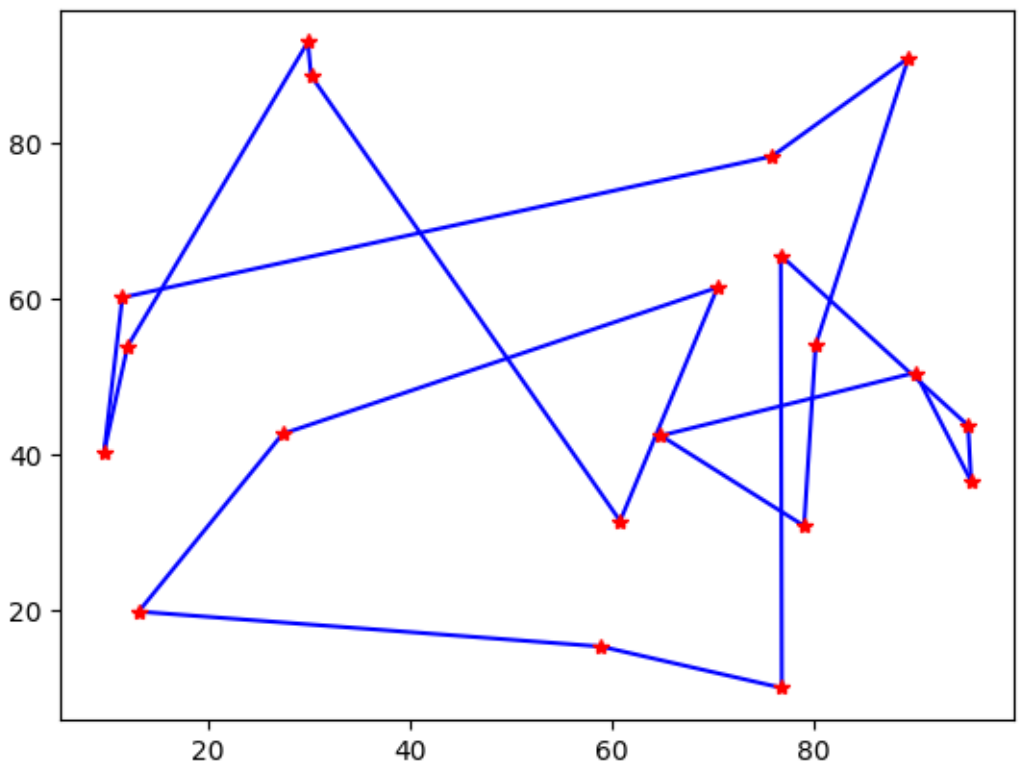
694.2849669766715



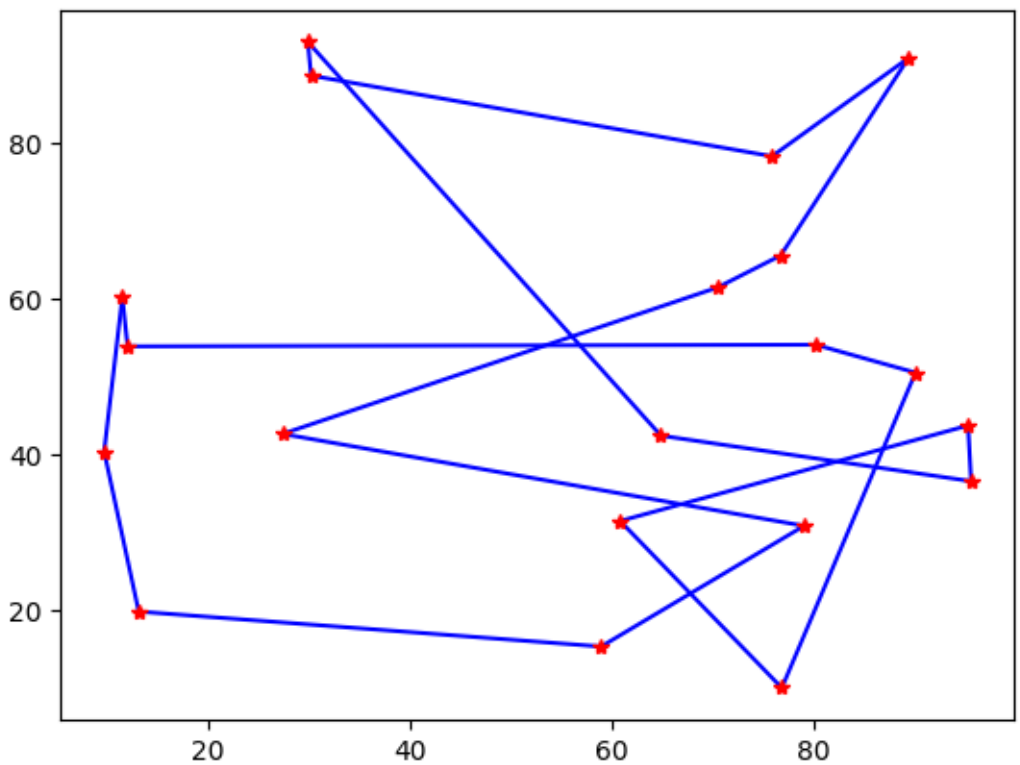
619.1119496434286



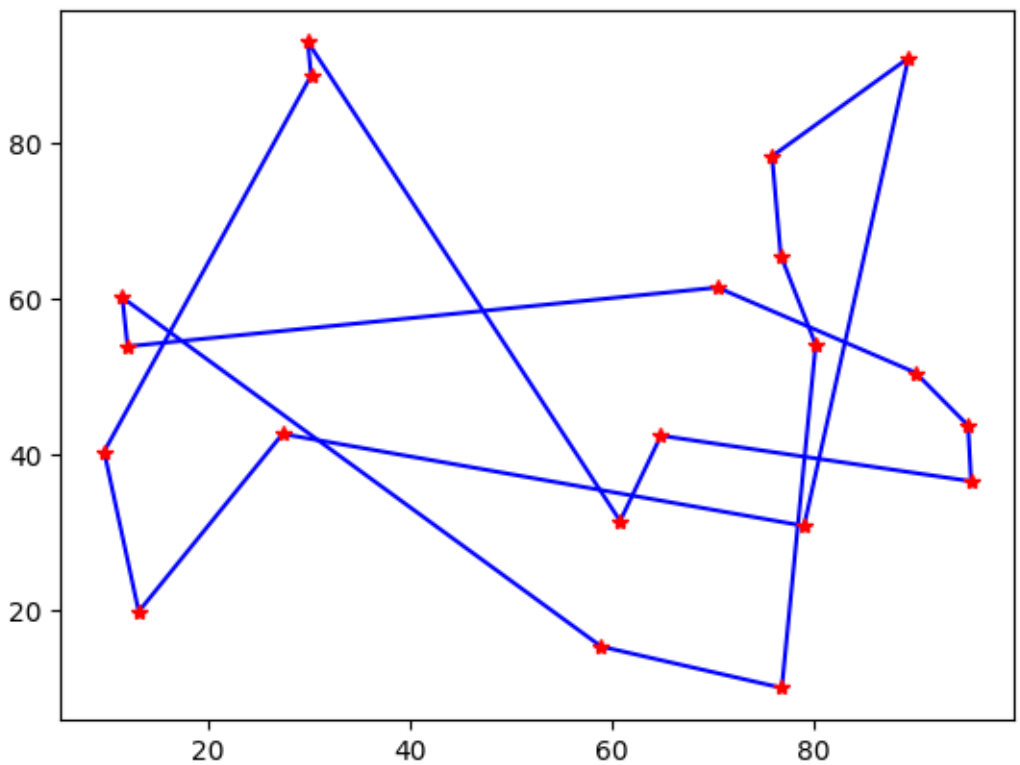
613.3076015418492



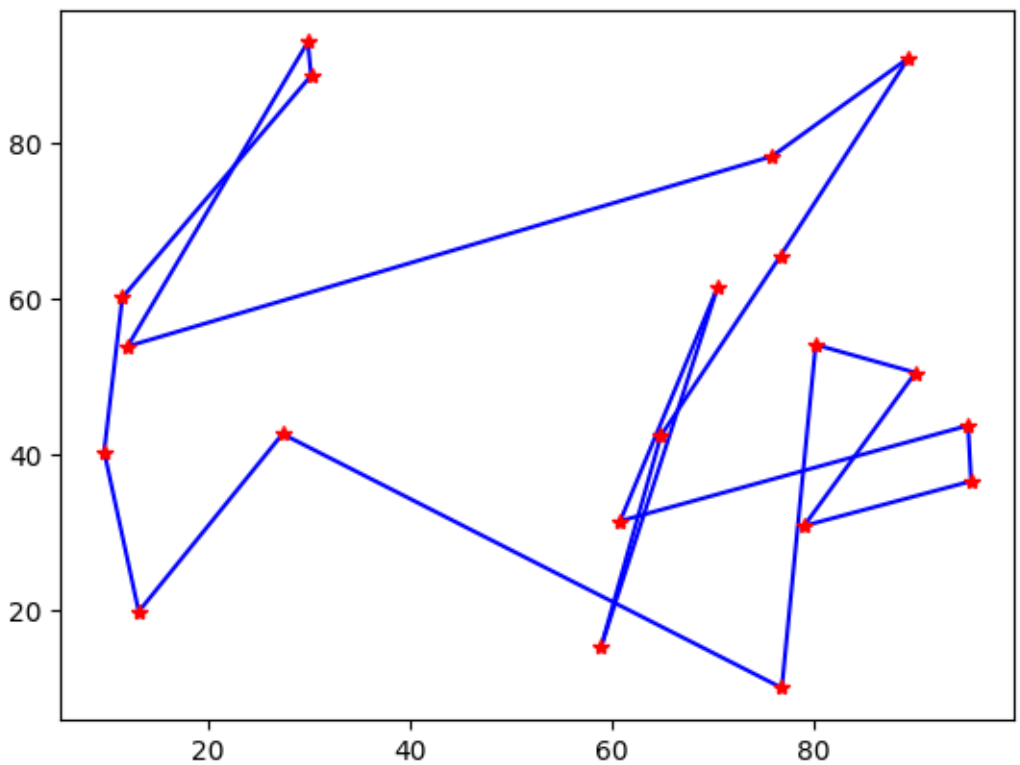
608.3642027413433



604.7404722932342

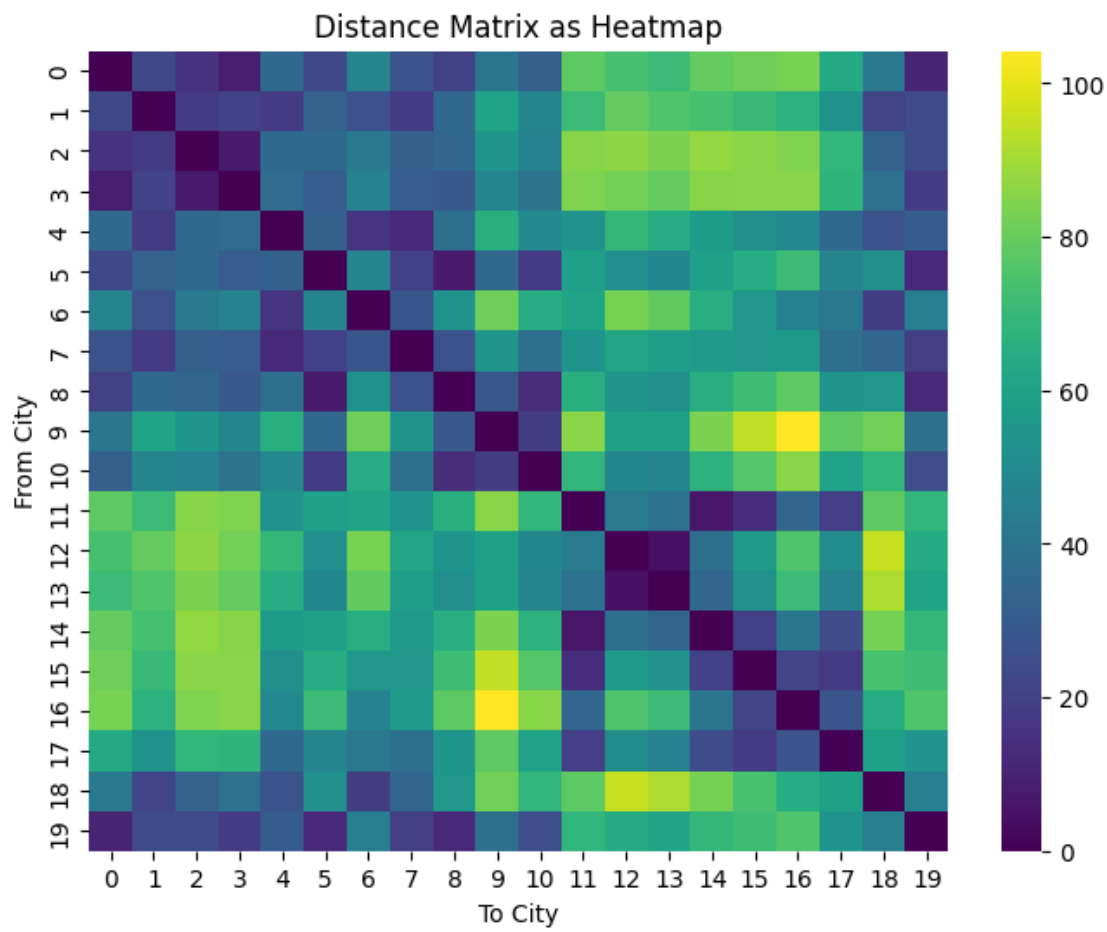


594.3311560695312



	0	1	2	3	4	5	6	7	8	9	10	11	\
0	0.0	22.5	15.0	8.6	34.9	22.5	47.0	26.5	20.1	40.3	31.2	78.2	
1	22.5	0.0	17.5	20.7	18.2	31.8	25.4	18.3	34.8	60.9	47.5	70.9	
2	15.0	17.5	0.0	7.1	35.2	35.3	42.4	31.3	34.6	54.6	46.1	85.4	
3	8.6	20.7	7.1	0.0	36.6	30.5	46.1	30.5	28.6	47.5	39.6	83.9	
4	34.9	18.2	35.2	36.6	0.0	31.5	16.2	11.7	37.6	65.9	49.1	53.7	
5	22.5	31.8	35.3	30.5	31.5	0.0	47.5	19.8	7.5	35.0	17.7	59.0	
6	47.0	25.4	42.4	46.1	16.2	47.5	0.0	27.7	53.2	81.4	65.1	60.7	
7	26.5	18.3	31.3	30.5	11.7	19.8	27.7	0.0	26.0	54.3	37.5	54.1	
8	20.1	34.8	34.6	28.6	37.6	7.5	53.2	26.0	0.0	28.3	12.7	65.8	
9	40.3	60.9	54.6	47.5	65.9	35.0	81.4	54.3	28.3	0.0	18.5	85.8	
10	31.2	47.5	46.1	39.6	49.1	17.7	65.1	37.5	12.7	18.5	0.0	68.4	
11	78.2	70.9	85.4	83.9	53.7	59.0	60.7	54.1	65.8	85.8	68.4	0.0	
12	73.7	79.2	86.6	81.9	68.8	51.4	82.8	61.4	54.3	59.6	48.3	42.9	
13	71.0	75.7	83.6	79.1	64.8	48.6	78.7	57.8	52.0	59.3	46.9	39.2	
14	79.2	73.6	87.4	85.4	57.1	59.0	65.2	56.2	65.5	83.7	66.9	6.3	
15	81.1	70.0	86.0	85.7	51.9	64.3	55.3	55.2	71.6	94.3	76.3	13.7	
16	82.9	66.9	84.2	85.6	49.1	70.8	46.1	56.5	78.3	104.2	85.7	34.0	
17	63.2	53.0	68.5	68.0	35.3	47.0	41.7	37.5	54.4	78.6	60.2	19.0	
18	42.5	20.9	32.5	38.4	26.7	51.7	18.7	34.5	55.5	81.7	68.2	78.3	
19	10.5	23.3	23.3	18.3	29.8	12.2	44.2	19.3	12.0	37.9	24.6	68.2	

	12	13	14	15	16	17	18	19
0	73.7	71.0	79.2	81.1	82.9	63.2	42.5	10.5
1	79.2	75.7	73.6	70.0	66.9	53.0	20.9	23.3
2	86.6	83.6	87.4	86.0	84.2	68.5	32.5	23.3
3	81.9	79.1	85.4	85.7	85.6	68.0	38.4	18.3
4	68.8	64.8	57.1	51.9	49.1	35.3	26.7	29.8
5	51.4	48.6	59.0	64.3	70.8	47.0	51.7	12.2
6	82.8	78.7	65.2	55.3	46.1	41.7	18.7	44.2
7	61.4	57.8	56.2	55.2	56.5	37.5	34.5	19.3
8	54.3	52.0	65.5	71.6	78.3	54.4	55.5	12.0
9	59.6	59.3	83.7	94.3	104.2	78.6	81.7	37.9
10	48.3	46.9	66.9	76.3	85.7	60.2	68.2	24.6
11	42.9	39.2	6.3	13.7	34.0	19.0	78.3	68.2
12	0.0	4.3	37.6	56.3	75.0	50.3	95.2	63.6
13	4.3	0.0	34.1	52.4	70.8	46.0	91.3	60.8
14	37.6	34.1	0.0	19.8	40.3	23.6	82.3	69.0
15	56.3	52.4	19.8	0.0	20.8	17.8	73.7	71.9
16	75.0	70.8	40.3	20.8	0.0	26.9	64.5	75.3
17	50.3	46.0	23.6	17.8	26.9	0.0	59.2	54.1
18	95.2	91.3	82.3	73.7	64.5	59.2	0.0	44.1
19	63.6	60.8	69.0	71.9	75.3	54.1	44.1	0.0



1.4 Task 4

[4]:
```
priorities = [0.4, 0.2, 1.1, 0.1]
permutation = np.argsort(priorities)[::-1]
print(priorities, permutation)
```

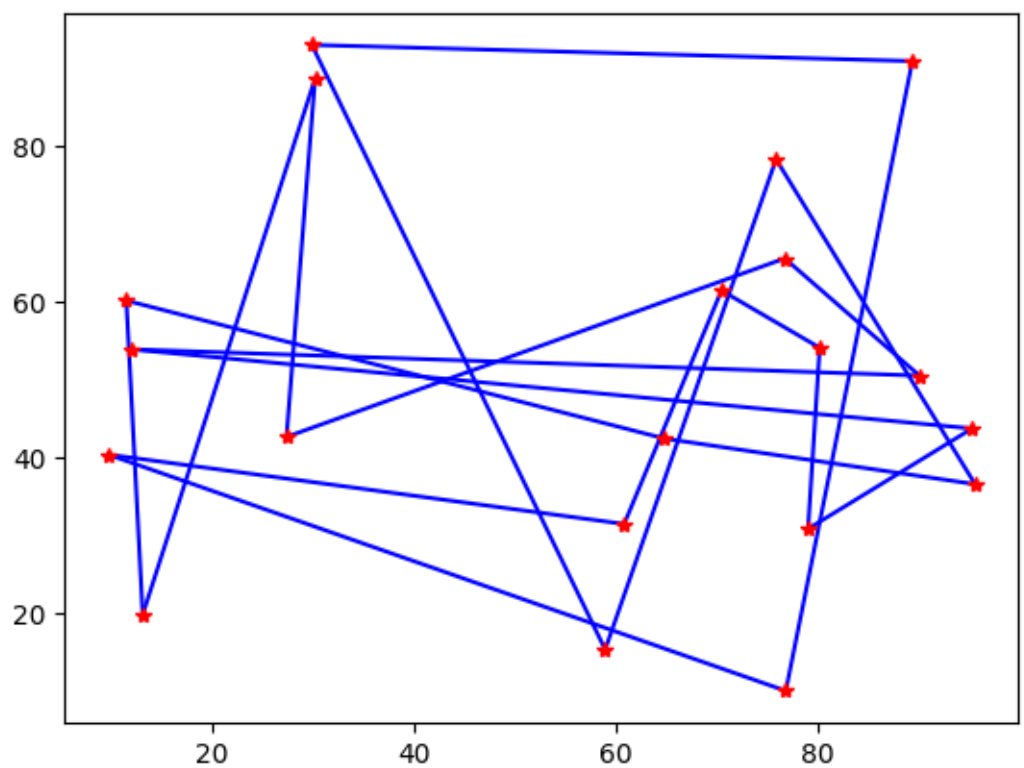
```
[0.4, 0.2, 1.1, 0.1] [2 0 1 3]
```

1.5 Task 5

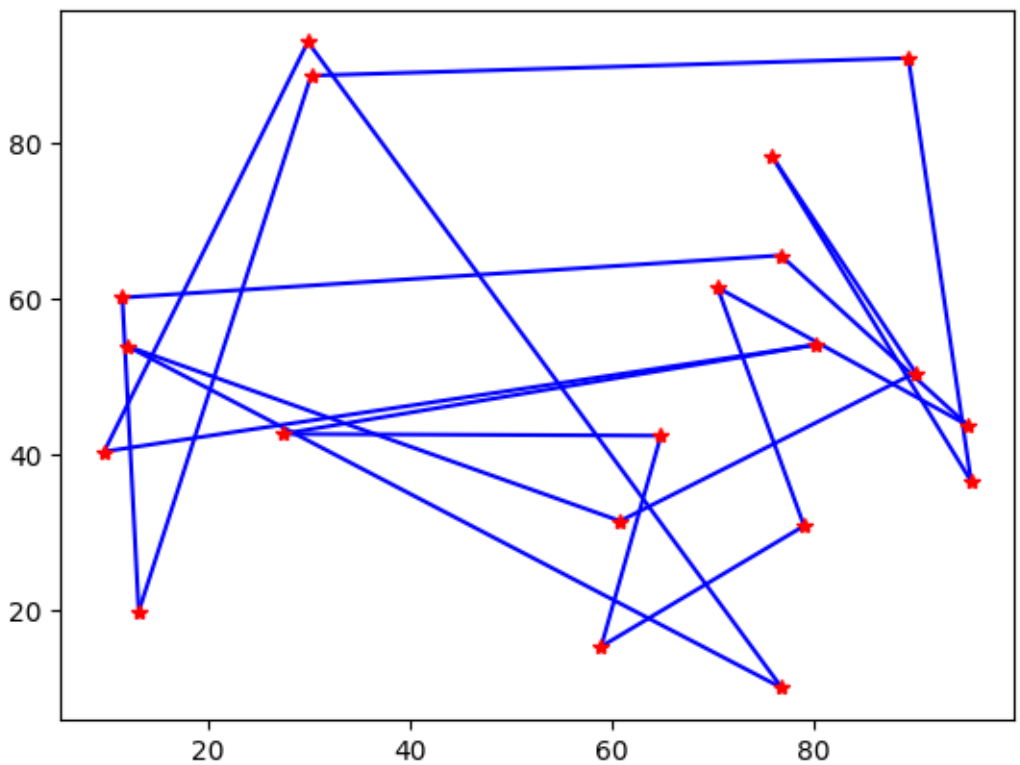
[5]:
```
from math import inf

# Plotting tours
def plot_tour(perm):
    for city in range(N-1):
        plt.plot([xy[perm[city]][0],xy[perm[city+1]][0]],
                 [xy[perm[city]][1],xy[perm[city+1]][1]],'b-')
    plt.plot([xy[perm[0]][0],xy[perm[city+1]][0]],
```

```
                [xy[perm[0]][1],xy[perm[city+1]][1]],'b-')
    plt.plot(xy[:,0],xy[:,1],'r*')
    plt.show()

best_perm = None
best_distance = inf

for _ in range(1_000_000):
    priorities = np.random.rand(N)
    perm = np.argsort(priorities)[::-1]
    distance = tsp_simulator(perm)
    if distance < best_distance:
        best_distance = distance
        best_perm = perm
        print(best_distance)
        plot_tour(best_perm)

permuted_d = d[best_perm][:,best_perm]
visualise(permuted_d)
```

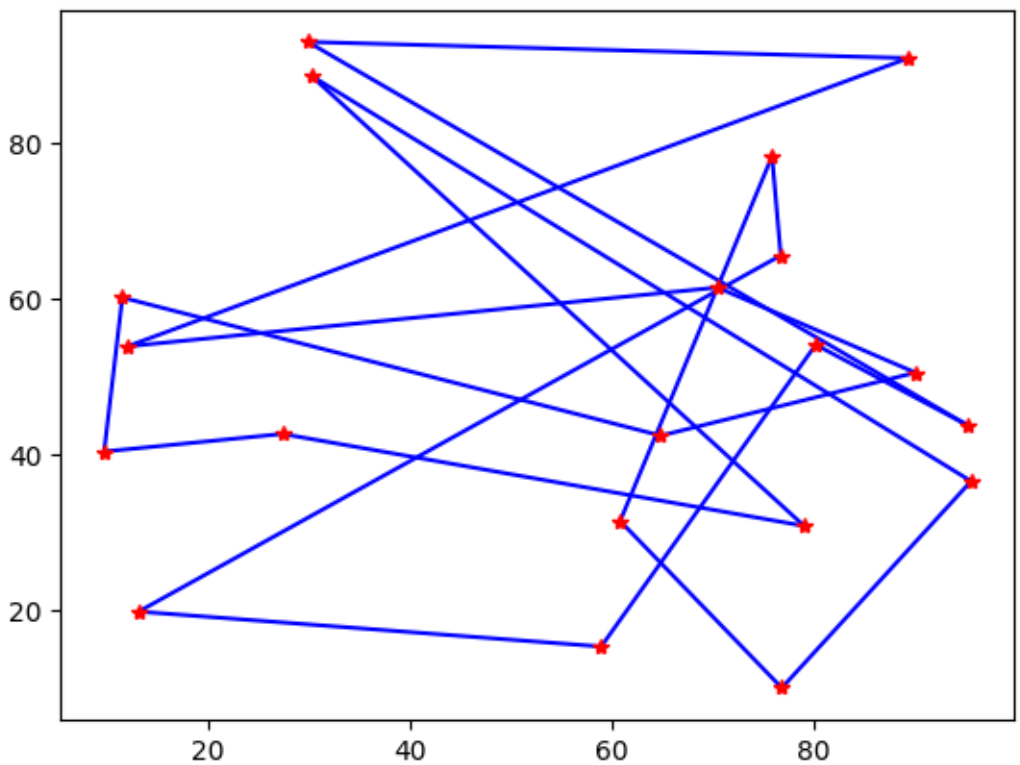
```
1030.010004608764
```



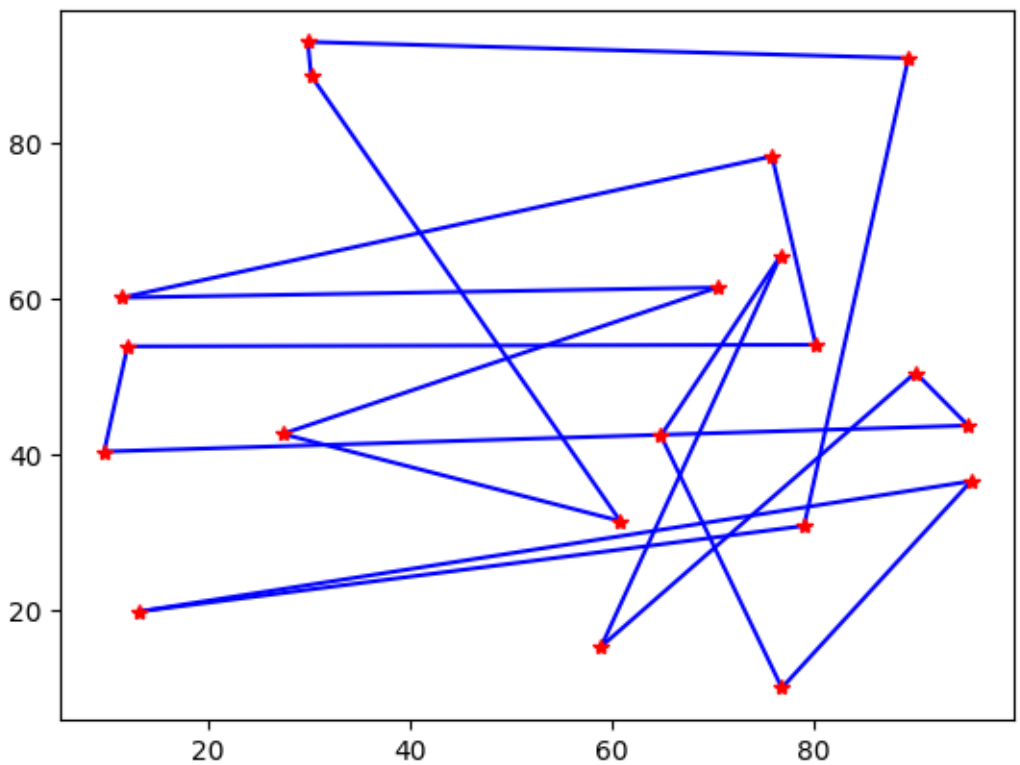

```
993.6273957836371
```



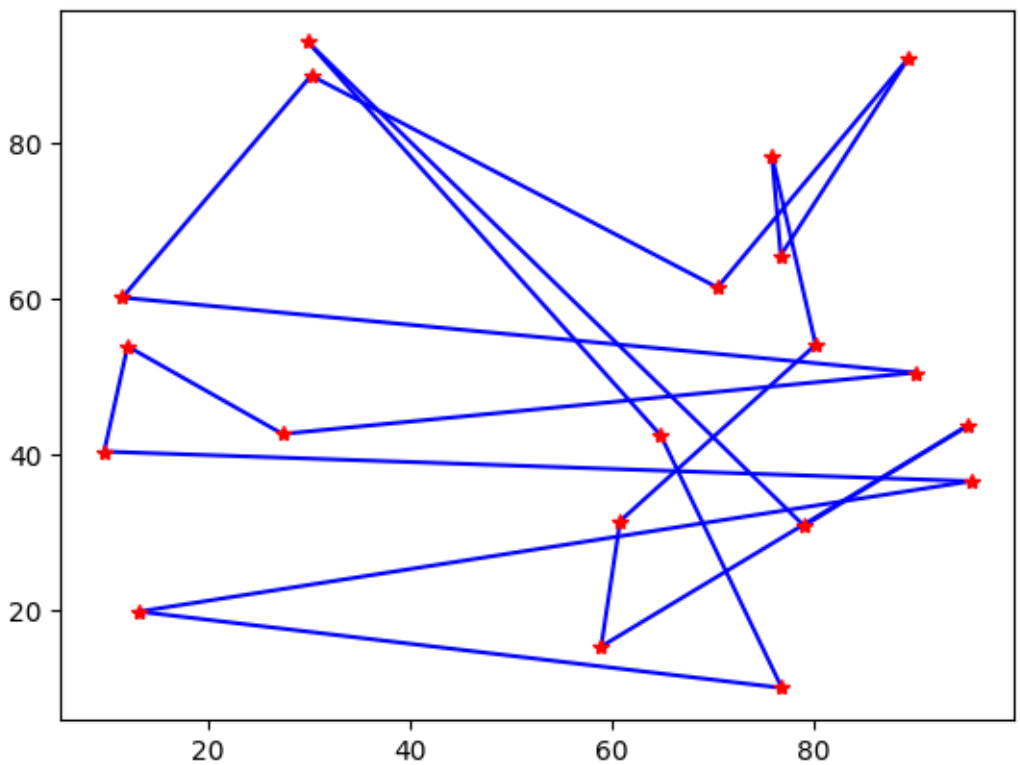
949.2756676351923



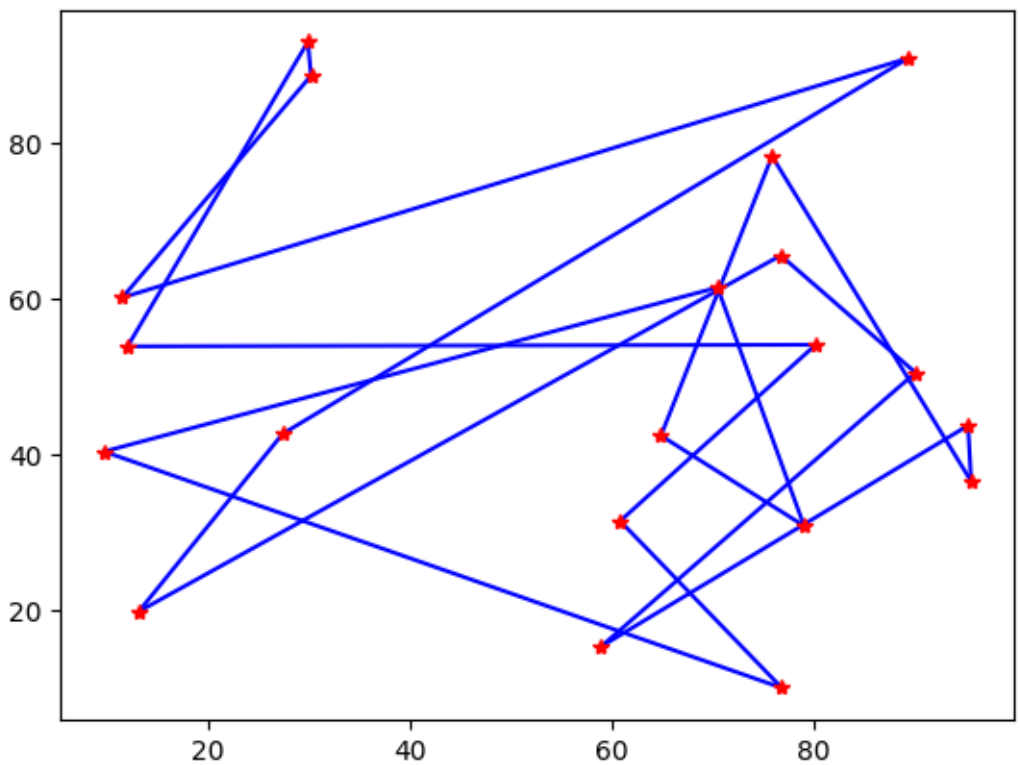
942.7370336094652



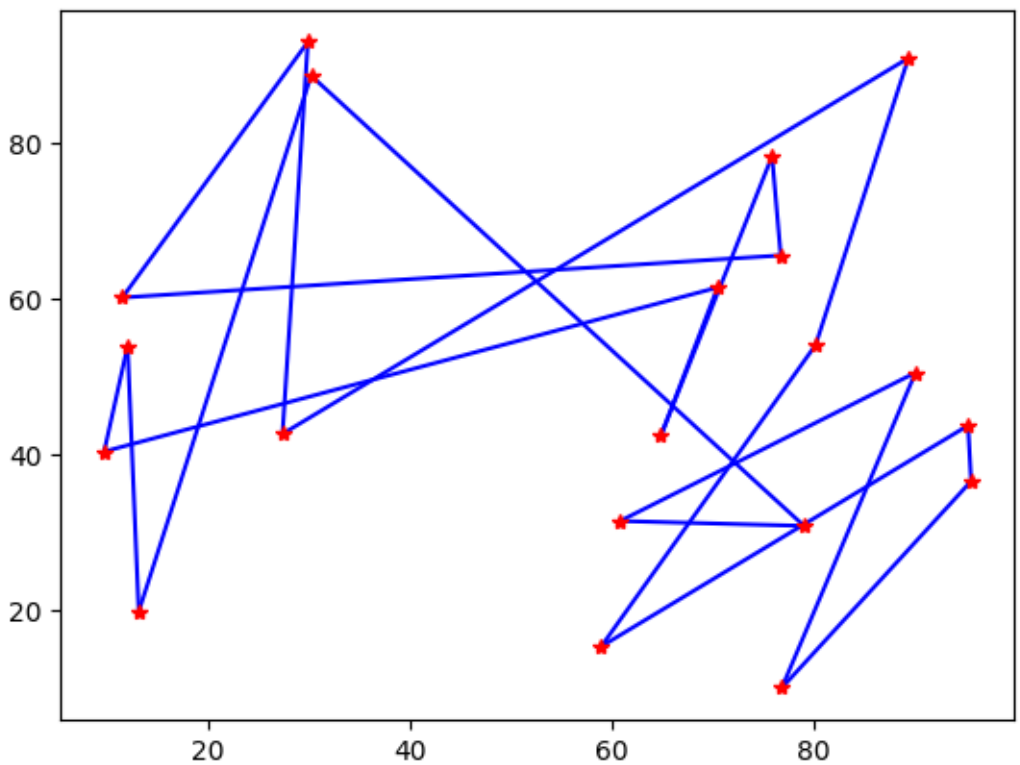
881.0719646971872



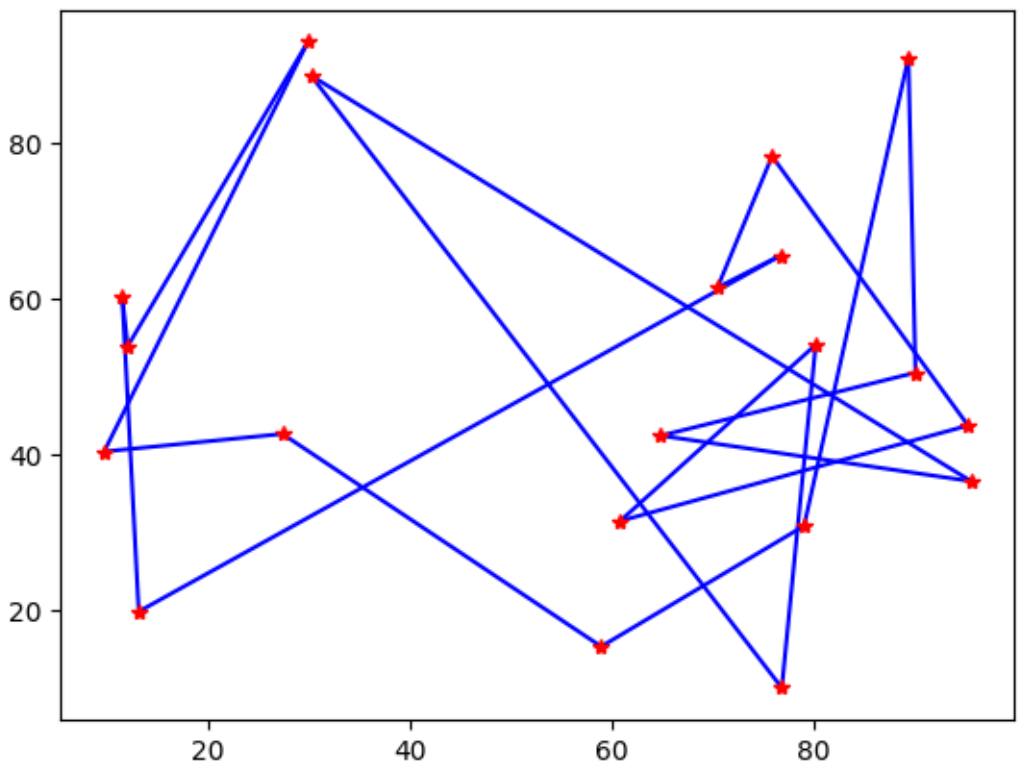
865.6168182071574



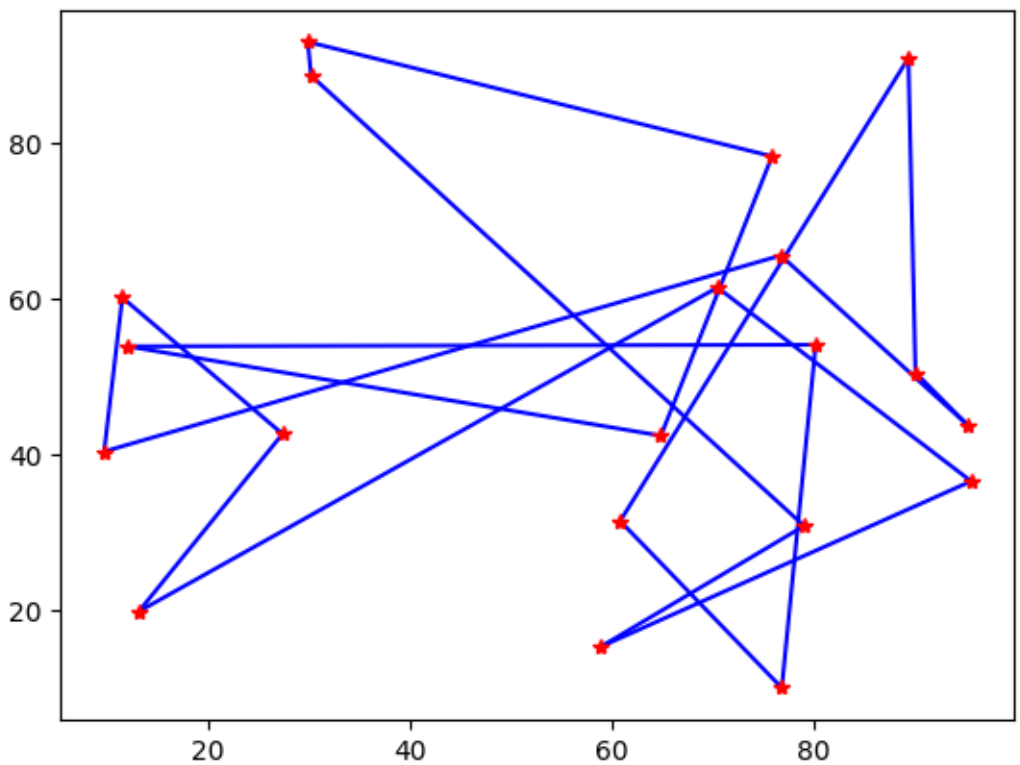
824.0803131899927



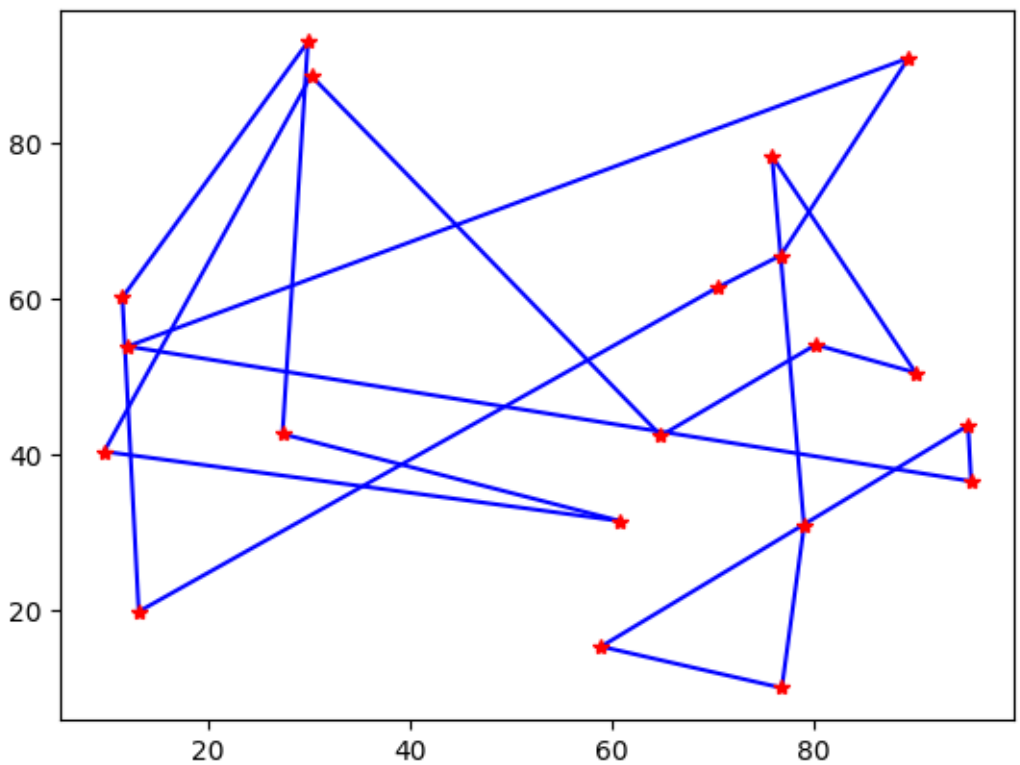
818.3692383011058



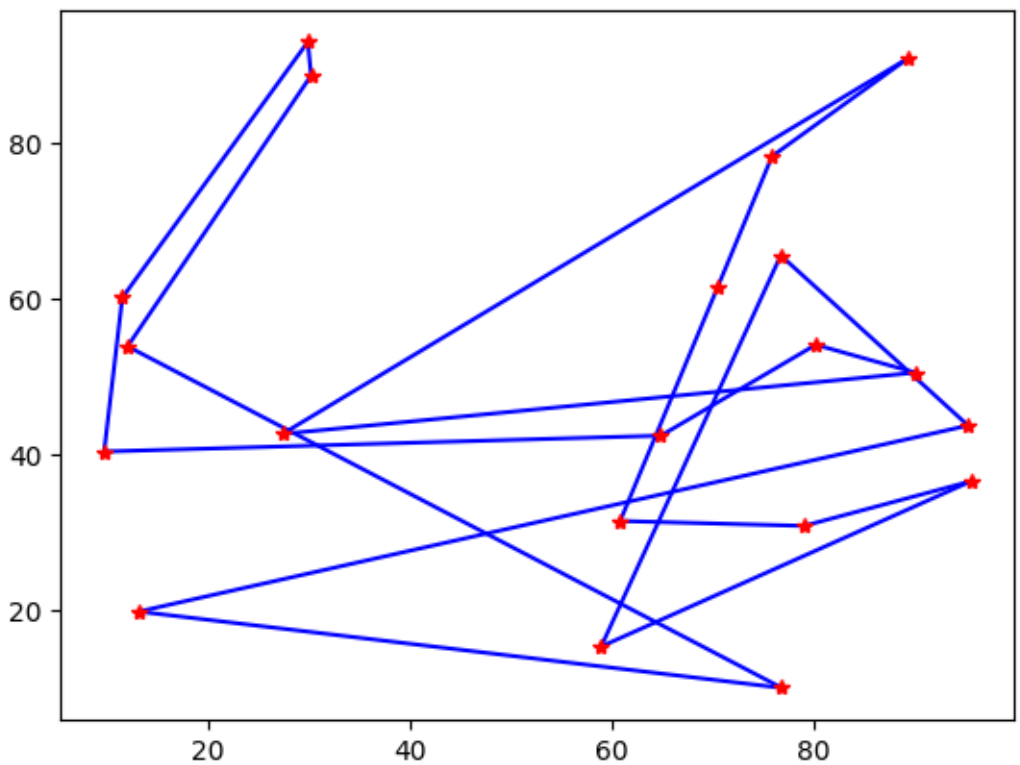

```
818.2722828370638
```



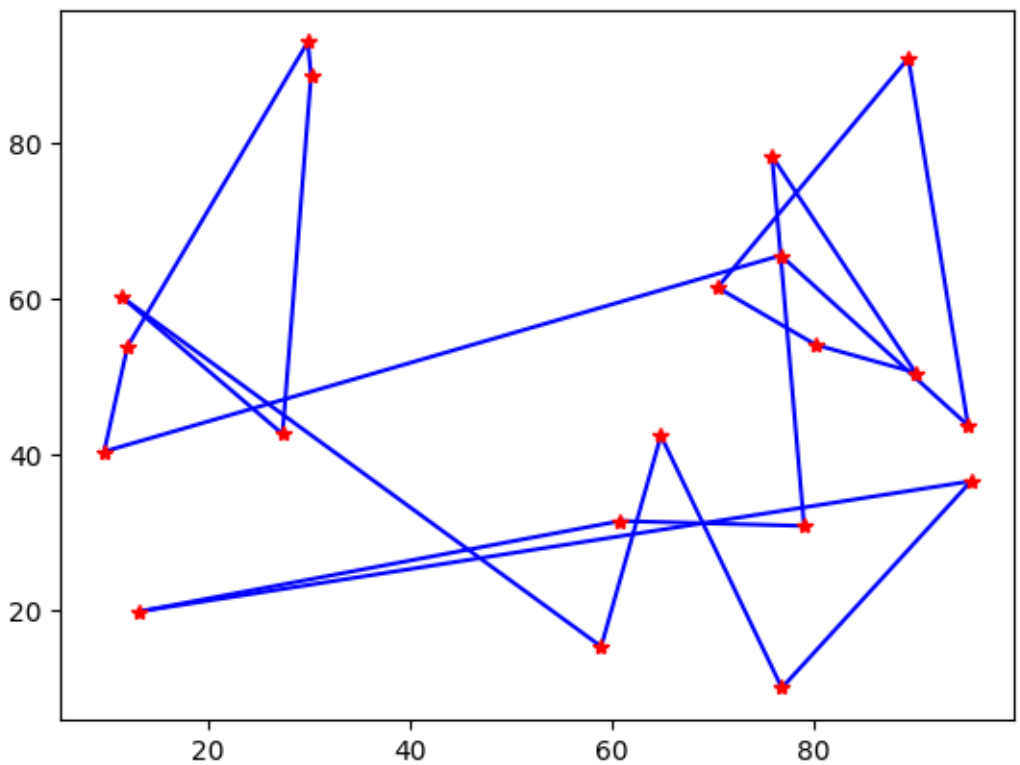
804.7014427111337



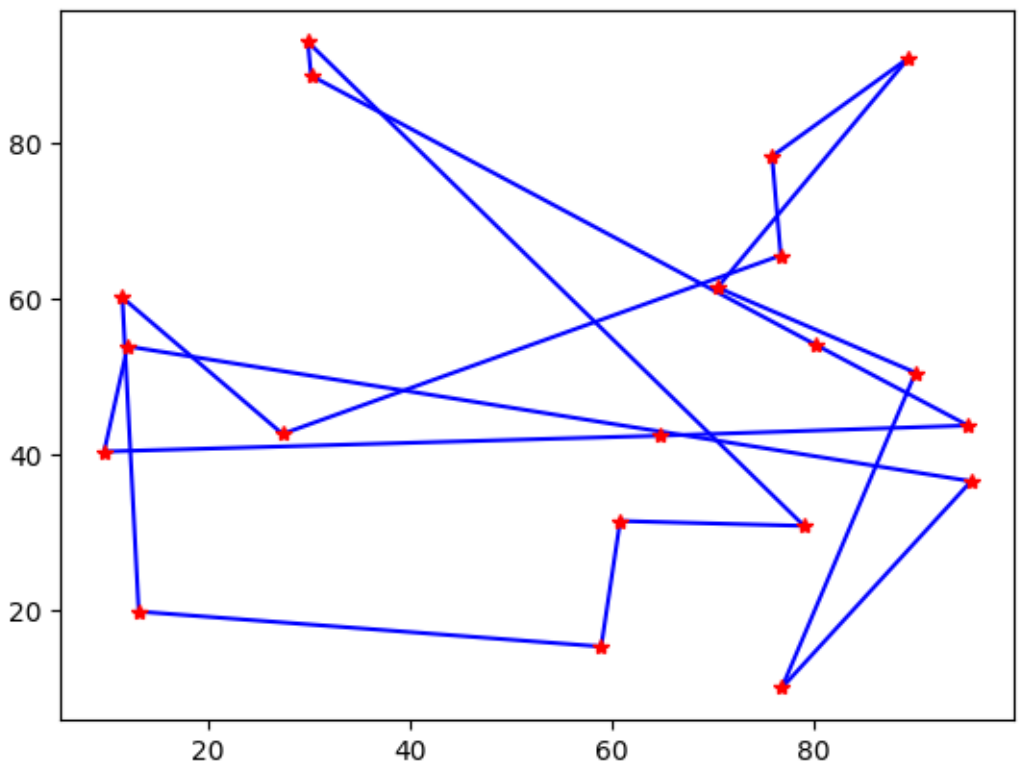
783.7418270614975



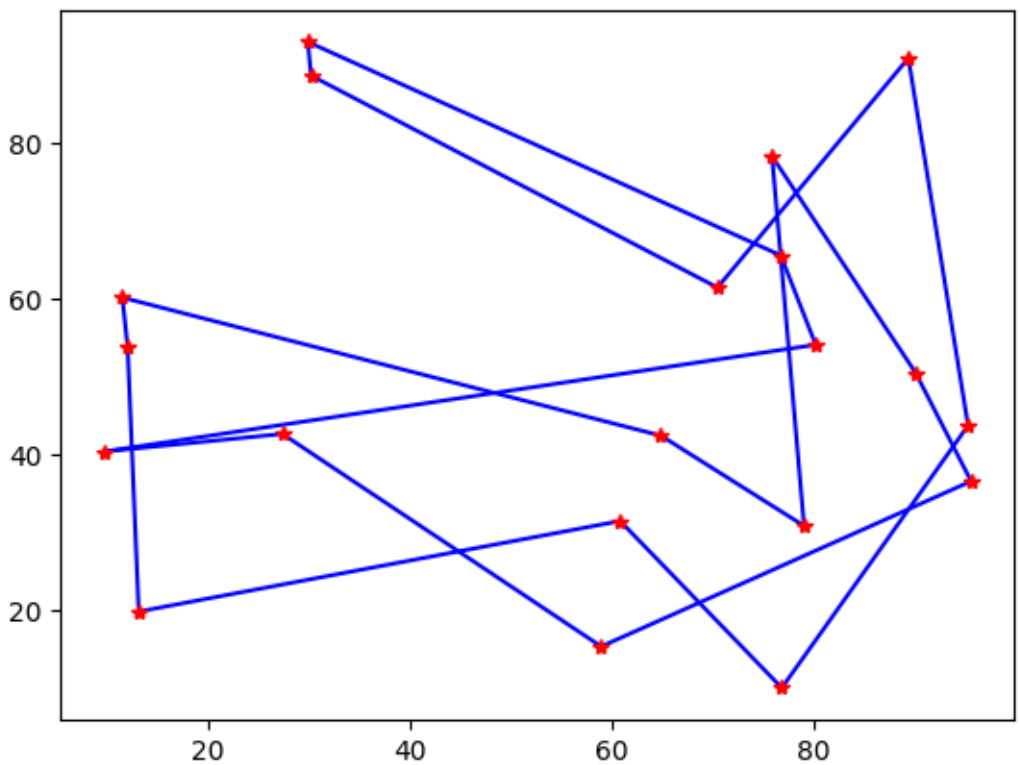
726.2408465592293



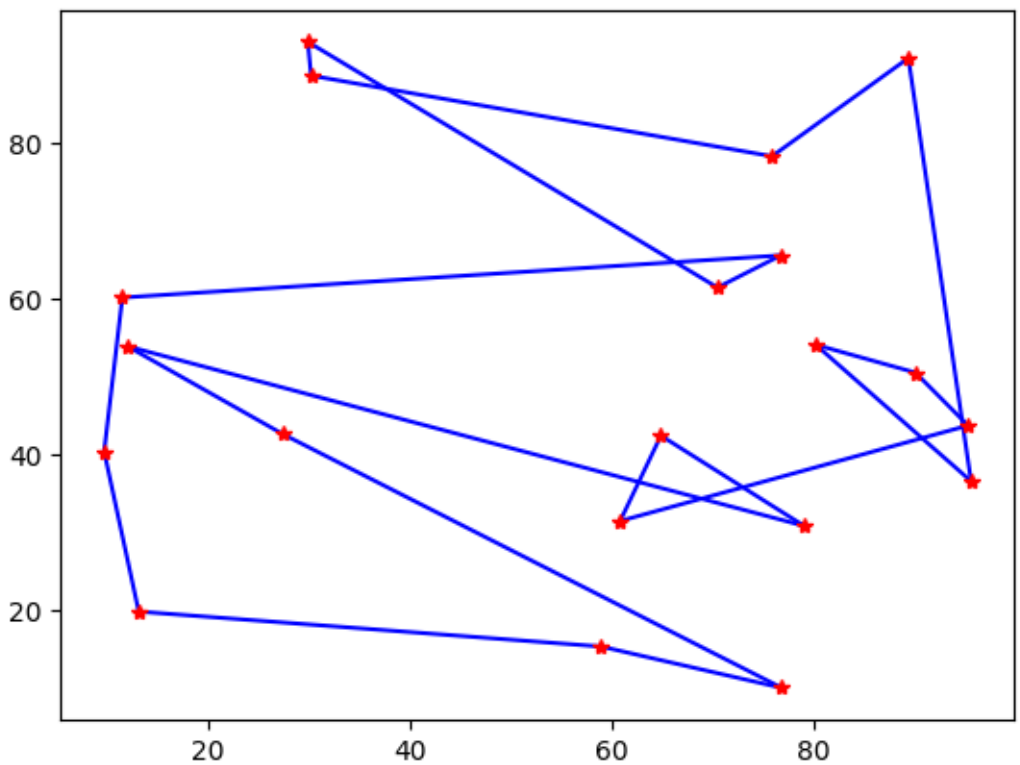
709.9596996916455



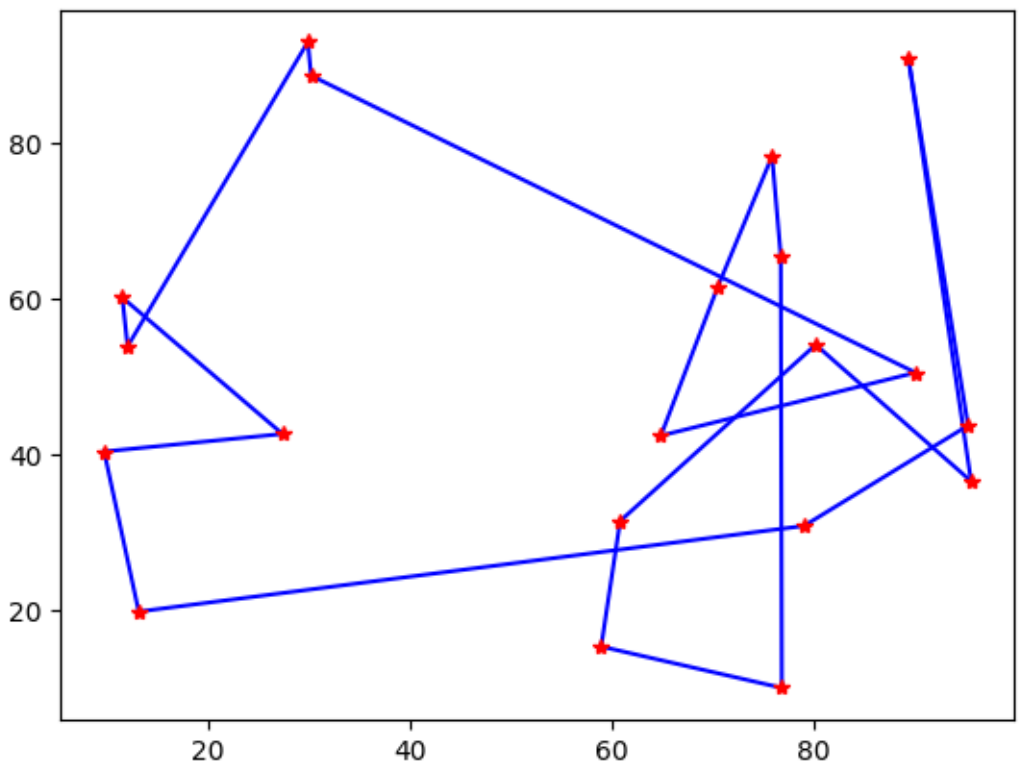
698.2066463883692



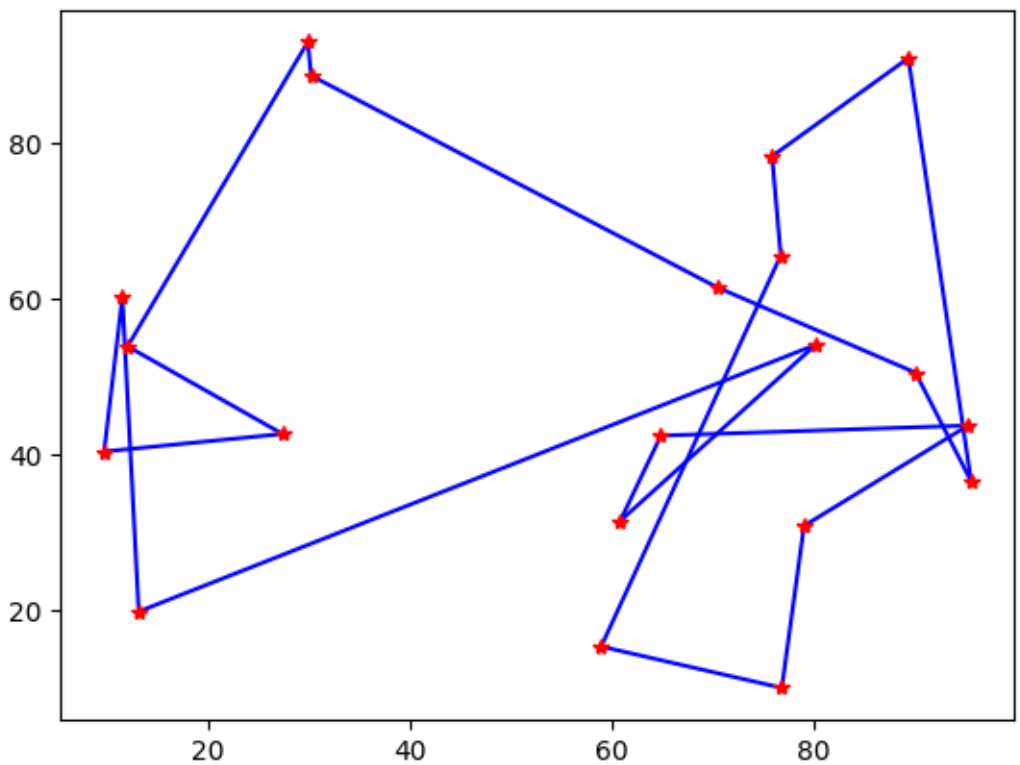
612.1745417910771



596.7029150007868

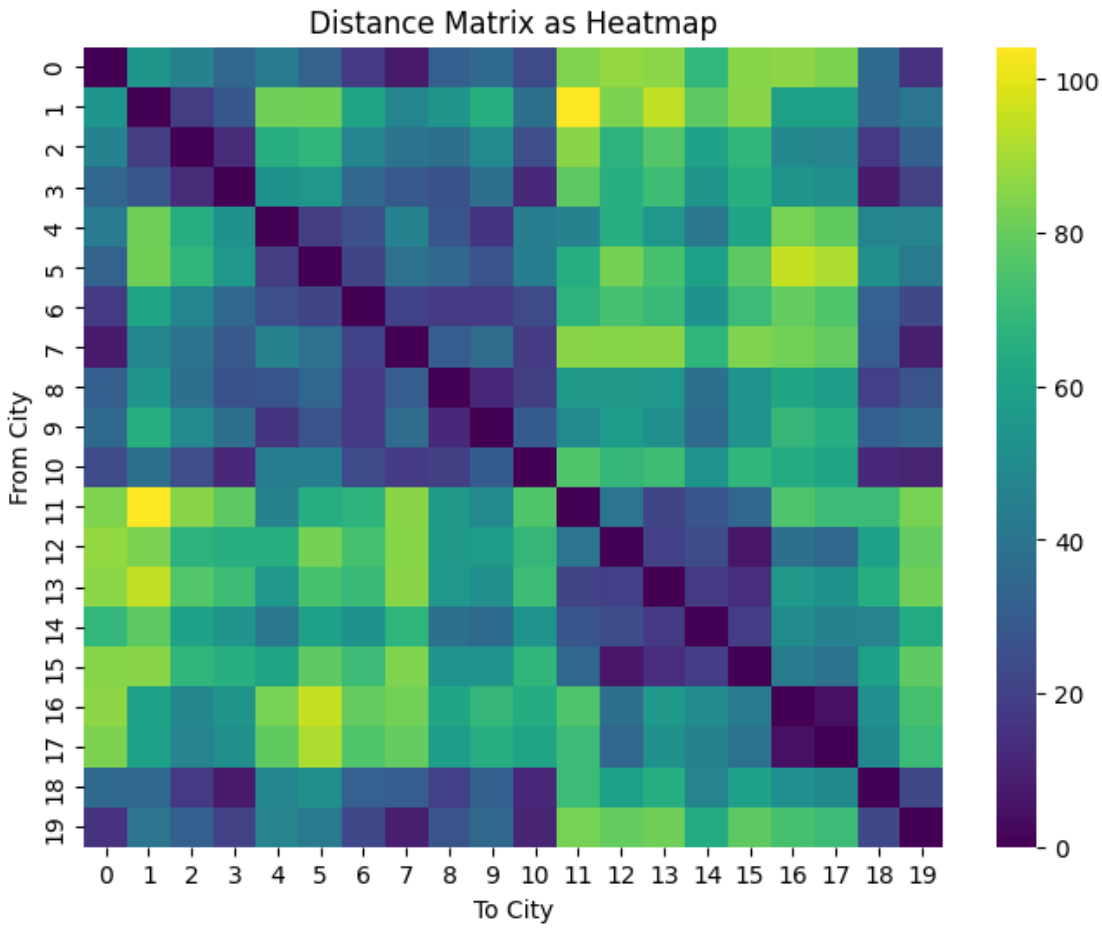


576.9291753229494



	0	1	2	3	4	5	6	7	8	9	10	11	\
0	0.0	54.6	46.1	34.6	42.4	32.5	17.5	7.1	31.3	35.2	23.3	84.2	
1	54.6	0.0	18.5	28.3	81.4	81.7	60.9	47.5	54.3	65.9	37.9	104.2	
2	46.1	18.5	0.0	12.7	65.1	68.2	47.5	39.6	37.5	49.1	24.6	85.7	
3	34.6	28.3	12.7	0.0	53.2	55.5	34.8	28.6	26.0	37.6	12.0	78.3	
4	42.4	81.4	65.1	53.2	0.0	18.7	25.4	46.1	27.7	16.2	44.2	46.1	
5	32.5	81.7	68.2	55.5	18.7	0.0	20.9	38.4	34.5	26.7	44.1	64.5	
6	17.5	60.9	47.5	34.8	25.4	20.9	0.0	20.7	18.3	18.2	23.3	66.9	
7	7.1	47.5	39.6	28.6	46.1	38.4	20.7	0.0	30.5	36.6	18.3	85.6	
8	31.3	54.3	37.5	26.0	27.7	34.5	18.3	30.5	0.0	11.7	19.3	56.5	
9	35.2	65.9	49.1	37.6	16.2	26.7	18.2	36.6	11.7	0.0	29.8	49.1	
10	23.3	37.9	24.6	12.0	44.2	44.1	23.3	18.3	19.3	29.8	0.0	75.3	
11	84.2	104.2	85.7	78.3	46.1	64.5	66.9	85.6	56.5	49.1	75.3	0.0	
12	87.4	83.7	66.9	65.5	65.2	82.3	73.6	85.4	56.2	57.1	69.0	40.3	
13	86.0	94.3	76.3	71.6	55.3	73.7	70.0	85.7	55.2	51.9	71.9	20.8	
14	68.5	78.6	60.2	54.4	41.7	59.2	53.0	68.0	37.5	35.3	54.1	26.9	
15	85.4	85.8	68.4	65.8	60.7	78.3	70.9	83.9	54.1	53.7	68.2	34.0	
16	86.6	59.6	48.3	54.3	82.8	95.2	79.2	81.9	61.4	68.8	63.6	75.0	
17	83.6	59.3	46.9	52.0	78.7	91.3	75.7	79.1	57.8	64.8	60.8	70.8	
18	35.3	35.0	17.7	7.5	47.5	51.7	31.8	30.5	19.8	31.5	12.2	70.8	
19	15.0	40.3	31.2	20.1	47.0	42.5	22.5	8.6	26.5	34.9	10.5	82.9	

	12	13	14	15	16	17	18	19
0	87.4	86.0	68.5	85.4	86.6	83.6	35.3	15.0
1	83.7	94.3	78.6	85.8	59.6	59.3	35.0	40.3
2	66.9	76.3	60.2	68.4	48.3	46.9	17.7	31.2
3	65.5	71.6	54.4	65.8	54.3	52.0	7.5	20.1
4	65.2	55.3	41.7	60.7	82.8	78.7	47.5	47.0
5	82.3	73.7	59.2	78.3	95.2	91.3	51.7	42.5
6	73.6	70.0	53.0	70.9	79.2	75.7	31.8	22.5
7	85.4	85.7	68.0	83.9	81.9	79.1	30.5	8.6
8	56.2	55.2	37.5	54.1	61.4	57.8	19.8	26.5
9	57.1	51.9	35.3	53.7	68.8	64.8	31.5	34.9
10	69.0	71.9	54.1	68.2	63.6	60.8	12.2	10.5
11	40.3	20.8	26.9	34.0	75.0	70.8	70.8	82.9
12	0.0	19.8	23.6	6.3	37.6	34.1	59.0	79.2
13	19.8	0.0	17.8	13.7	56.3	52.4	64.3	81.1
14	23.6	17.8	0.0	19.0	50.3	46.0	47.0	63.2
15	6.3	13.7	19.0	0.0	42.9	39.2	59.0	78.2
16	37.6	56.3	50.3	42.9	0.0	4.3	51.4	73.7
17	34.1	52.4	46.0	39.2	4.3	0.0	48.6	71.0
18	59.0	64.3	47.0	59.0	51.4	48.6	0.0	22.5
19	79.2	81.1	63.2	78.2	73.7	71.0	22.5	0.0



1.6 Task 6 & 7

[6]:
```
# ORIGINAL VERSION OF BINARY GA FROM MOODLE
from random import randint
from random import sample
from random import random

# Initialize Global Parameters
iterations = 500
N = 20          # Genome length
M = 50          # Population size
crossover_prob = 0.5
mutation_rate = 1 / N

def fitness_function(genome):
    return sum(genome) # Fitness is simply the count of 1s.

def random_individual(N):
```

```
    return [randint(0, 1) for _ in range(N)]

def random_population(M, N):
    return [random_individual(N) for _ in range(M)]

def eval_fitness(population, fitness_func):
    return [fitness_func(ind) for ind in population]

def tournament():
    i, j = sample(range(M), 2)
    return i if fitnesses[i] >= fitnesses[j] else j

def negative_tournament():
    i, j = sample(range(M), 2)
    # Returns the index of the weaker individual to be replaced
    return i if fitnesses[i] <= fitnesses[j] else j

def mutate():
    idx = tournament()
    parent = population[idx]
    # Create offspring by flipping bits
    offspring = [(1 - bit) if random() < mutation_rate else bit for bit in␣
 ↪parent]
    return offspring

def crossover():
    idx1 = tournament()
    idx2 = tournament()
    parent1, parent2 = population[idx1], population[idx2]

    # One-point crossover
    point = randint(1, N - 1)
    offspring = parent1[:point] + parent2[point:]
    return offspring

population = random_population(M, N)
fitnesses = eval_fitness(population, fitness_function)

# Track the best individual
best_idx = fitnesses.index(max(fitnesses))
best_genome = population[best_idx][:]
best_fit = fitnesses[best_idx]

for i in range(iterations):
    if random() < crossover_prob:
        offspring = crossover()
    else:
```

```
        offspring = mutate()

    f = fitness_function(offspring)

    k = negative_tournament()
    population[k] = offspring
    fitnesses[k] = f

    if f > best_fit:
        best_fit = f
        best_genome = offspring[:]
        print(best_fit, best_genome)
```

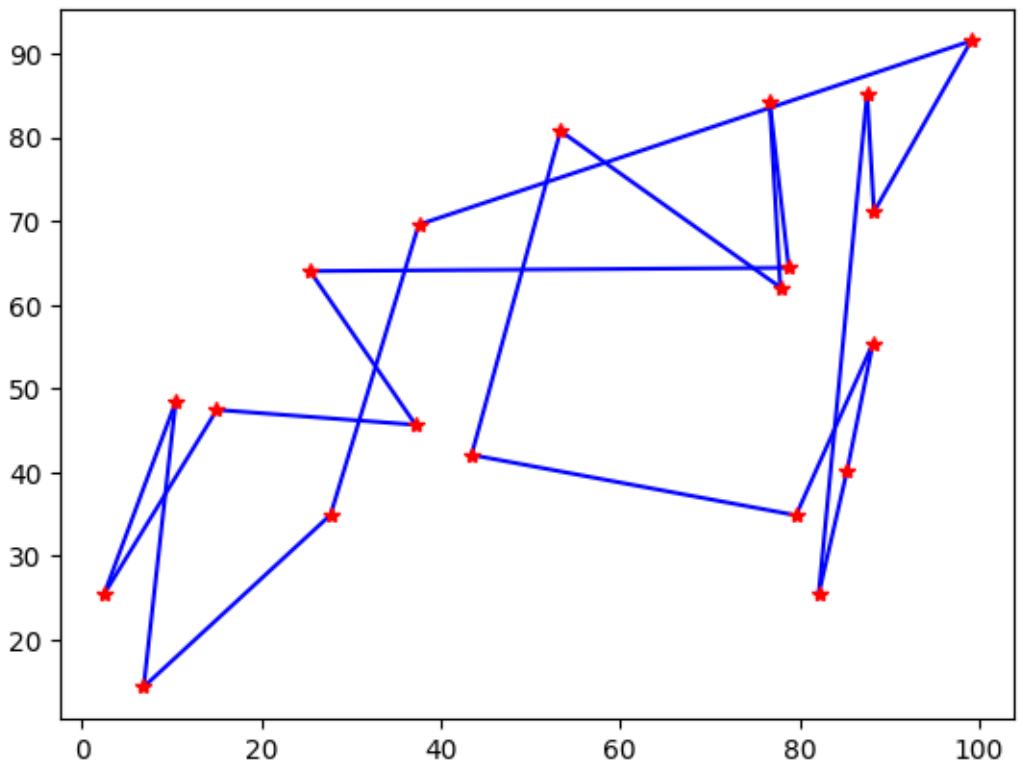
```
18 [1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
19 [1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
20 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

[7]:
```
# MODIFIED VERSION
from random import randint
from random import sample
from random import random

# Creating the cities
N = 20
xy = 100 * np.random.rand(N,2)

# Computing distance matrix
d = np.empty((N,N))
for i in range(N):
    for j in range(N):
        # Calculate Euclidean distance
        dx = xy[i,0] - xy[j,0]
        dy = xy[i,1] - xy[j,1]
        d[i,j] = np.sqrt(dx**2 + dy**2)

# Initialize Global Parameters
iterations = 1_000_000
# N = Genome length is determined by cities
M = 1000          # Population size
crossover_prob = 0.5
mutation_rate = 1 / N

def fitness_function(priorities):
    permutation = np.argsort(priorities)[::-1]
    distance = tsp_simulator(permutation)
    fitness = 1.0 / distance # So the higher the distance the worse the fitness
    return fitness
```

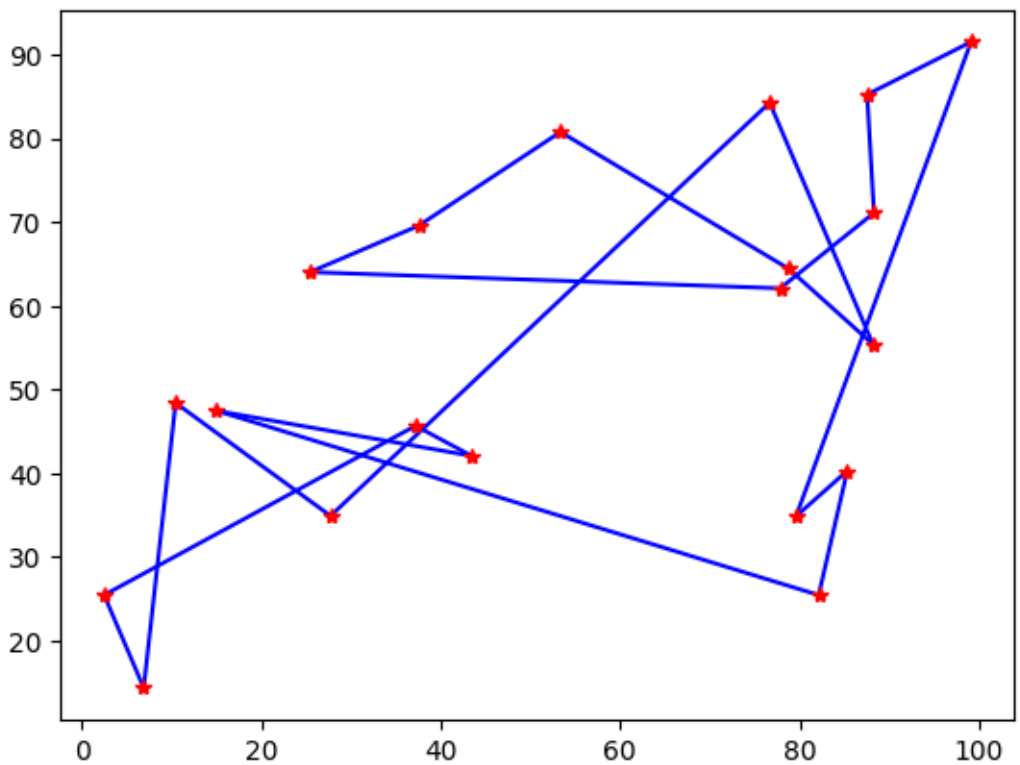
```python
def random_individual(N):
    return np.random.rand(N)

def random_population(M, N):
    return [random_individual(N) for _ in range(M)]

def eval_fitness(population):
    return [fitness_function(ind) for ind in population]

def tournament():
    i, j = sample(range(M), 2)
    return i if fitnesses[i] >= fitnesses[j] else j

def negative_tournament():
    i, j = sample(range(M), 2)
    # Returns the index of the weaker individual to be replaced
    return i if fitnesses[i] <= fitnesses[j] else j

def mutate():
    idx = tournament()
    parent = population[idx]
    return parent

def crossover():
    idx1 = tournament()
    idx2 = tournament()
    parent1, parent2 = population[idx1], population[idx2]

    # One-point crossover
    point = randint(1, N - 1)
    offspring = np.concatenate((parent1[:point],parent2[point:]))
    return offspring

population = random_population(M, N)
fitnesses = eval_fitness(population)

# Track the best individual
best_idx = fitnesses.index(max(fitnesses))
best_genome = np.argsort(population[best_idx])[::-1]
best_fit = fitnesses[best_idx]
for i in range(iterations):
    if random() < crossover_prob:
        offspring = crossover()
    else:
        offspring = mutate()
```

```
    f = fitness_function(offspring)

    k = negative_tournament()
    population[k] = offspring
    fitnesses[k] = f

    if f > best_fit:
        best_fit = f
        best_genome = np.argsort(offspring)[::-1]
        print(1/best_fit, best_genome)
        plot_tour(best_genome)

permuted_d = d[best_genome][:,best_genome]
visualise(permuted_d)
```

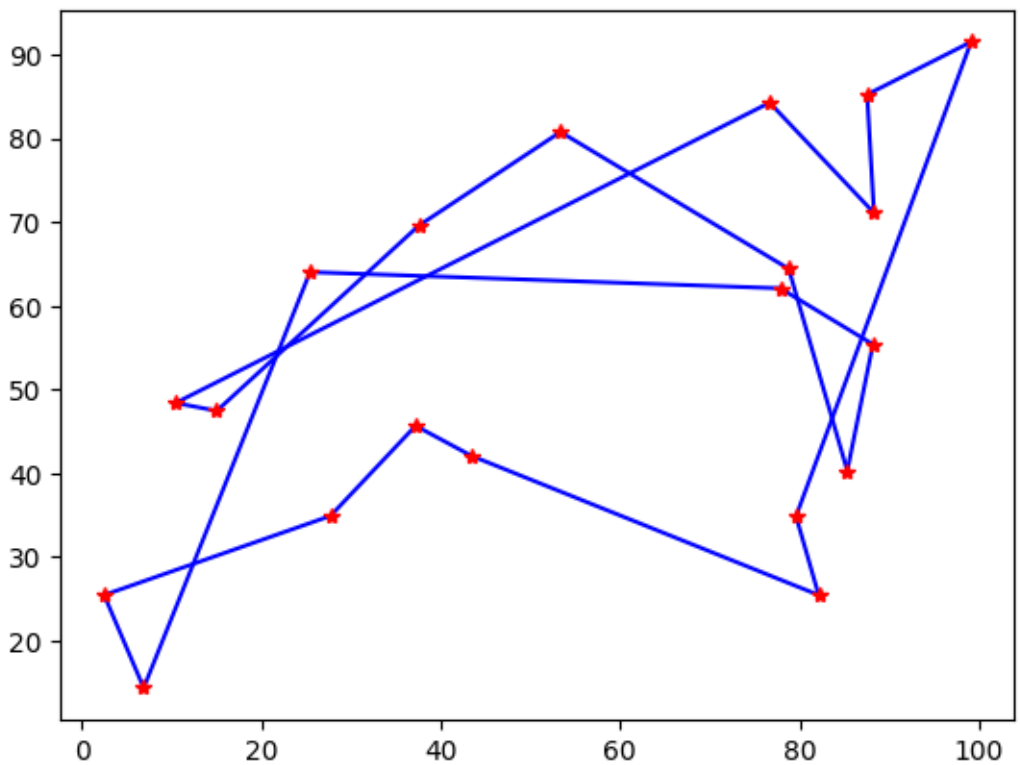
```
611.5394548962068 [ 6  4  1 14  8 18  2 15  5 13 12 19 17 11  0 10  3 16  7  9]
```



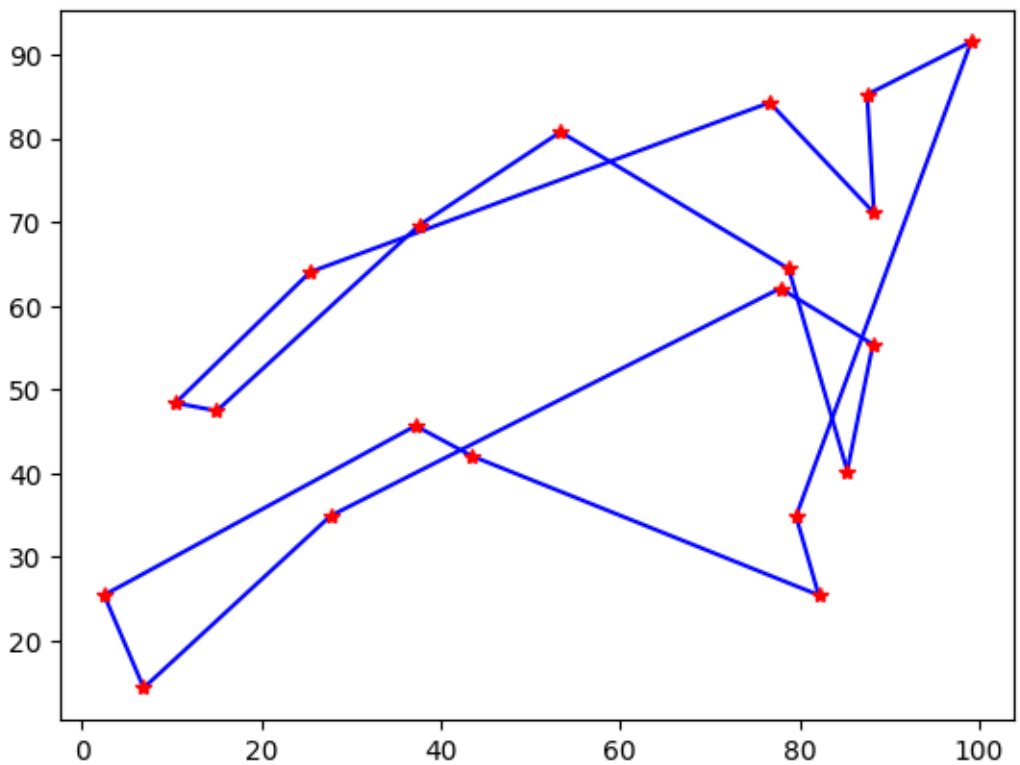

```
567.8178289301885 [11 12  3 19 14 18  8  2  5  0 15  7 16 10  4  9  6 17 13  1]
```



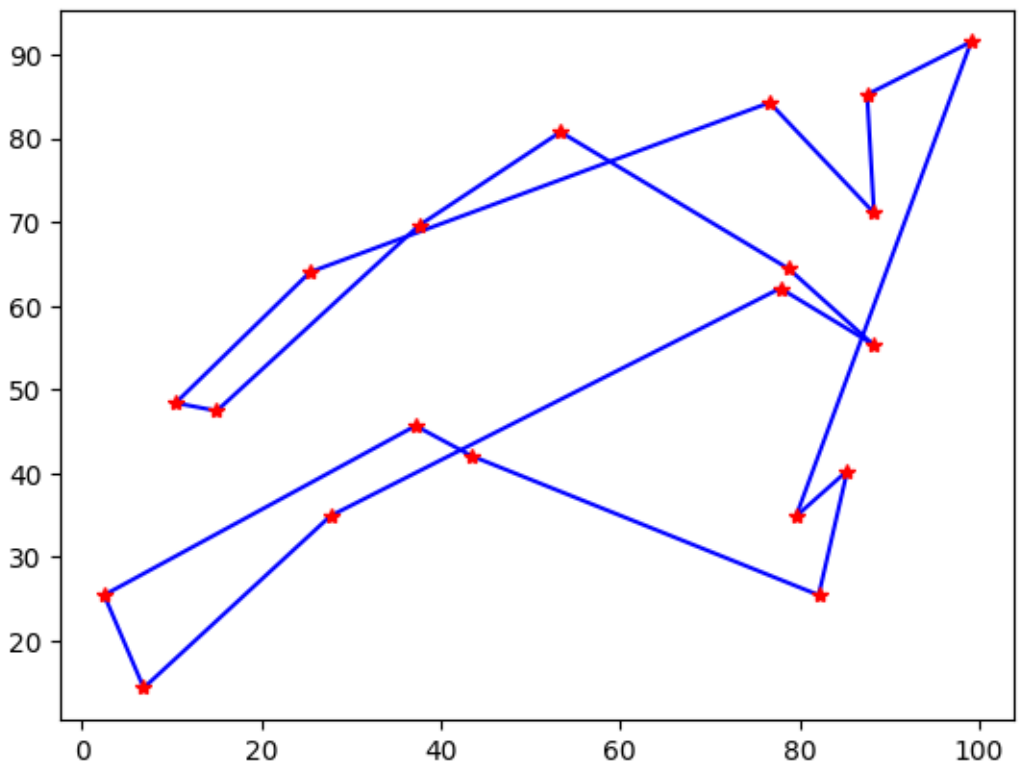

```
536.1800772204921 [11 12 16  3 17 13  8  2 14  5  0  7 10  4  9  6 19 18 15  1]
```




```
517.3174565916867 [11 12 16  3 17 14  8  2  5  0  7 10  4  9  6 19 13 18 15  1]
```

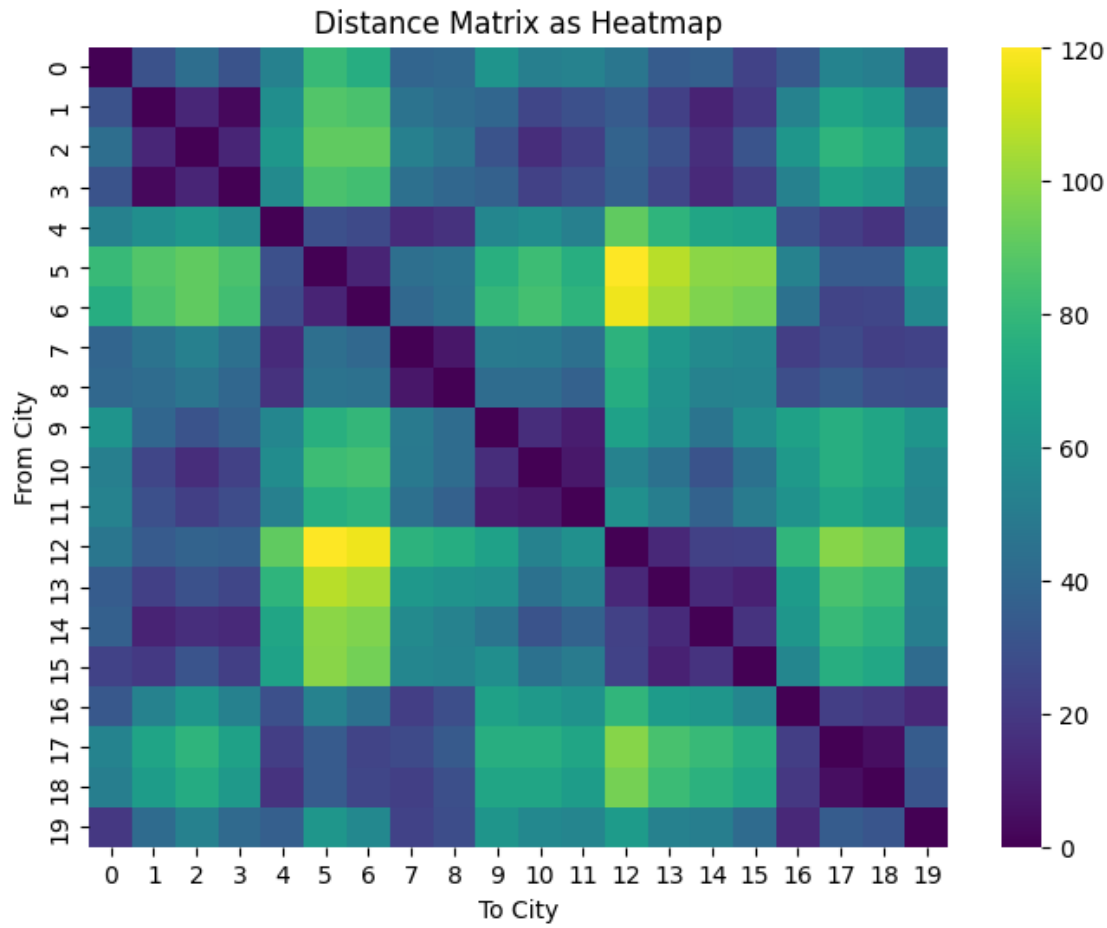



```
503.2869135919359 [11 12  3 17 14  8  2  5  0  7 16 10  4  9  6 19 13 18 15  1]
```



	0	1	2	3	4	5	6	7	8	9	10	11	\
0	0.0	30.2	43.1	30.8	52.5	81.0	75.1	38.6	39.9	62.3	51.6	52.9	
1	30.2	0.0	13.0	2.5	59.0	87.6	85.6	45.6	41.8	39.1	25.0	29.6	
2	43.1	13.0	0.0	12.3	63.8	91.0	90.7	51.9	46.7	30.5	15.4	22.2	
3	30.8	2.5	12.3	0.0	57.0	85.5	83.8	43.8	39.8	36.9	23.0	27.3	
4	52.5	59.0	63.8	57.0	0.0	29.2	26.9	14.3	17.3	55.2	57.8	51.9	
5	81.0	87.6	91.0	85.5	29.2	0.0	11.9	43.6	45.8	76.0	82.6	75.5	
6	75.1	85.6	90.7	83.8	26.9	11.9	0.0	40.2	44.2	79.6	84.1	77.7	
7	38.6	45.6	51.9	43.8	14.3	43.6	40.2	0.0	7.2	49.3	48.4	43.7	
8	39.9	41.8	46.7	39.8	17.3	45.8	44.2	7.2	0.0	42.1	41.9	36.9	
9	62.3	39.1	30.5	36.9	55.2	76.0	79.6	49.3	42.1	0.0	15.2	9.7	
10	51.6	25.0	15.4	23.0	57.8	82.6	84.1	48.4	41.9	15.2	0.0	7.9	
11	52.9	29.6	22.2	27.3	51.9	75.5	77.7	43.7	36.9	9.7	7.9	0.0	
12	47.0	33.9	37.7	36.3	91.1	120.2	117.0	77.0	74.4	68.2	53.0	59.9	
13	34.5	22.6	29.8	25.1	78.1	107.3	103.9	64.0	61.7	60.0	45.0	51.0	
14	36.2	11.6	15.7	13.8	70.5	99.2	97.2	57.1	53.4	46.0	30.9	37.3	
15	23.6	20.0	31.1	22.2	69.5	98.7	94.6	55.2	53.7	59.0	44.8	49.5	
16	32.5	53.3	63.3	52.4	29.2	53.0	44.9	21.8	28.4	68.6	64.4	61.5	
17	53.7	70.2	78.0	68.7	21.9	34.2	24.3	26.9	33.6	75.2	75.3	70.4	
18	50.7	65.9	73.5	64.5	17.8	34.1	25.3	22.2	28.9	70.6	70.6	65.8	
19	19.4	41.5	52.6	41.0	36.0	63.1	56.3	23.9	28.1	62.7	56.0	54.5	

	12	13	14	15	16	17	18	19
0	47.0	34.5	36.2	23.6	32.5	53.7	50.7	19.4
1	33.9	22.6	11.6	20.0	53.3	70.2	65.9	41.5
2	37.7	29.8	15.7	31.1	63.3	78.0	73.5	52.6
3	36.3	25.1	13.8	22.2	52.4	68.7	64.5	41.0
4	91.1	78.1	70.5	69.5	29.2	21.9	17.8	36.0
5	120.2	107.3	99.2	98.7	53.0	34.2	34.1	63.1
6	117.0	103.9	97.2	94.6	44.9	24.3	25.3	56.3
7	77.0	64.0	57.1	55.2	21.8	26.9	22.2	23.9
8	74.4	61.7	53.4	53.7	28.4	33.6	28.9	28.1
9	68.2	60.0	46.0	59.0	68.6	75.2	70.6	62.7
10	53.0	45.0	30.9	44.8	64.4	75.3	70.6	56.0
11	59.9	51.0	37.3	49.5	61.5	70.4	65.8	54.5
12	0.0	13.2	23.1	23.6	78.6	98.6	94.9	65.3
13	13.2	0.0	14.1	10.9	65.6	85.4	81.7	52.4
14	23.1	14.1	0.0	17.6	63.2	81.1	76.9	50.7
15	23.6	10.9	17.6	0.0	55.1	75.3	71.8	41.8
16	78.6	65.6	63.2	55.1	0.0	21.6	19.5	13.3
17	98.6	85.4	81.1	75.3	21.6	0.0	4.7	34.4
18	94.9	81.7	76.9	71.8	19.5	4.7	0.0	31.5
19	65.3	52.4	50.7	41.8	13.3	34.4	31.5	0.0



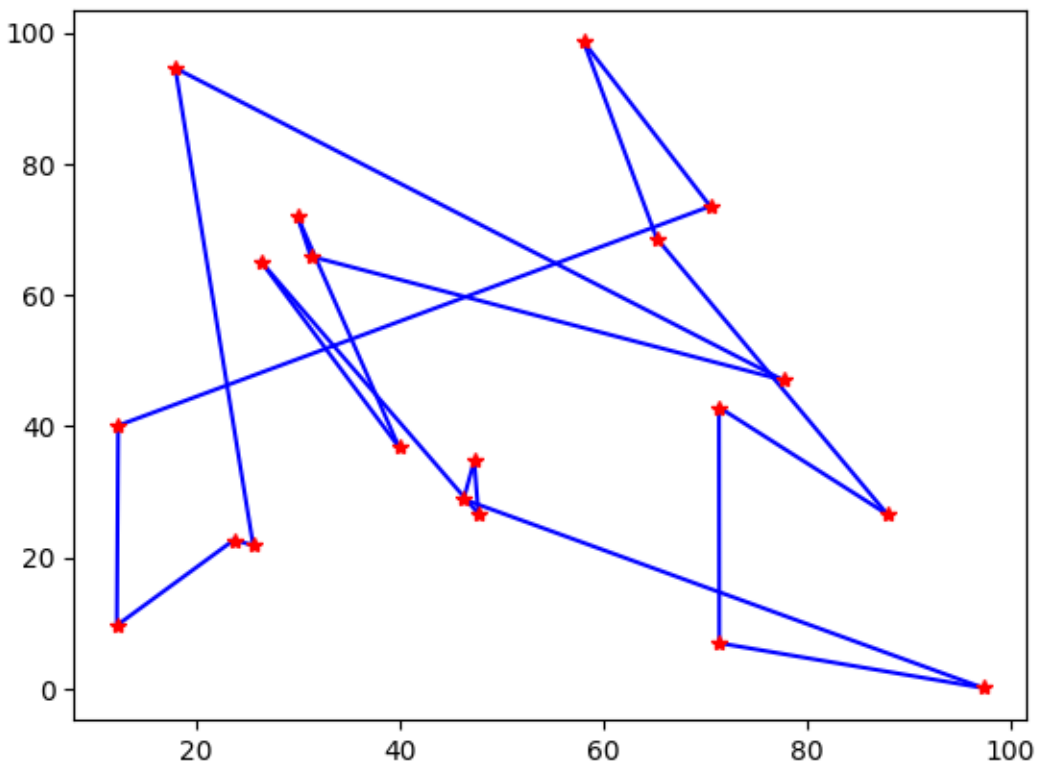
1.7 Task 8

[8]:
```
# MODIFIED MUTATION
from random import randint
from random import sample
from random import random

# Creating the cities
N = 20
xy = 100 * np.random.rand(N,2)

# Computing distance matrix
d = np.empty((N,N))
for i in range(N):
    for j in range(N):
        # Calculate Euclidean distance
        dx = xy[i,0] - xy[j,0]
        dy = xy[i,1] - xy[j,1]
```

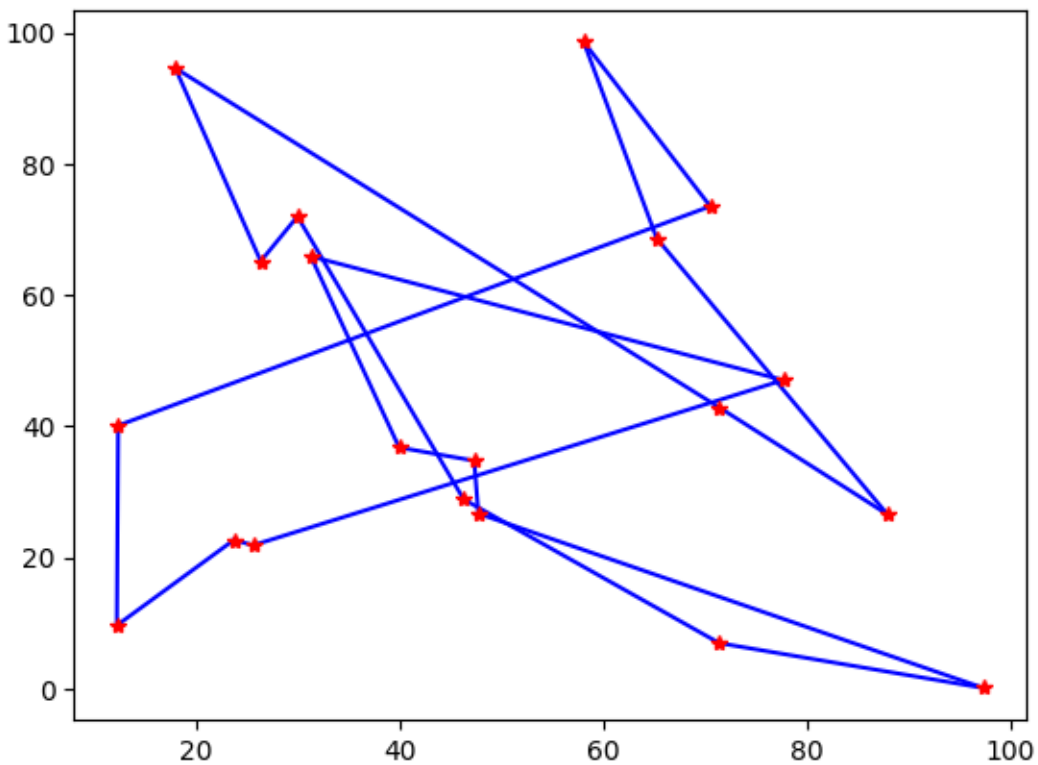
```
        d[i,j] = np.sqrt(dx**2 + dy**2)

# Initialize Global Parameters
iterations = 1_000_000
# N = Genome length is determined by cities
M = 1000         # Population size
crossover_prob = 0.5
mutation_rate = 1 / N

def fitness_function(priorities):
    permutation = np.argsort(priorities)[::-1]
    distance = tsp_simulator(permutation)
    fitness = 1.0 / distance # So the higher the distance the worse the fitness
    return fitness

def random_individual(N):
    return np.random.rand(N)

def random_population(M, N):
    return [random_individual(N) for _ in range(M)]

def eval_fitness(population):
    return [fitness_function(ind) for ind in population]

def tournament():
    i, j = sample(range(M), 2)
    return i if fitnesses[i] >= fitnesses[j] else j

def negative_tournament():
    i, j = sample(range(M), 2)
    # Returns the index of the weaker individual to be replaced
    return i if fitnesses[i] <= fitnesses[j] else j

def mutate():
    idx = tournament()
    parent = population[idx]
    offset = 0.05 * (np.random.rand(N) - 0.5)
    offspring = parent + offset
    return offspring

def crossover():
    idx1 = tournament()
    idx2 = tournament()
    parent1, parent2 = population[idx1], population[idx2]

    # One-point crossover
    point = randint(1, N - 1)
```

```
    offspring = np.concatenate((parent1[:point],parent2[point:]))
    return offspring

population = random_population(M, N)
fitnesses = eval_fitness(population)

# Track the best individual
best_idx = fitnesses.index(max(fitnesses))
best_genome = np.argsort(population[best_idx])[::-1]
best_fit = fitnesses[best_idx]
for i in range(iterations):
    if random() < crossover_prob:
        offspring = crossover()
    else:
        offspring = mutate()

    f = fitness_function(offspring)

    k = negative_tournament()
    population[k] = offspring
    fitnesses[k] = f

    if f > best_fit:
        best_fit = f
        best_genome = np.argsort(offspring)[::-1]
        print(1/best_fit, best_genome)
        plot_tour(best_genome)

permuted_d = d[best_genome][:,best_genome]
visualise(permuted_d)
```

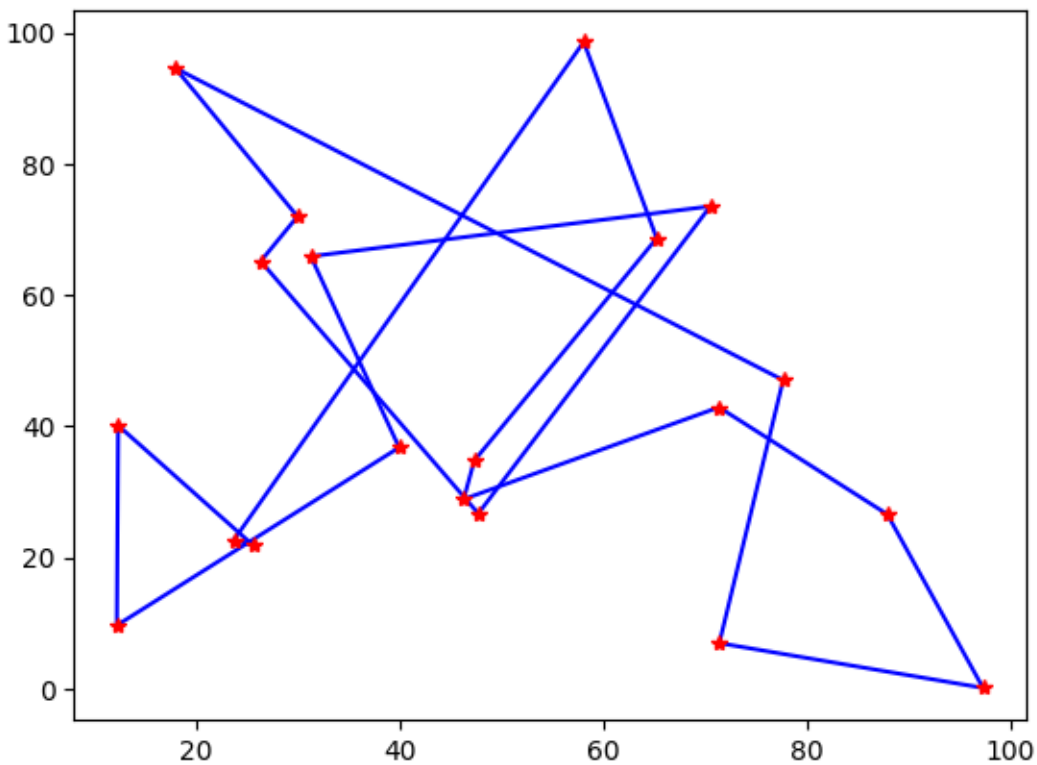
```
700.1759977286954 [ 4 13  0  2 14 11 12  5  3 19  1  8 17  9 16 18  6 15  7 10]
```



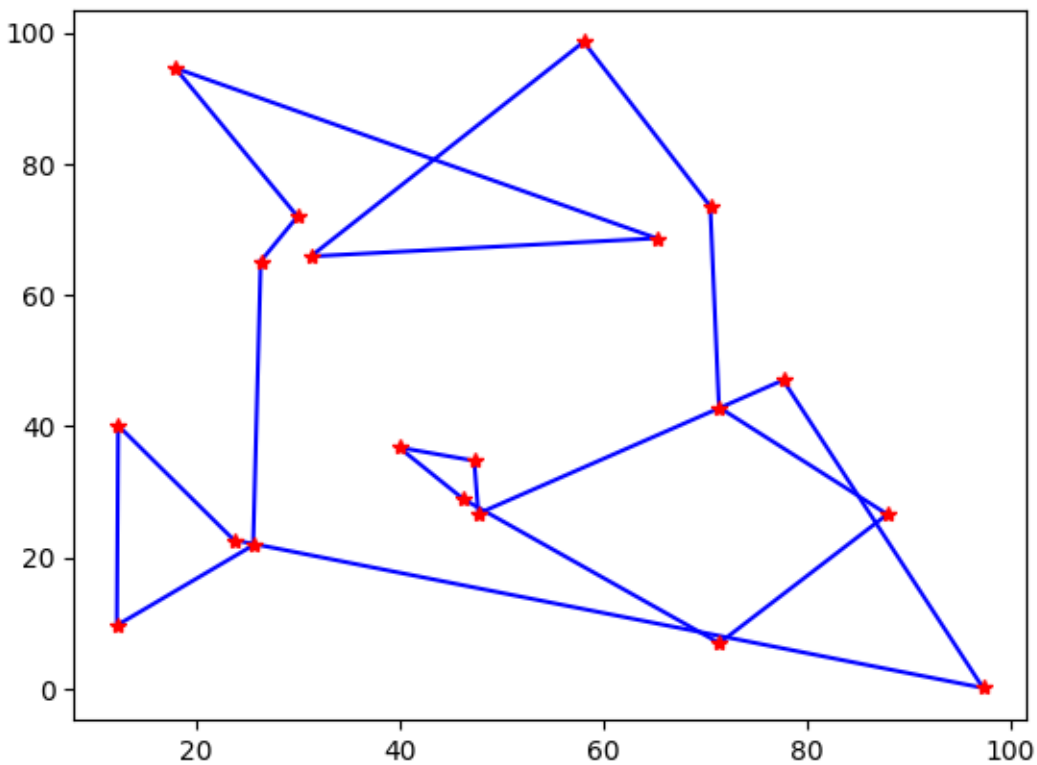

```
676.142702835279 [ 4 13  0  2 14 11 12  5  3  1  8  9  6 18  7 10 15 17 16 19]
```



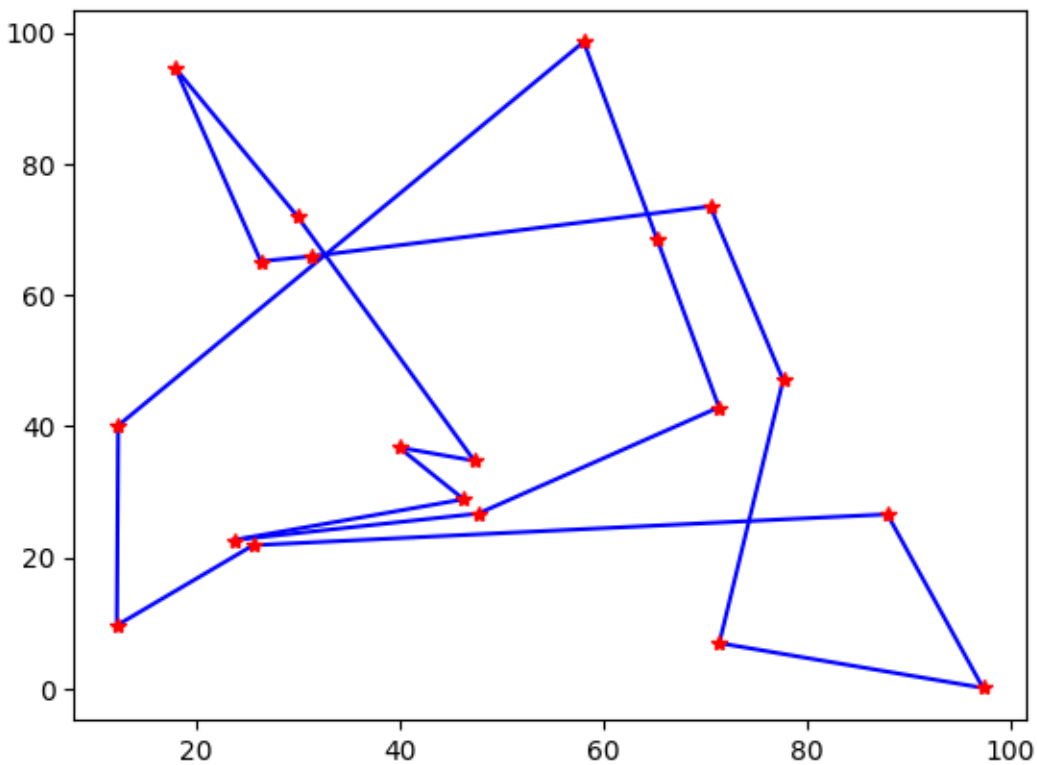

```
676.0575865850535 [ 7 10  1 19 17 16 18 14  8  9 12 11  3  5  2  0  6 15  4 13]
```



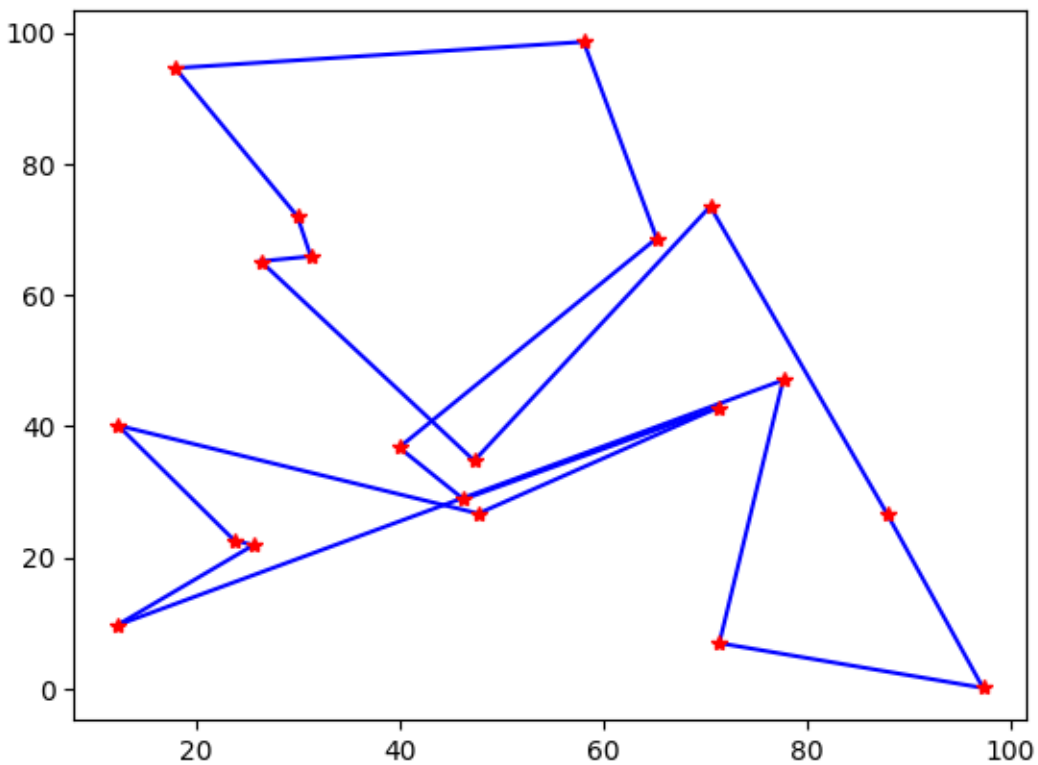

```
607.2454652759698 [ 2  8  0 19 17 16  3 12 11  5  7  1 18  6  9 15 10 13  4 14]
```



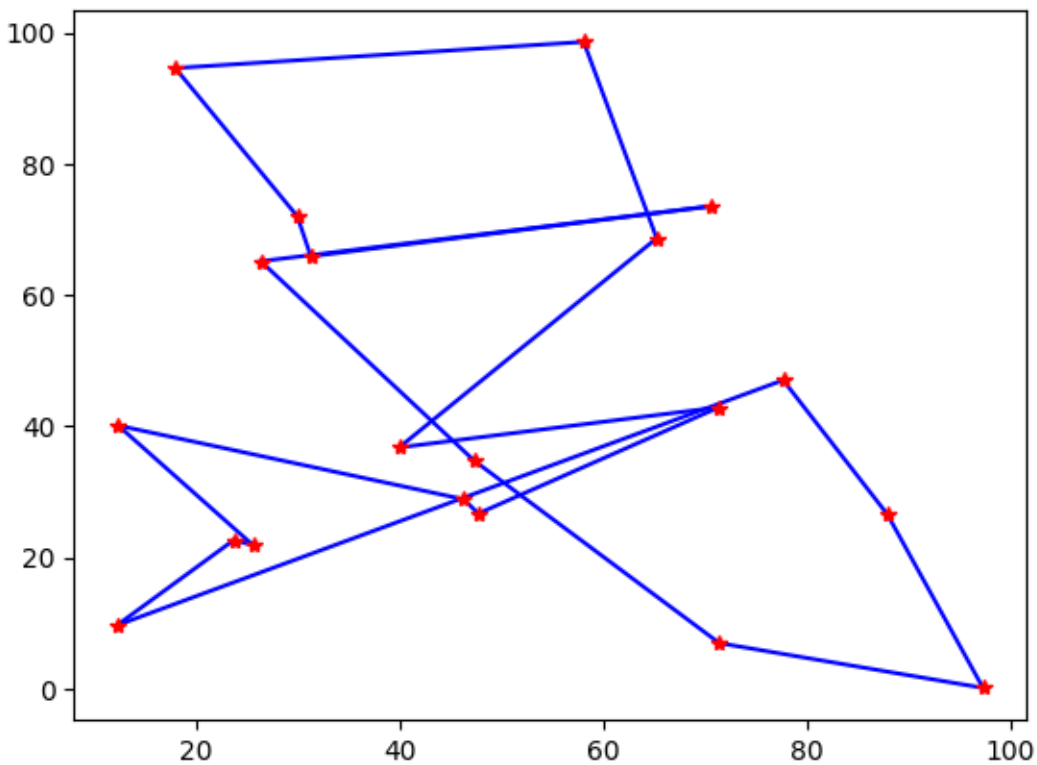

```
602.0845512535444 [14  8 16 19 17  6  9 15  5 18  4  0  2 11 12  3 13  7 10  1]
```



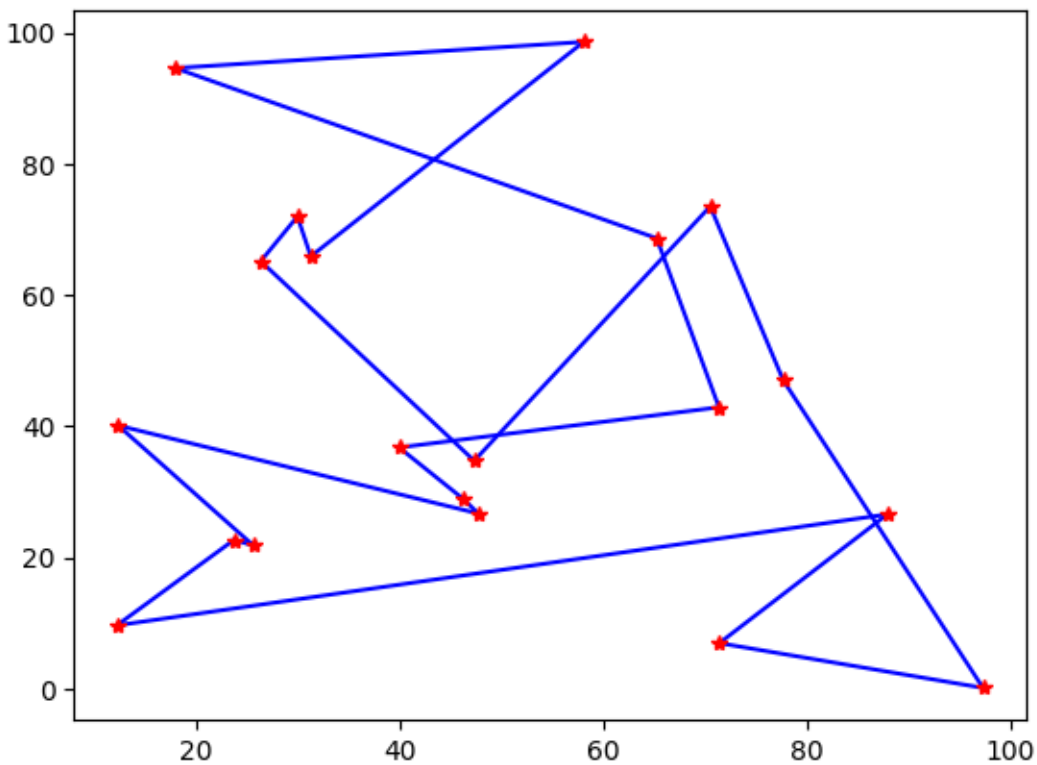

```
598.1733405655522 [14  6 16  8 17 19  2  0  9 15  4 18 11  5  3 12  1 10  7 13]
```



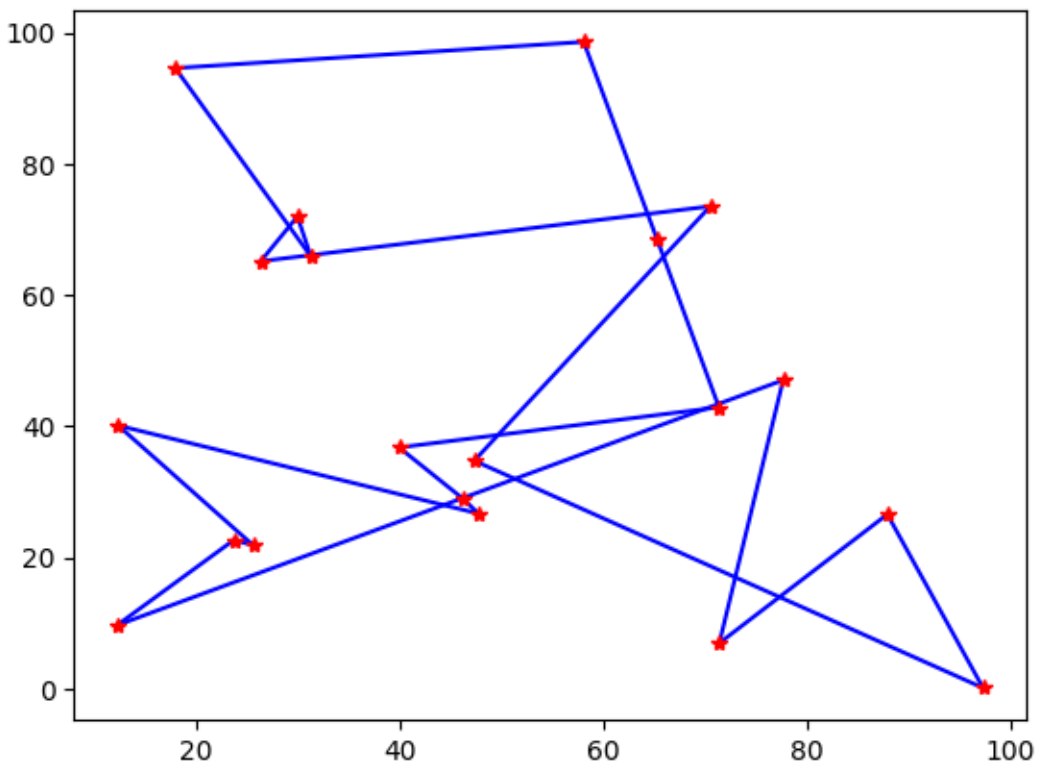

```
596.6019755467074 [ 6 16 14  8 17 19  2  0  9  4 18 15 11  3  5 12  1 13  7 10]
```



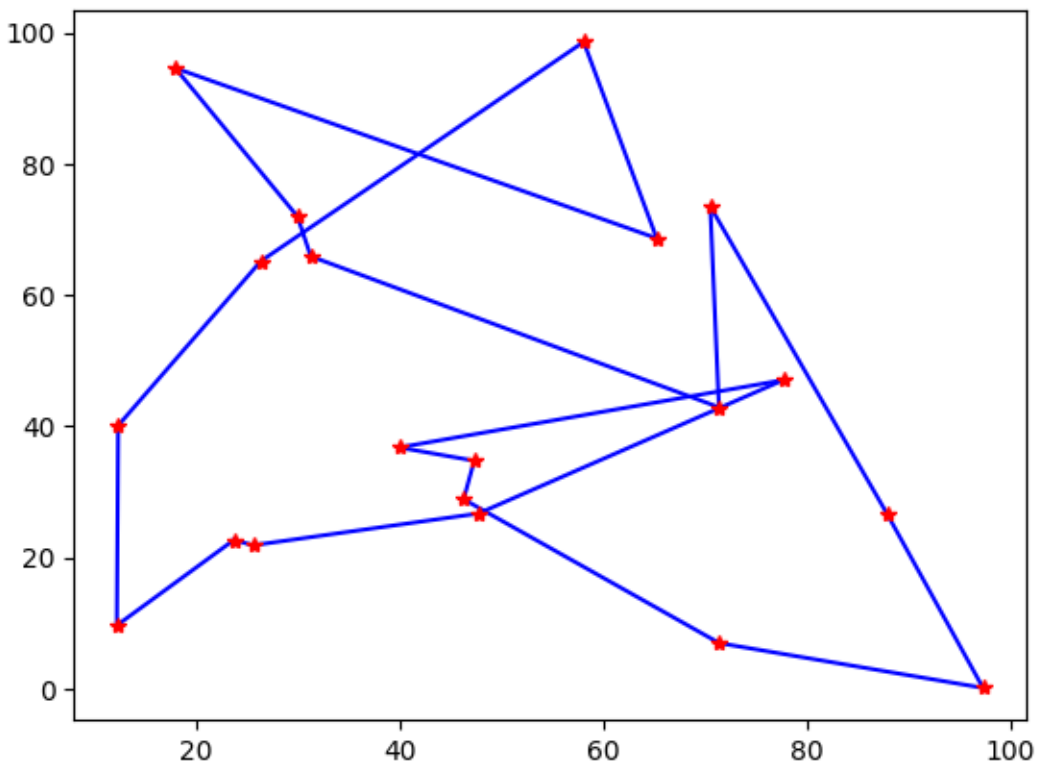

```
592.1334123325464 [14  6 16 17  8  2 19  0  4  9 15 18 11  3  5 12 13 10  7  1]
```



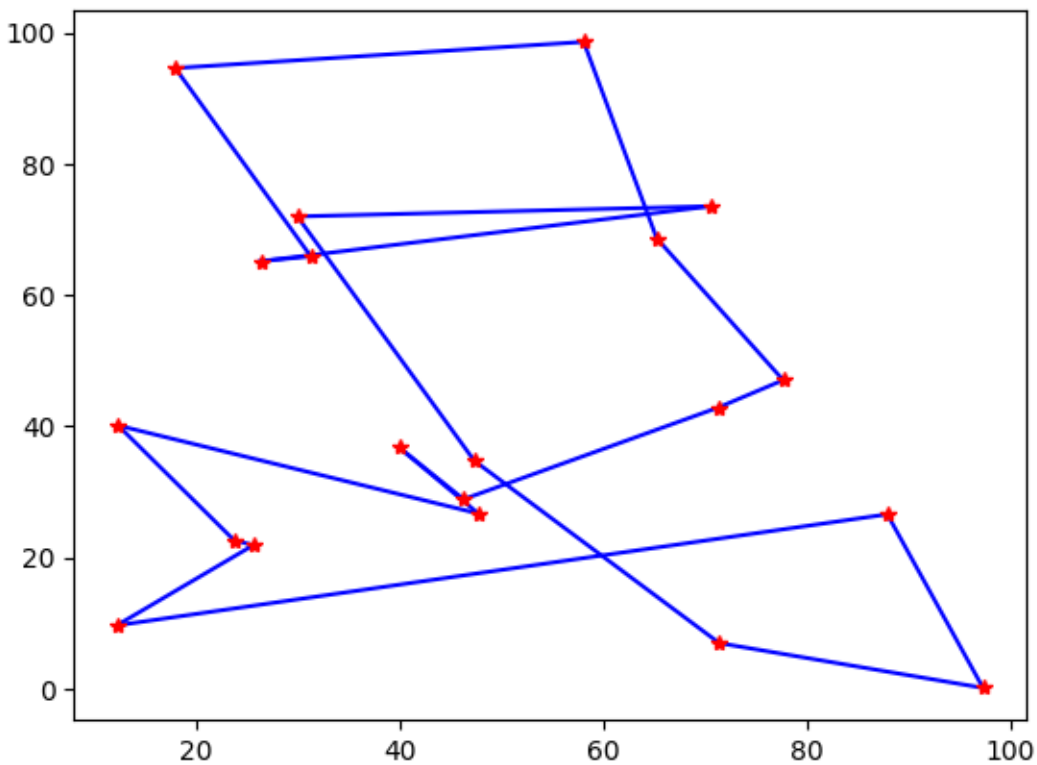

```
588.523954429239 [ 6 14 16 17  8 19  2  0  4  9 15 18 11  3  5 12  1 10 13  7]
```



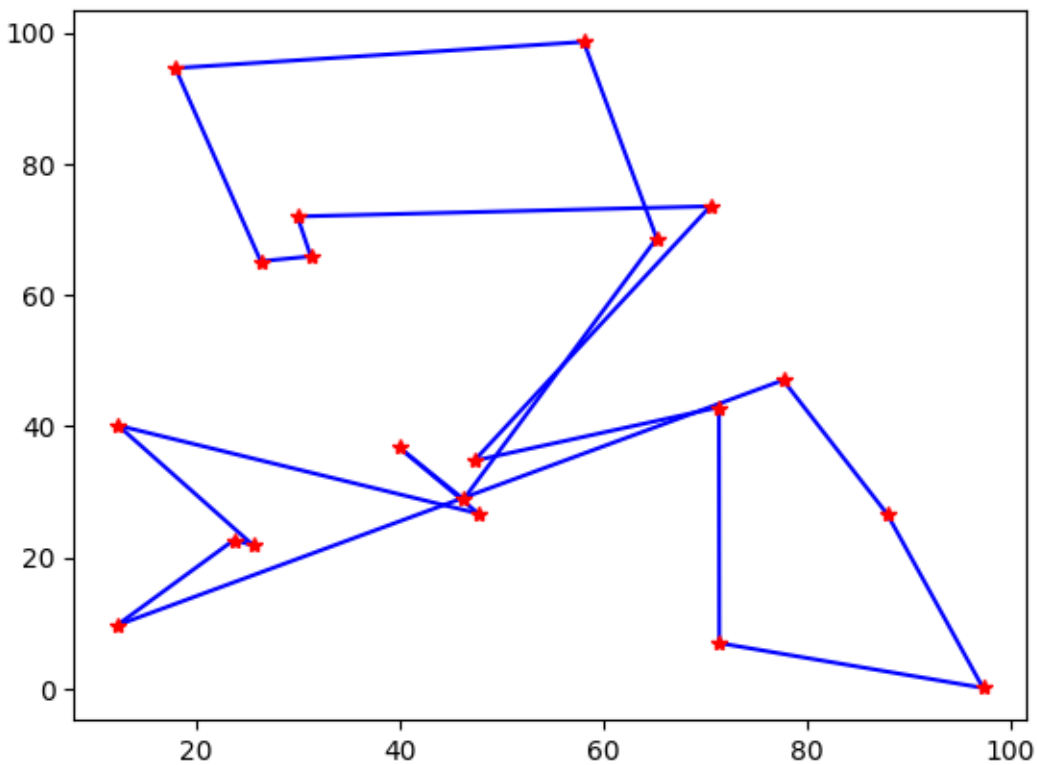

```
568.0992739907073 [ 4  8 17 19  0  2 16 11 12  5  3 18  1  9  6 15 10  7 13 14]
```



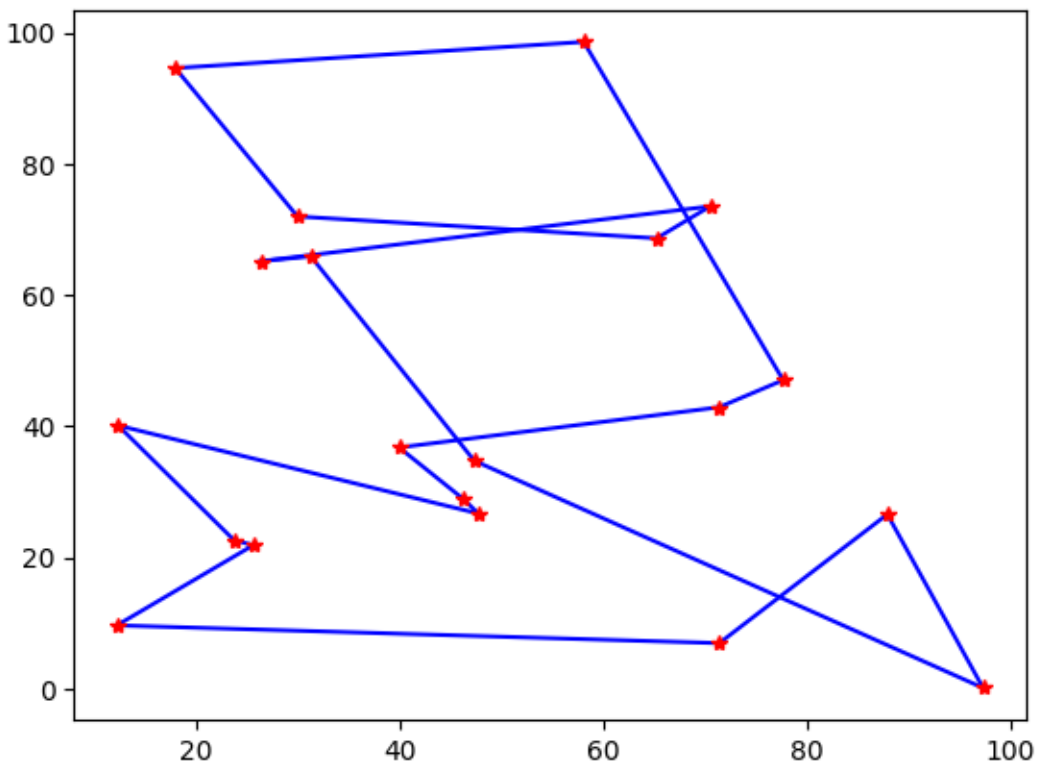

```
566.6228419483033 [ 6 17 14 16  8 19  2  0  1  4 15  9 18 11  5  3 12 13  7 10]
```



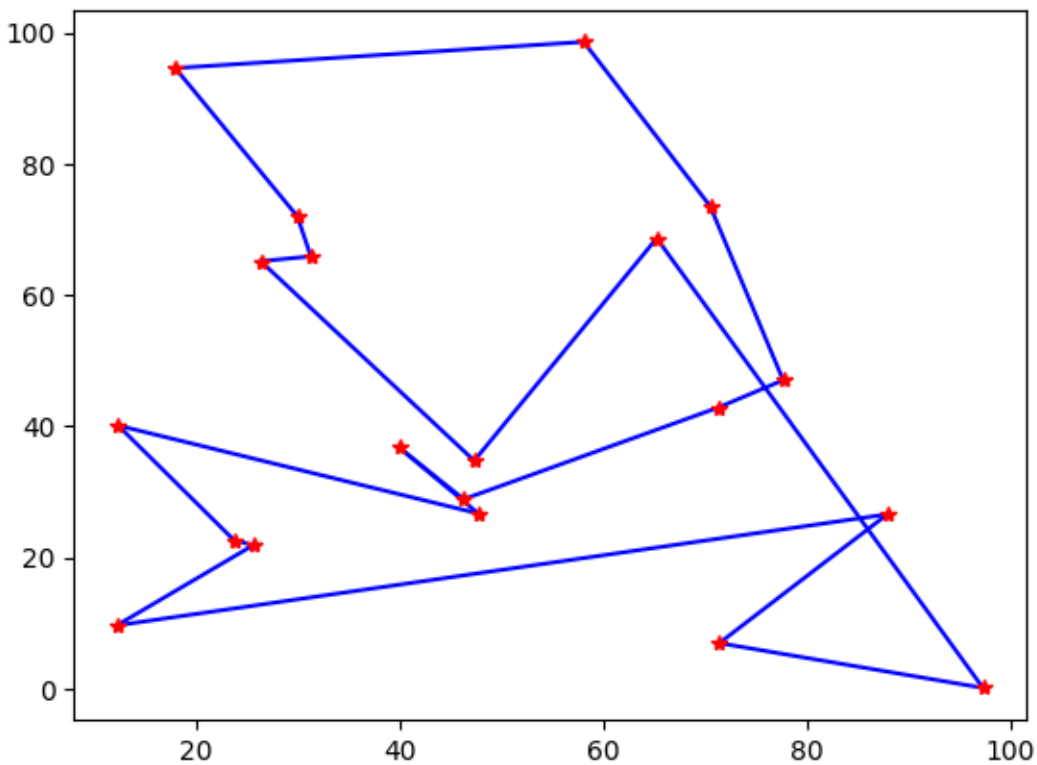

```
560.0493587289903 [ 6 14 17  8 16 19  2  0 15  9 18 11  3  5 12  1 13  7 10  4]
```



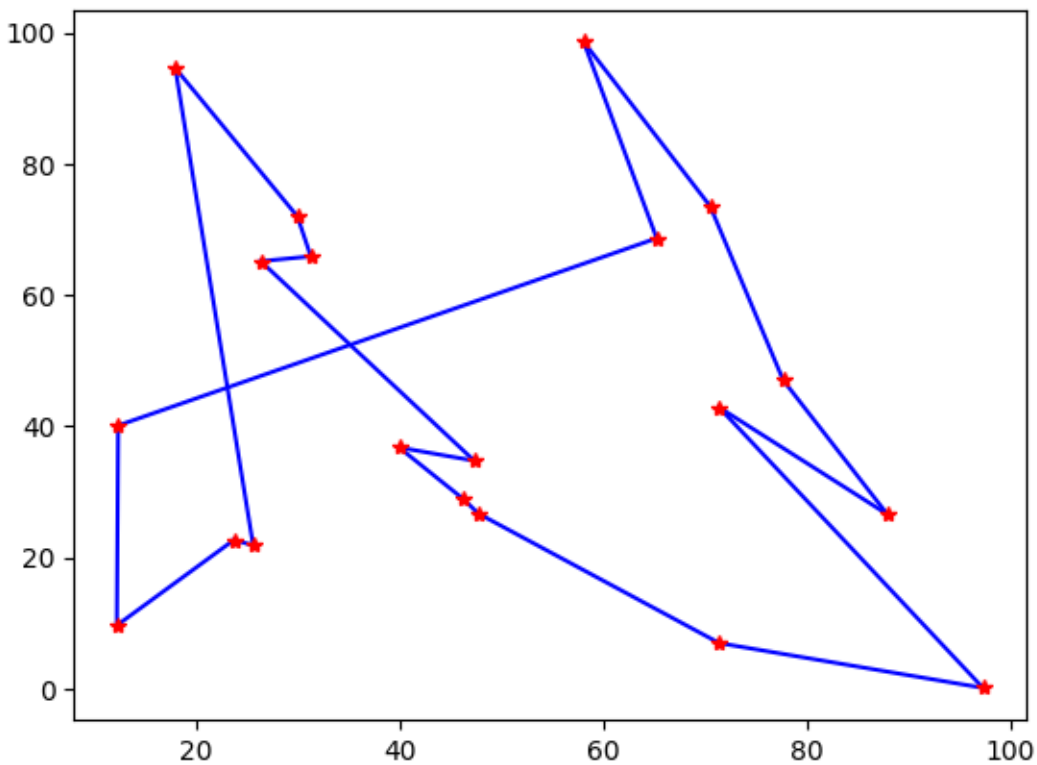

```
553.7555095969881 [ 6  8 16 14  0 17 19  2  1  4  9 15 18 11  5  3 12 10 13  7]
```



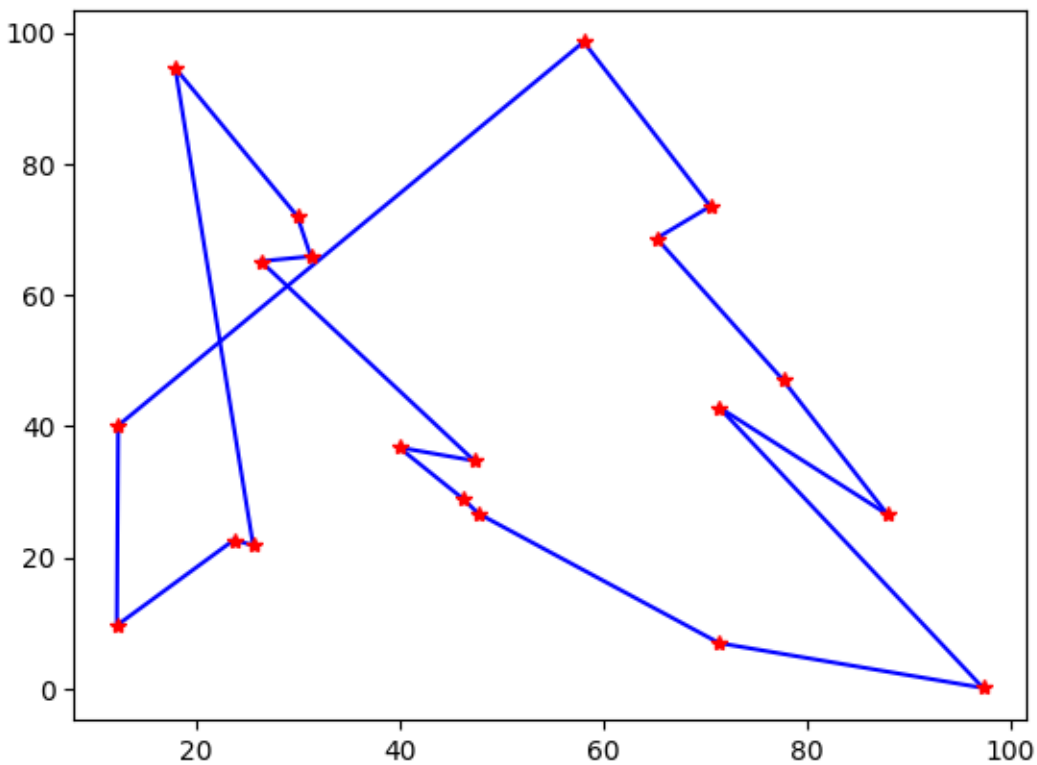

```
551.6402268936378 [ 6 16  8 17 19  2 14  1  4 15  9 18 11  5  3 12 13 10  7  0]
```



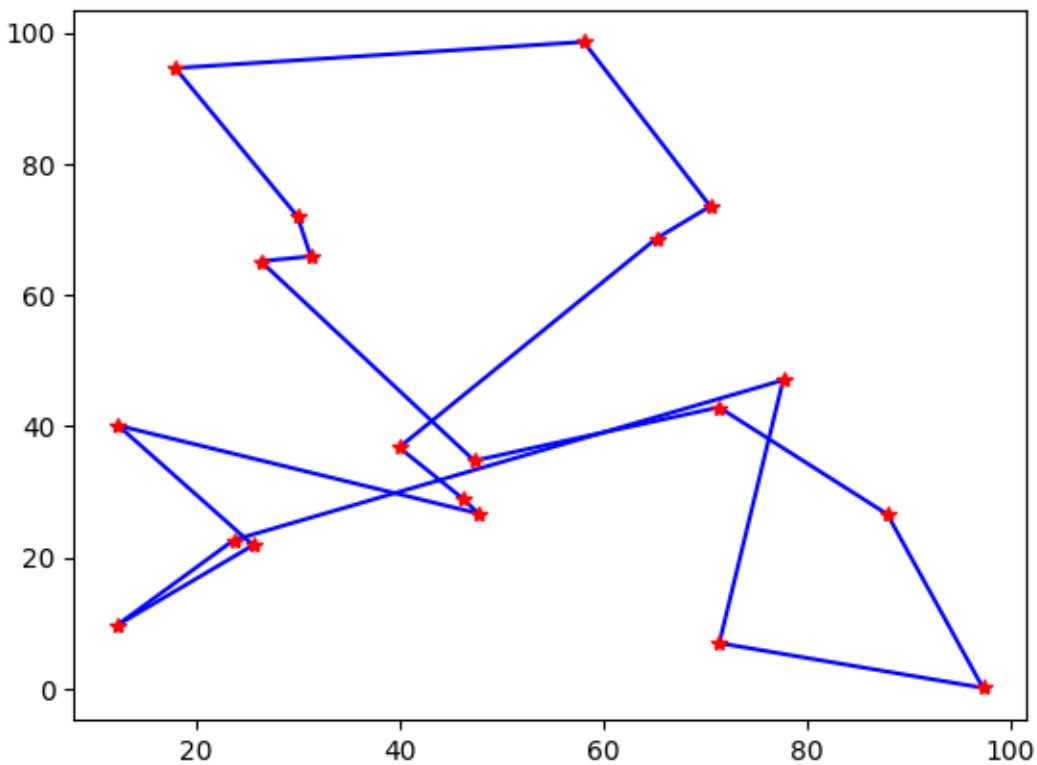

```
517.3722512399908 [ 4 13  1 14  2  0 11 12  5  3 19 17  8 16  6  9 15 18 10  7]
```



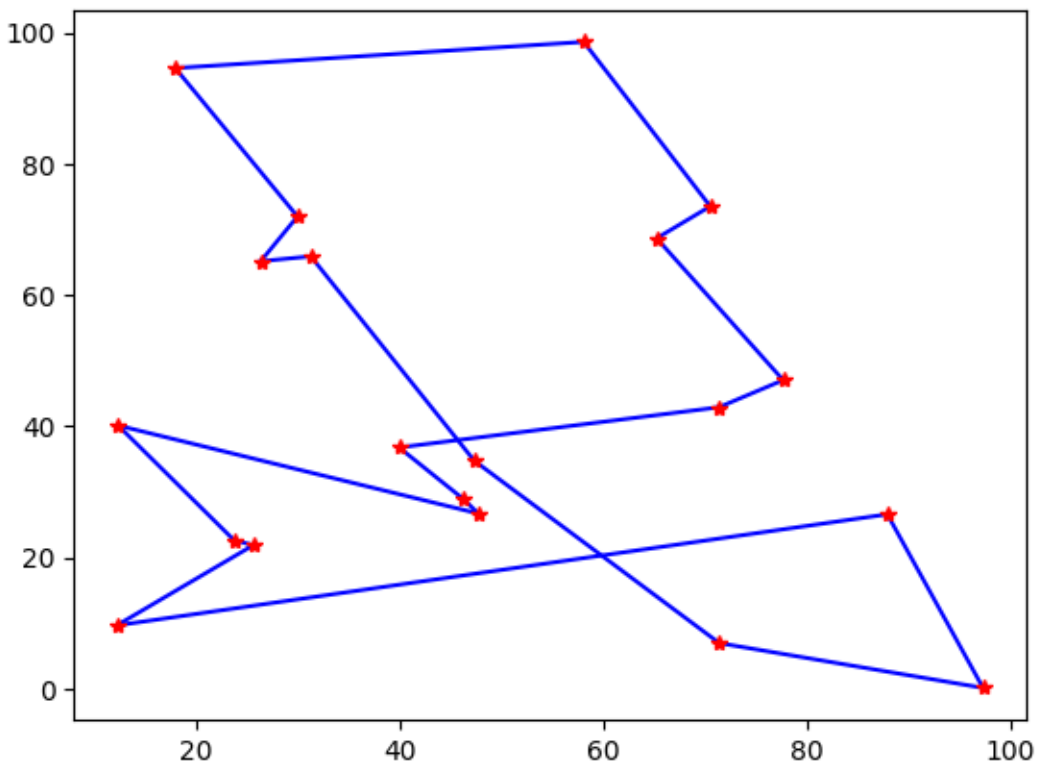

```
505.37533506618416 [ 4 13  1  0 14  2 11 12  5  3 19 17  8 16  6  9 15 18 10  7]
```



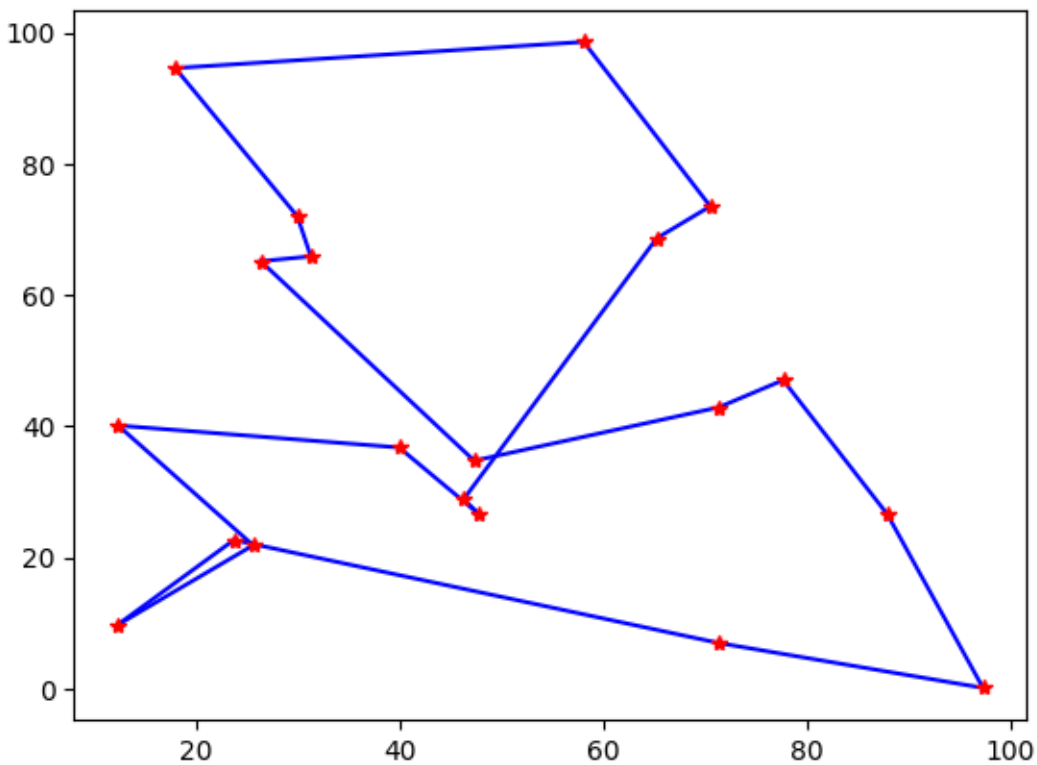

```
501.96371581308387 [ 6 16  8 17 19  2 14  0  9 15 18 11  3 12  5  1 10  7 13  4]
```



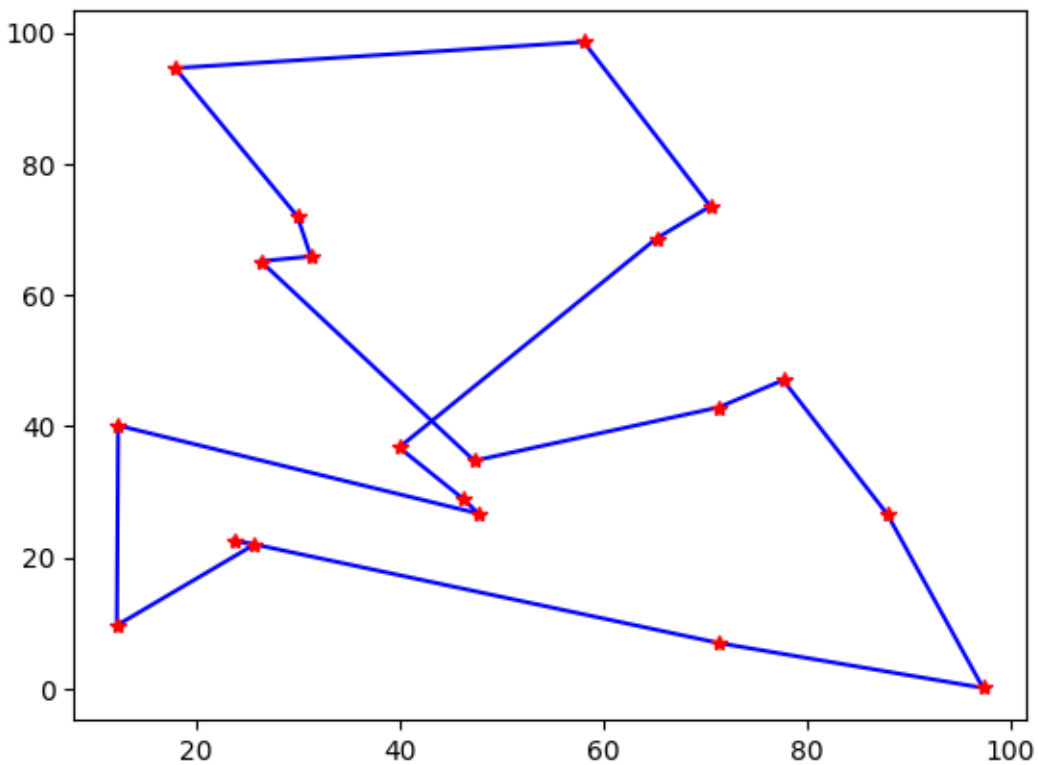

```
474.23745621060215 [ 6  8 16 17 19  2 14  0  1  4  9 15 18 11  5  3 12 13  7 10]
```



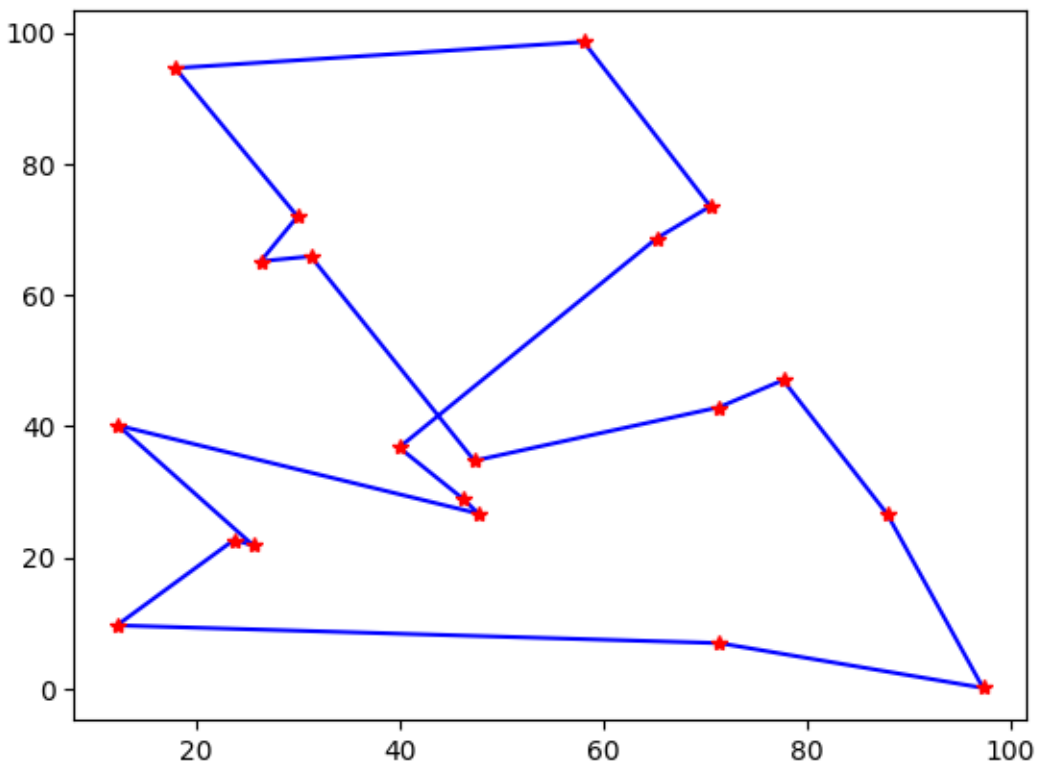

```
455.4851824125883 [ 6 16  8 17 19  2 14  0 15 18  9 11  3 12  5 10  7 13  1  4]
```



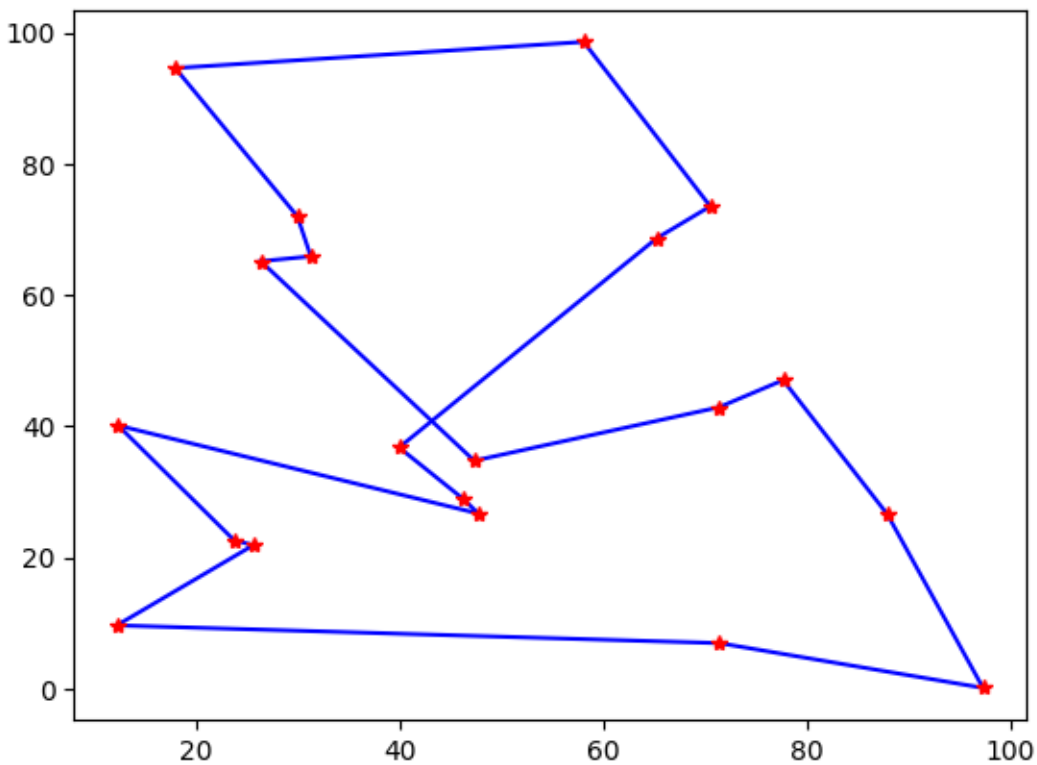

```
452.156840637376 [ 6 16  8 17 19  2 14  0  9 15 18 11 12  3  5 10  7 13  1  4]
```



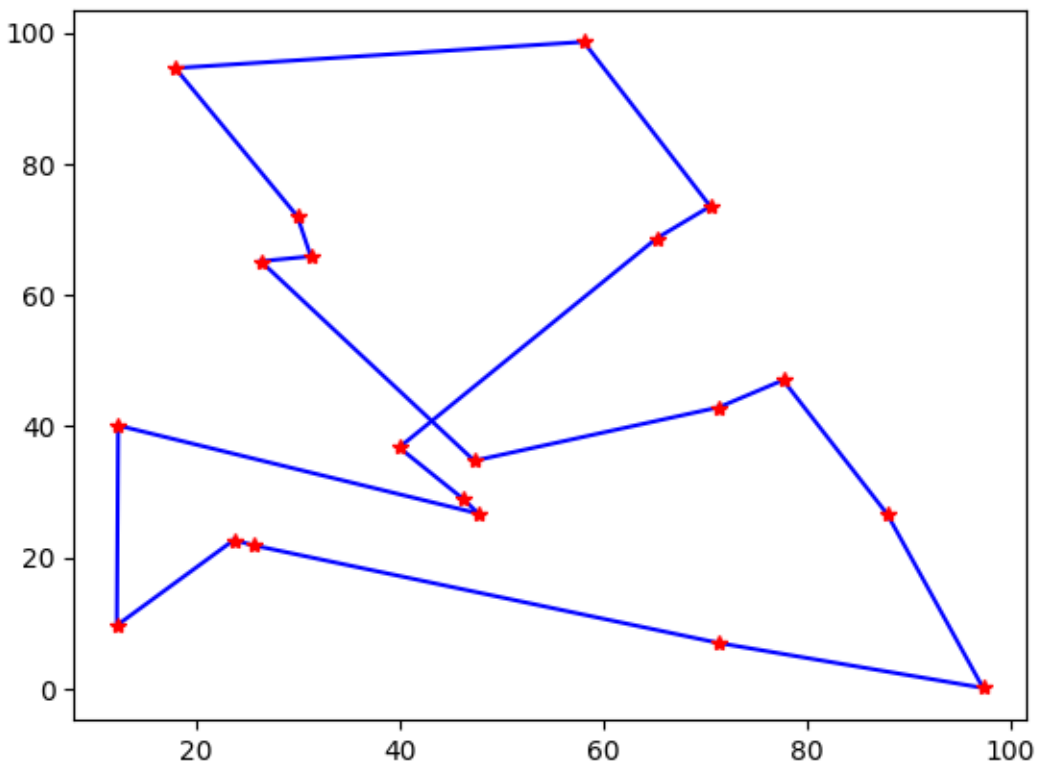

```
452.1276037288205 [ 6  8 16 17 19  2 14  0  9 15 18 11  3  5 12 10  7 13  1  4]
```



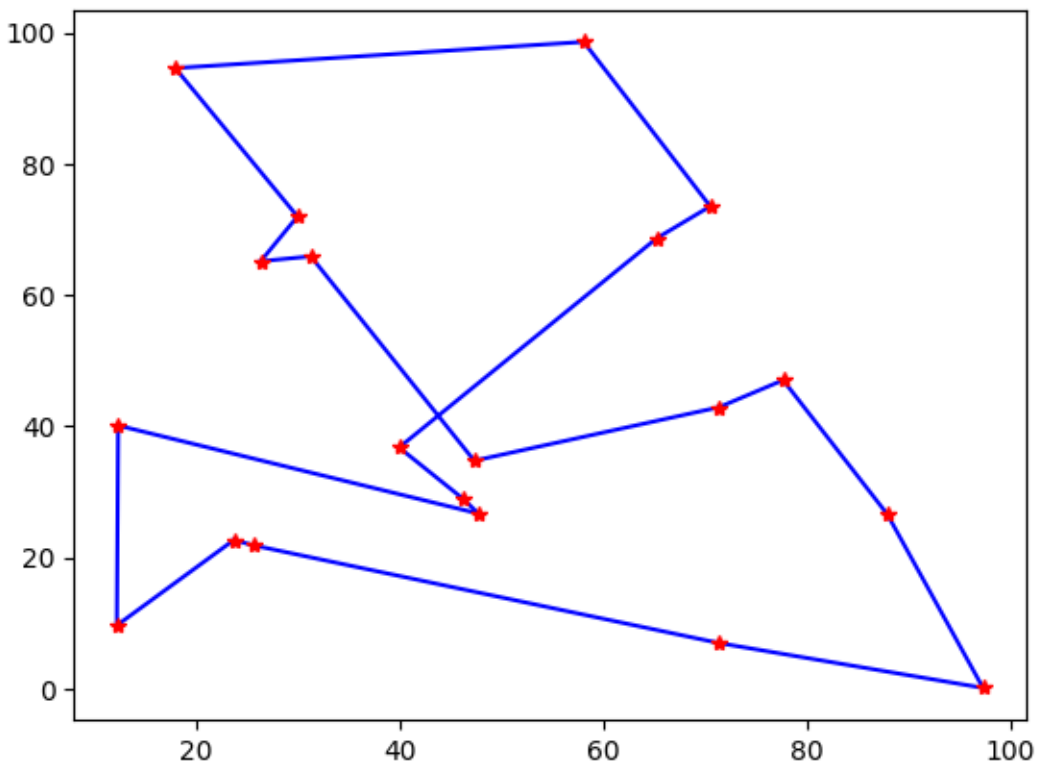

```
451.54931087538455 [ 6 16  8 17 19  2 14  0  9 15 18 11  5  3 12 10  7 13  1  4]
```



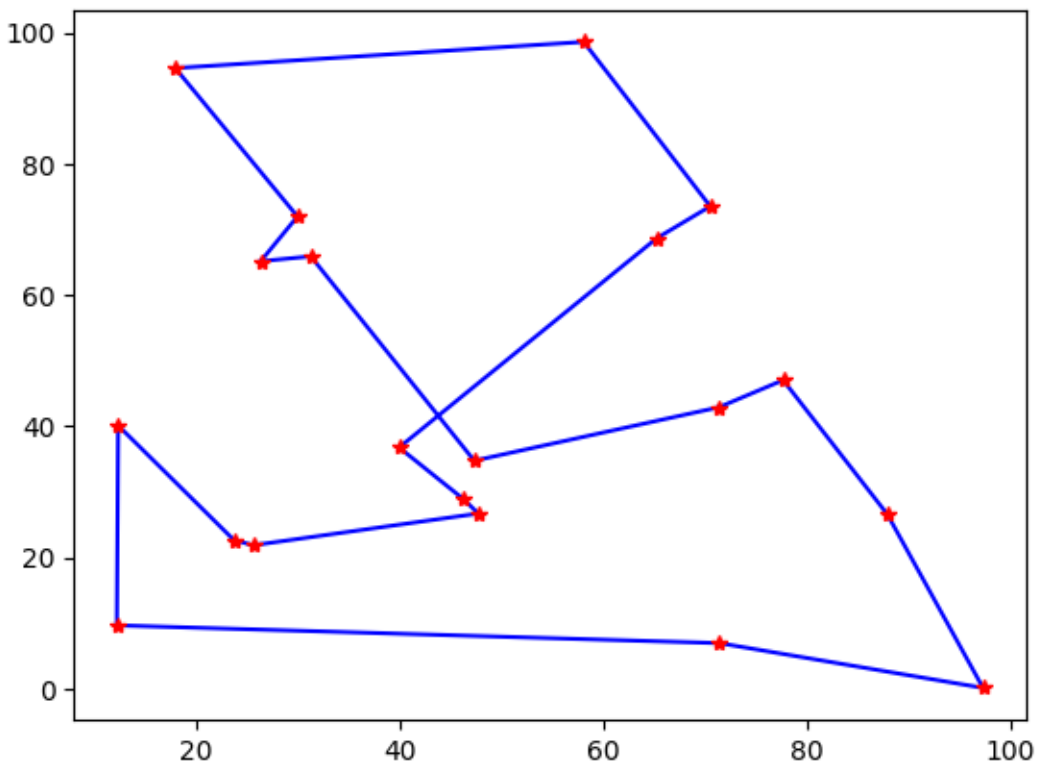

```
449.34333111018924 [ 6 16  8 17 19  2 14  0  9 15 18 11 12  5  3 10  7 13  1  4]
```



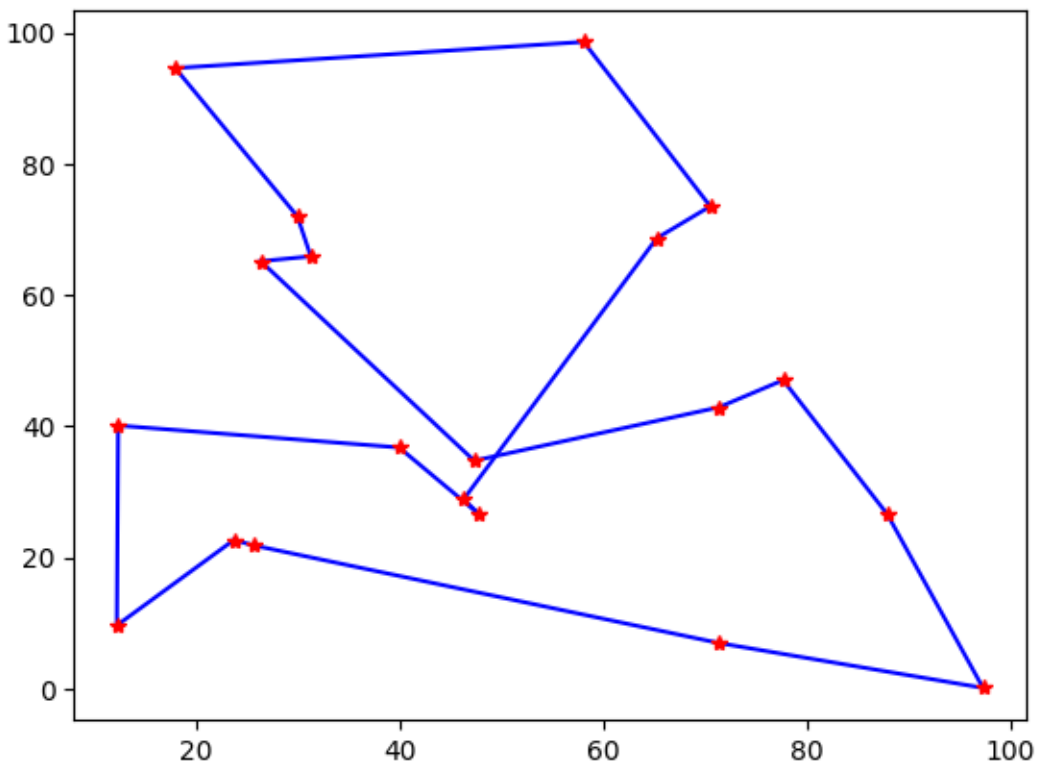

```
448.9461948346329 [ 6  8 16 17 19  2 14  0  9 15 18 11 12  5  3 10  7 13  1  4]
```



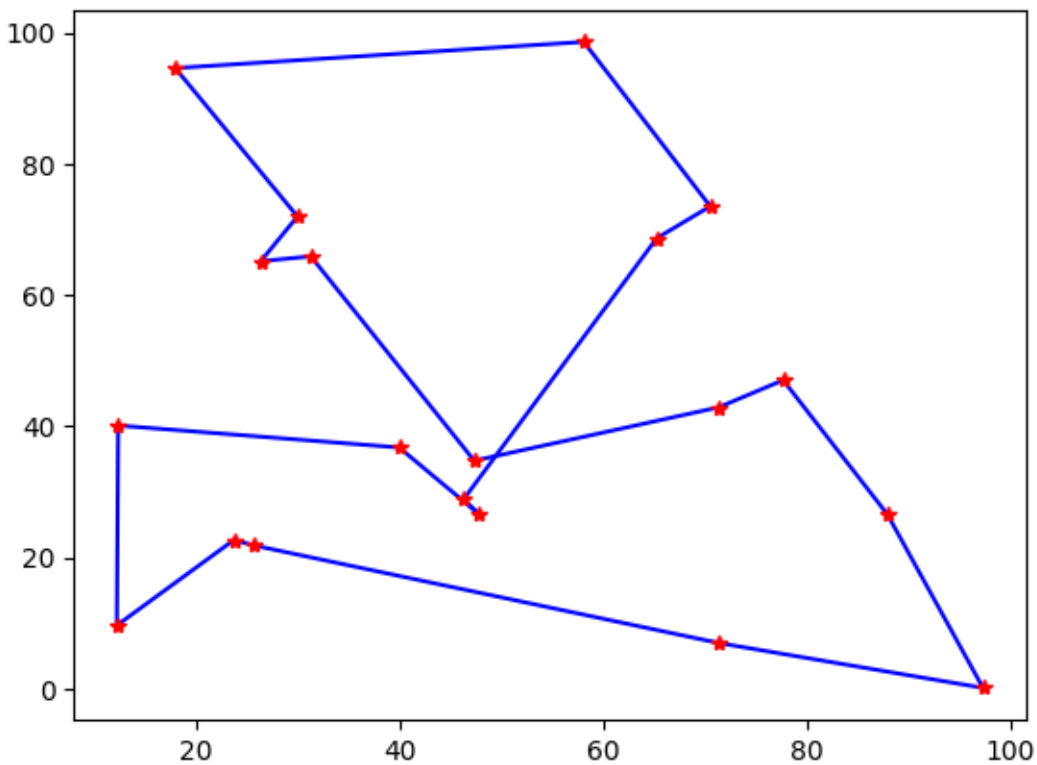

```
448.29714333082944 [ 6  8 16 17 19  2 14  0  9 15 18  3  5 11 12 10  7 13  1  4]
```



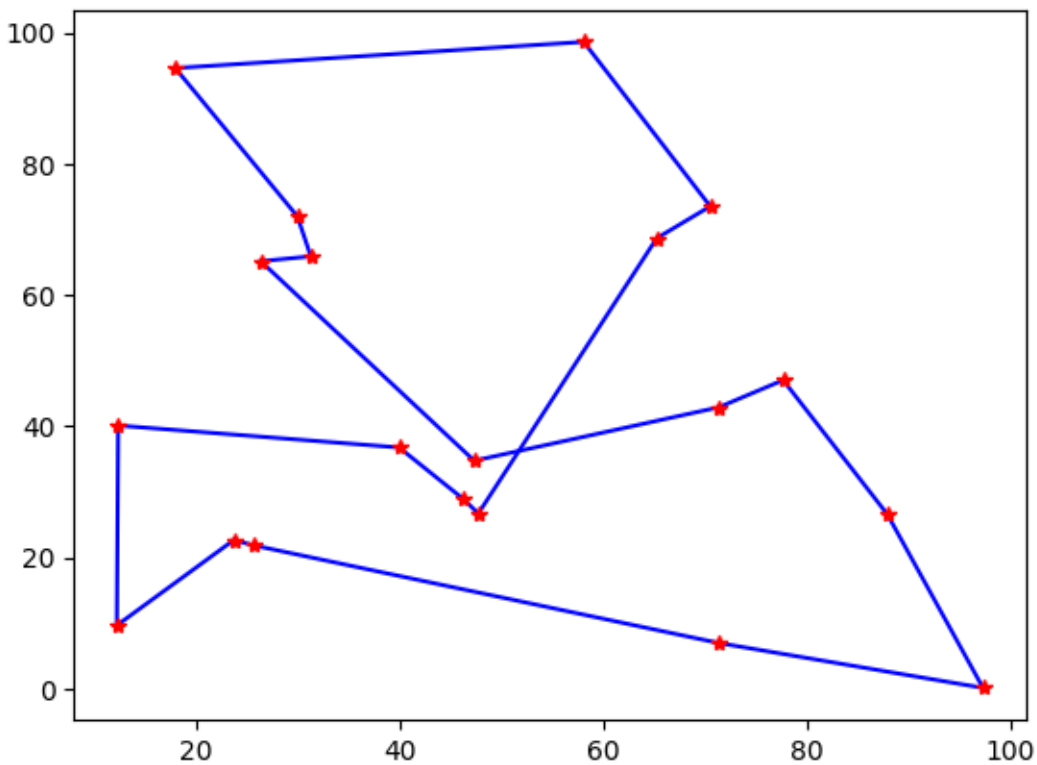

```
445.2698278050404 [ 6 16  8 17 19  2 14  0 15 18  9 11 12  5  3 10  7 13  1  4]
```



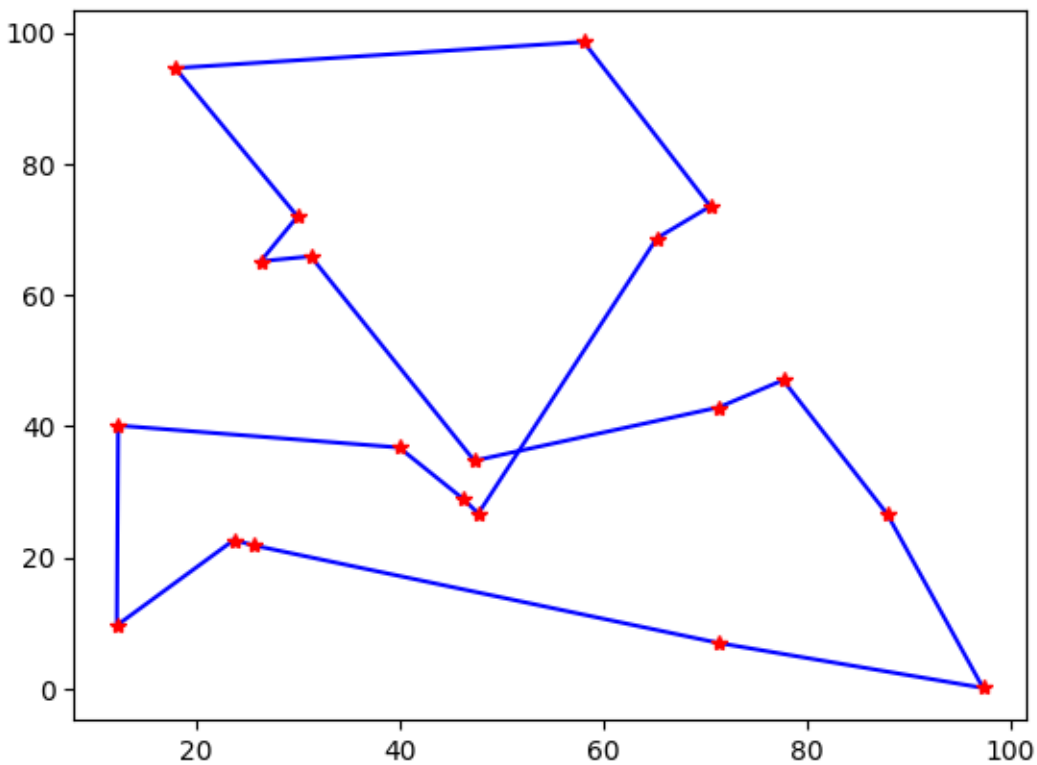

```
444.872691529484 [ 6  8 16 17 19  2 14  0 15 18  9 11 12  5  3 10  7 13  1  4]
```




```
444.066619294246 [ 6 16  8 17 19  2 14  0 18 15  9 11 12  5  3 10  7 13  1  4]
```

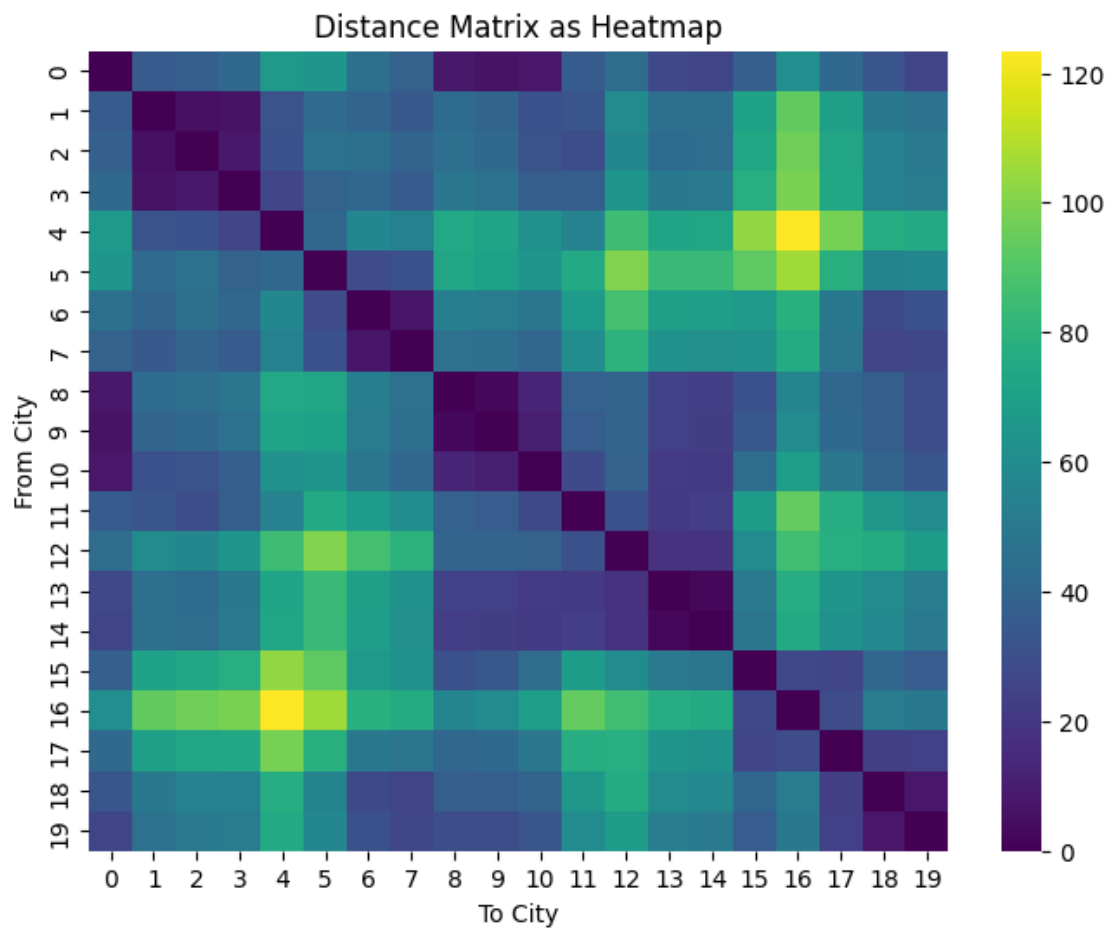



```
443.6694830186895 [ 6  8 16 17 19  2 14  0 18 15  9 11 12  5  3 10  7 13  1  4]
```



	0	1	2	3	4	5	6	7	8	9	10	11	\
0	0.0	35.1	37.0	41.1	66.7	64.8	45.2	38.4	8.1	6.0	7.7	35.4	
1	35.1	0.0	5.0	6.2	31.6	42.3	40.0	34.1	42.5	40.0	30.4	32.0	
2	37.0	5.0	0.0	7.7	30.6	46.1	45.0	39.1	44.0	41.4	31.5	28.7	
3	41.1	6.2	7.7	0.0	25.6	38.7	40.6	35.5	48.7	46.1	36.6	36.4	
4	66.7	31.6	30.6	25.6	0.0	40.3	56.6	54.0	74.2	71.6	61.9	54.8	
5	64.8	42.3	46.1	38.7	40.3	0.0	28.0	30.8	72.7	70.7	64.4	74.3	
6	45.2	40.0	45.0	40.6	56.6	28.0	0.0	7.2	52.1	50.8	47.8	67.1	
7	38.4	34.1	39.1	35.5	54.0	30.8	7.2	0.0	45.5	44.1	40.7	60.1	
8	8.1	42.5	44.0	48.7	74.2	72.7	52.1	45.5	0.0	2.6	12.7	37.9	
9	6.0	40.0	41.4	46.1	71.6	70.7	50.8	44.1	2.6	0.0	10.1	35.8	
10	7.7	30.4	31.5	36.6	61.9	64.4	47.8	40.7	12.7	10.1	0.0	27.8	
11	35.4	32.0	28.7	36.4	54.8	74.3	67.1	60.1	37.9	35.8	27.8	0.0	
12	43.2	59.4	57.3	64.8	85.2	100.1	86.5	79.4	39.4	39.1	38.8	30.5	
13	26.5	43.9	42.6	49.7	72.2	83.4	69.1	62.0	24.4	23.4	21.5	20.8	
14	25.3	44.4	43.3	50.3	73.2	83.3	68.5	61.4	22.7	21.8	20.7	22.6	
15	36.8	71.3	73.6	77.1	102.7	92.6	66.6	62.0	30.8	33.4	43.4	67.8	
16	60.8	93.2	96.3	98.5	123.4	106.0	78.2	75.7	56.3	58.6	68.1	93.9	
17	41.4	69.0	72.7	73.6	97.6	78.0	50.1	47.8	40.2	41.7	49.0	76.8	
18	32.8	50.1	54.4	53.9	76.4	55.2	27.5	24.9	36.2	36.3	39.1	65.7	
19	25.4	46.2	50.2	50.6	74.4	57.3	30.7	26.5	28.7	28.7	32.0	59.1	

	12	13	14	15	16	17	18	19
0	43.2	26.5	25.3	36.8	60.8	41.4	32.8	25.4
1	59.4	43.9	44.4	71.3	93.2	69.0	50.1	46.2
2	57.3	42.6	43.3	73.6	96.3	72.7	54.4	50.2
3	64.8	49.7	50.3	77.1	98.5	73.6	53.9	50.6
4	85.2	72.2	73.2	102.7	123.4	97.6	76.4	74.4
5	100.1	83.4	83.3	92.6	106.0	78.0	55.2	57.3
6	86.5	69.1	68.5	66.6	78.2	50.1	27.5	30.7
7	79.4	62.0	61.4	62.0	75.7	47.8	24.9	26.5
8	39.4	24.4	22.7	30.8	56.3	40.2	36.2	28.7
9	39.1	23.4	21.8	33.4	58.6	41.7	36.3	28.7
10	38.8	21.5	20.7	43.4	68.1	49.0	39.1	32.0
11	30.5	20.8	22.6	67.8	93.9	76.8	65.7	59.1
12	0.0	17.4	18.1	59.3	85.7	77.6	75.4	67.9
13	17.4	0.0	2.1	50.2	77.0	64.3	59.2	51.8
14	18.1	2.1	0.0	48.2	74.9	62.5	57.9	50.4
15	59.3	50.2	48.2	0.0	26.8	25.7	40.6	36.0
16	85.7	77.0	74.9	26.8	0.0	28.1	50.9	50.0
17	77.6	64.3	62.5	25.7	28.1	0.0	22.9	23.2
18	75.4	59.2	57.9	40.6	50.9	22.9	0.0	7.5
19	67.9	51.8	50.4	36.0	50.0	23.2	7.5	0.0



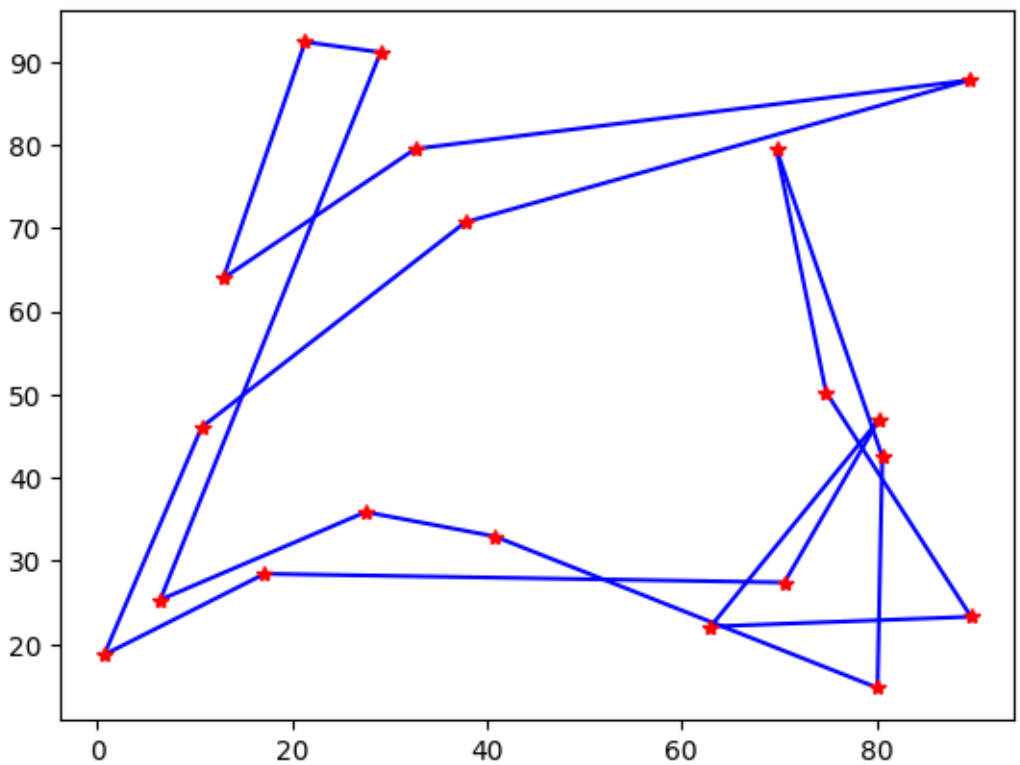
1.8 Task 9

[9]:
```
# MODIFIED ALSO CROSSOVER
from random import randint
from random import sample
from random import random

# Creating the cities
N = 20
xy = 100 * np.random.rand(N,2)

# Computing distance matrix
d = np.empty((N,N))
for i in range(N):
    for j in range(N):
        # Calculate Euclidean distance
        dx = xy[i,0] - xy[j,0]
        dy = xy[i,1] - xy[j,1]
```

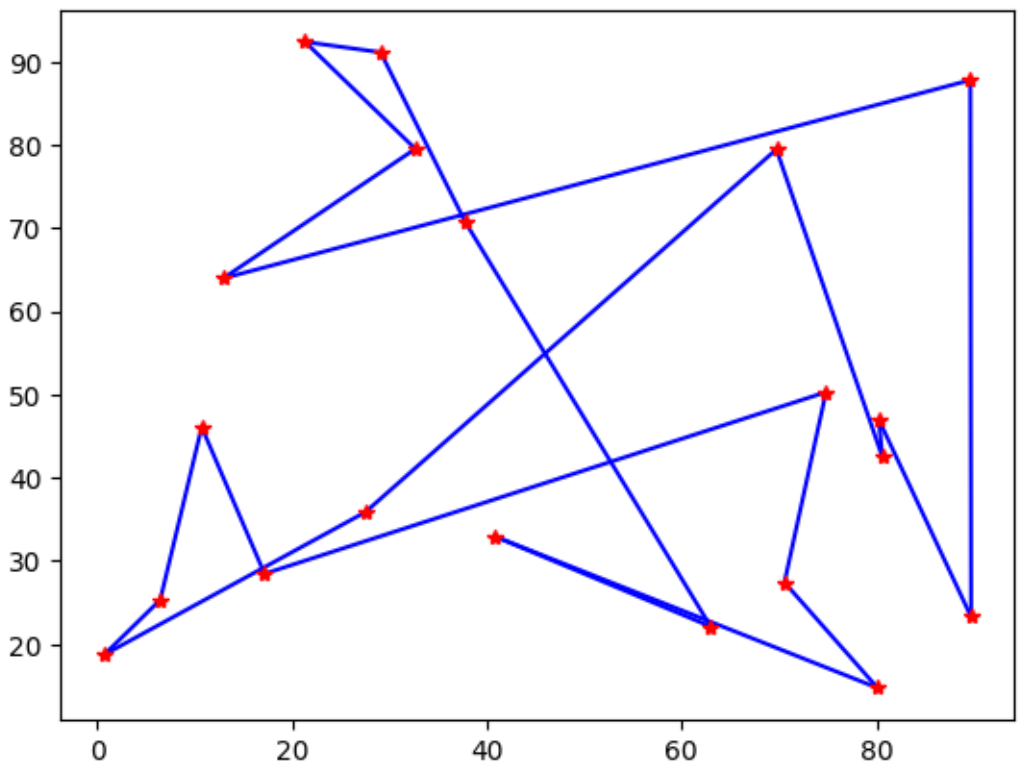
```
        d[i,j] = np.sqrt(dx**2 + dy**2)

# Initialize Global Parameters
iterations = 1_000_000
# N = Genome length is determined by cities
M = 1000         # Population size
crossover_prob = 0.5
mutation_rate = 1 / N

def fitness_function(priorities):
    permutation = np.argsort(priorities)[::-1]
    distance = tsp_simulator(permutation)
    fitness = 1.0 / distance # So the higher the distance the worse the fitness
    return fitness

def random_individual(N):
    return np.random.rand(N)

def random_population(M, N):
    return [random_individual(N) for _ in range(M)]

def eval_fitness(population):
    return [fitness_function(ind) for ind in population]

def tournament():
    i, j = sample(range(M), 2)
    return i if fitnesses[i] >= fitnesses[j] else j

def negative_tournament():
    i, j = sample(range(M), 2)
    # Returns the index of the weaker individual to be replaced
    return i if fitnesses[i] <= fitnesses[j] else j

def mutate():
    idx = tournament()
    parent = population[idx]
    offset = 0.05 * (np.random.rand(N) - 0.5)
    offspring = parent + offset
    return offspring

def crossover():
    idx1 = tournament()
    idx2 = tournament()
    parent1, parent2 = population[idx1], population[idx2]

    # line crossover
    alpha = np.random.rand()
```

```
    offspring = alpha * parent1 + (1 - alpha) * parent2
    return offspring

population = random_population(M, N)
fitnesses = eval_fitness(population)

# Track the best individual
best_idx = fitnesses.index(max(fitnesses))
best_genome = np.argsort(population[best_idx])[::-1]
best_fit = fitnesses[best_idx]
for i in range(iterations):
    if random() < crossover_prob:
        offspring = crossover()
    else:
        offspring = mutate()

    f = fitness_function(offspring)

    k = negative_tournament()
    population[k] = offspring
    fitnesses[k] = f

    if f > best_fit:
        best_fit = f
        best_genome = np.argsort(offspring)[::-1]
        print(1/best_fit, best_genome)
        plot_tour(best_genome)

permuted_d = d[best_genome][:,best_genome]
visualise(permuted_d)
```

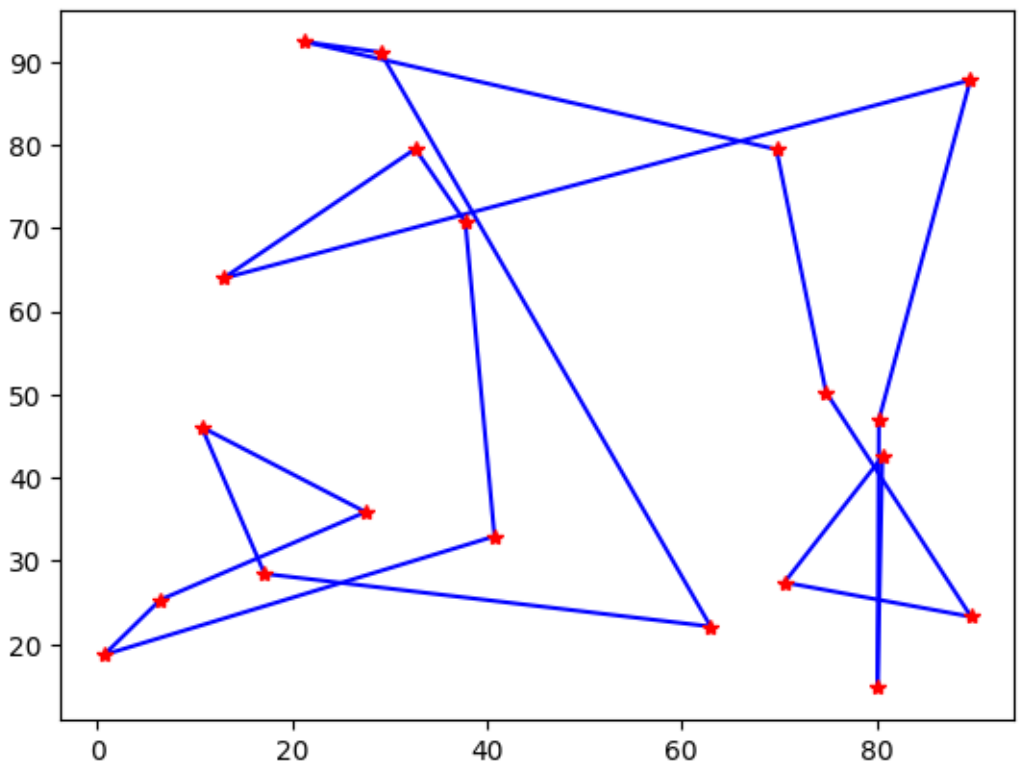
```
668.5833417673651 [15 18 12 11  9 10 16  3  7 17 19  2  4  1  6  5  8 13  0 14]
```



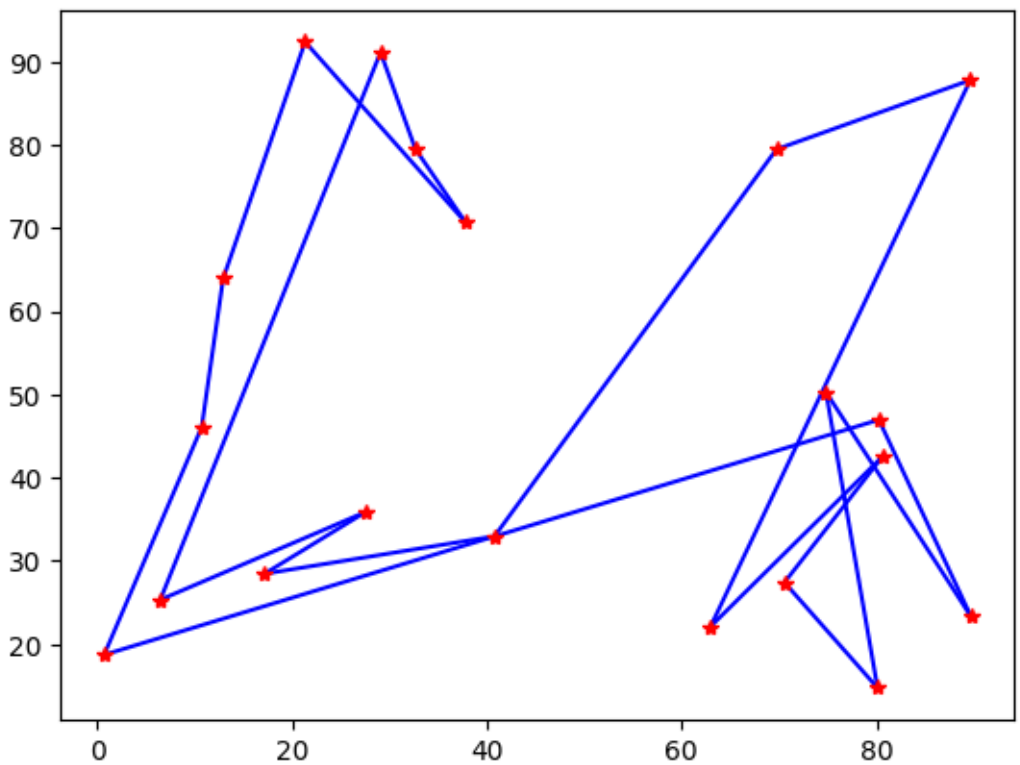

```
650.297205429169 [11  5  1  6  4  2  8 12  7  3 15  9 14 13 19  0 17 10 16 18]
```



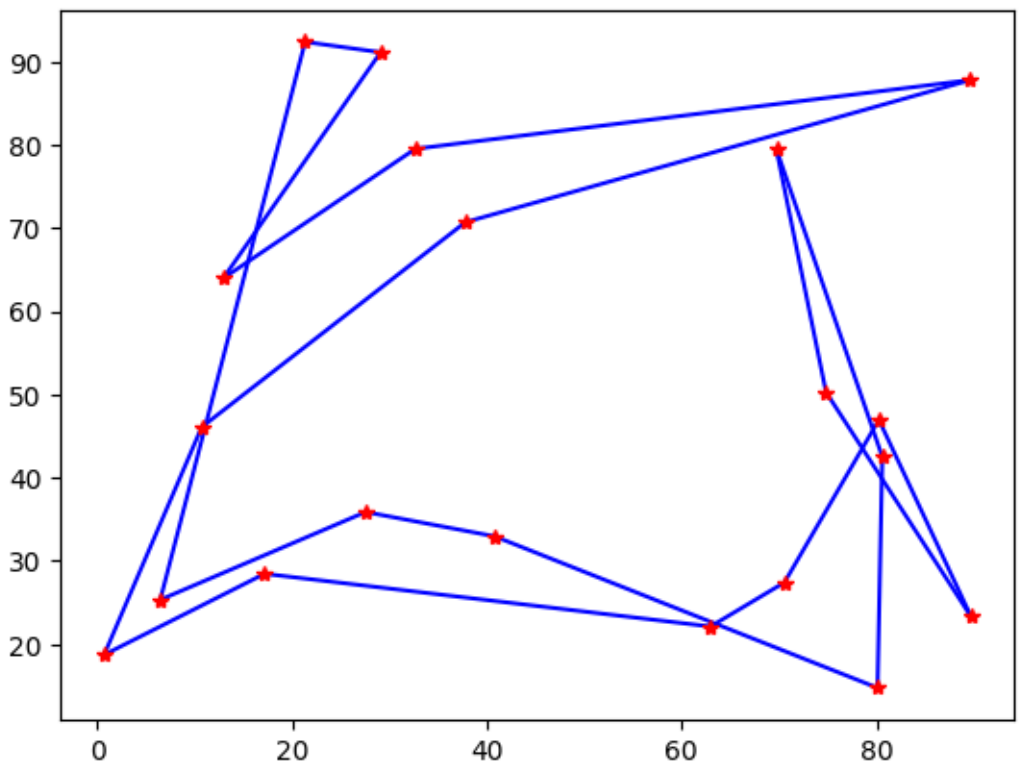

```
647.8977461180127 [12 14 13 17 19  0  7  8  6  1  5 18  3 16 15 11  9 10  4  2]
```



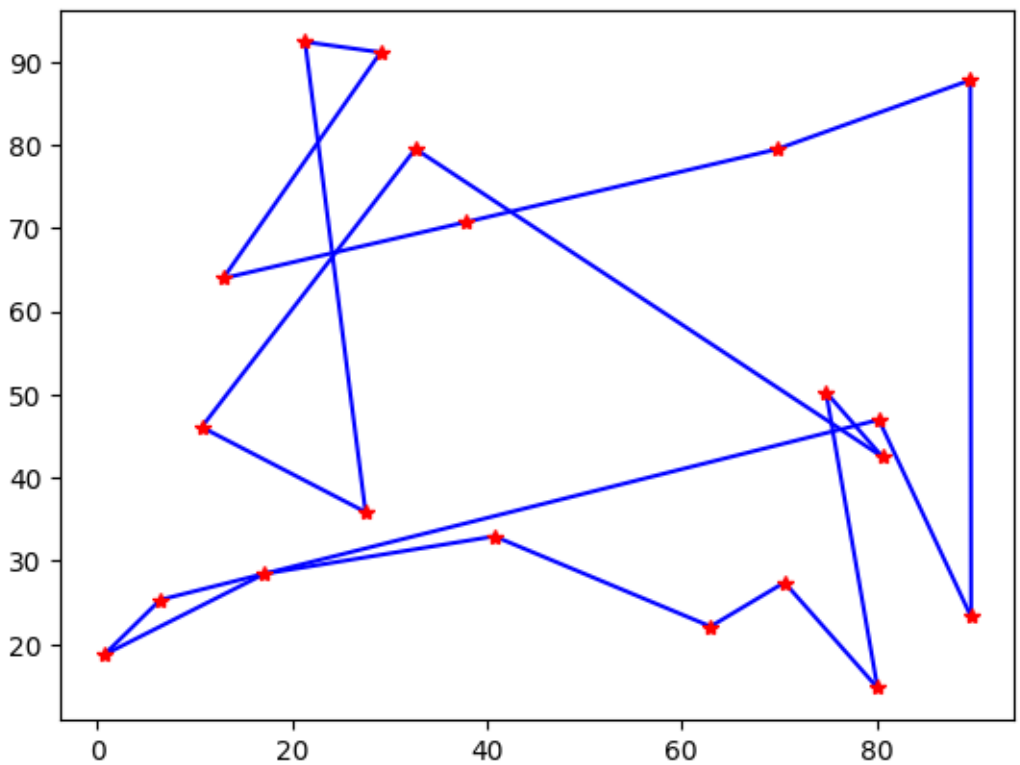

```
641.1557395513094 [18 11  9  3 15 16 12  5 10  7 14 17 19  2  6  8  4  1 13  0]
```



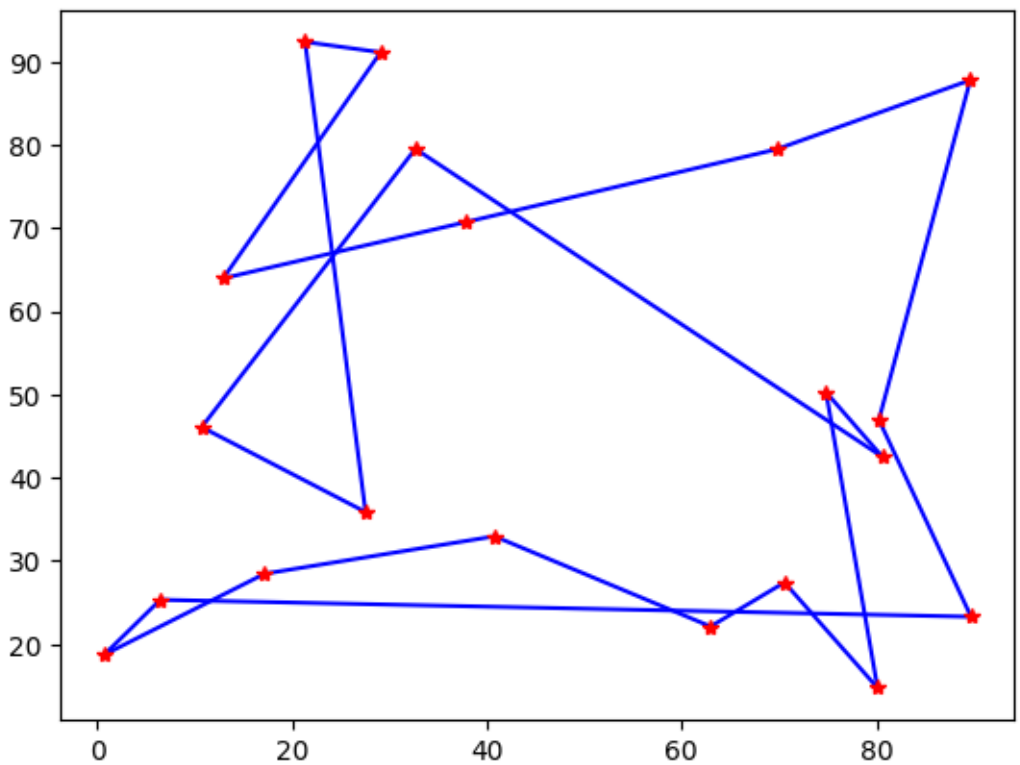

```
640.2347426039938 [12 15 18 11  9 10 16  3  7 17 19  4  2  1  6  5  8 13  0 14]
```



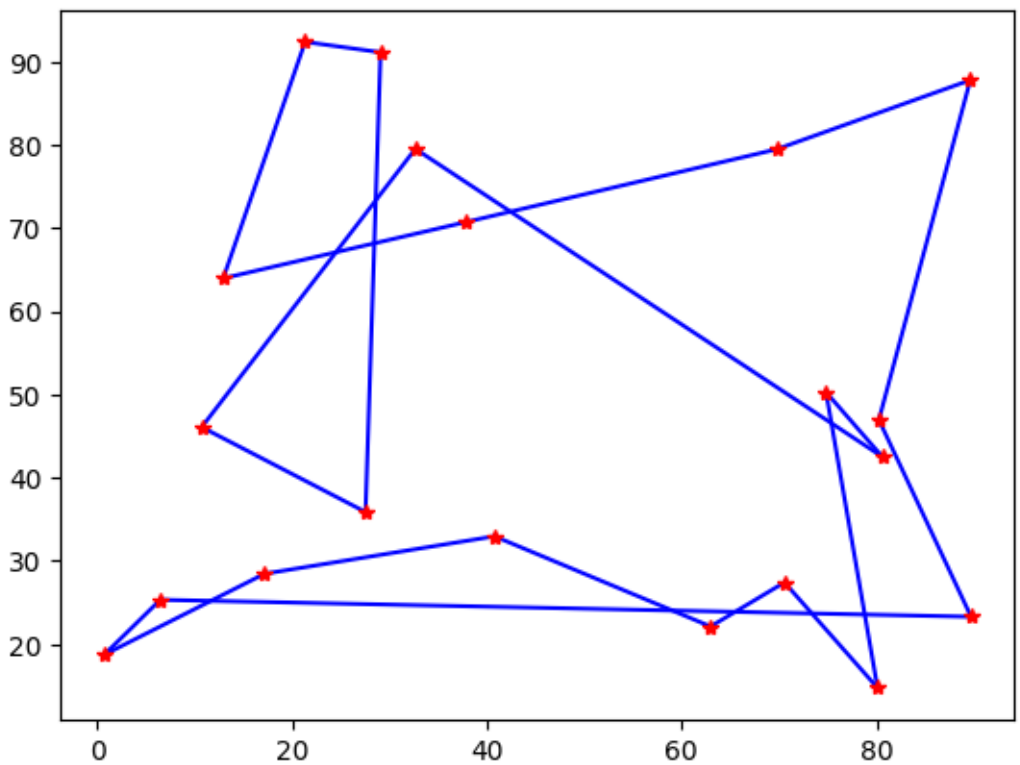

```
611.1601890589526 [17  4  2  1  8 10  5 11 18 19  0 14  7 12 15  3  9 16  6 13]
```



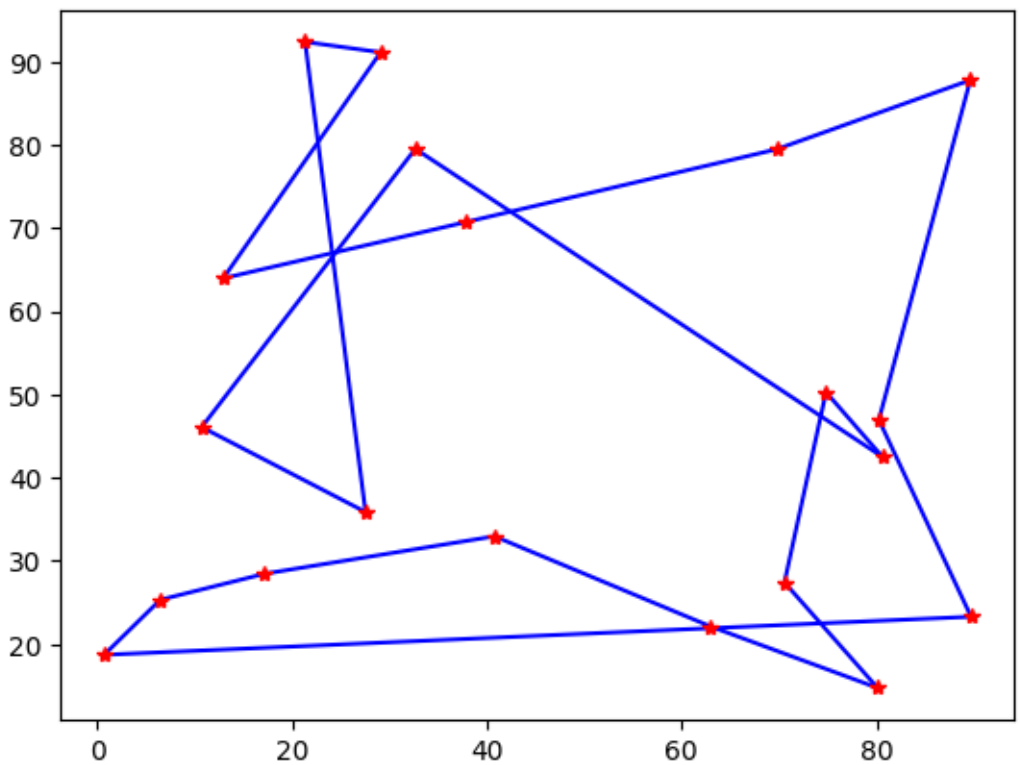

```
594.9221569642746 [17  4  2  1  8 10  5 18 11 19  0 14  7 12 15  3  9 16  6 13]
```



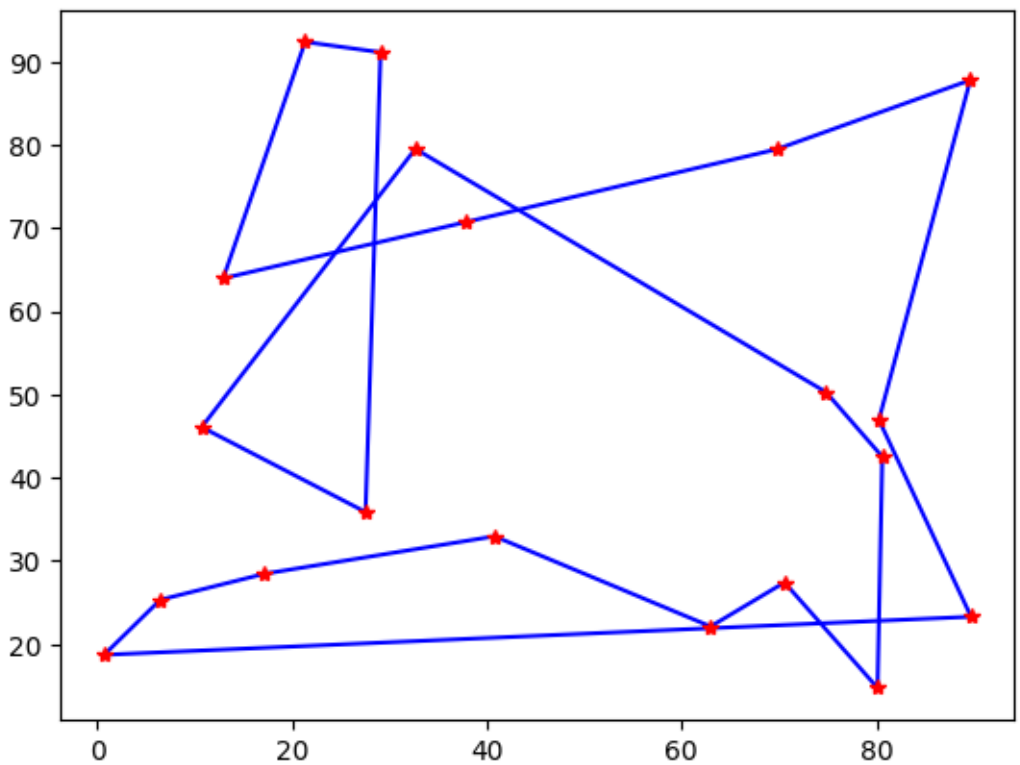

```
591.3843698570206 [17  2  4  1  8 10  5 18 11 19  0 14  7 12 15  3  9 16  6 13]
```



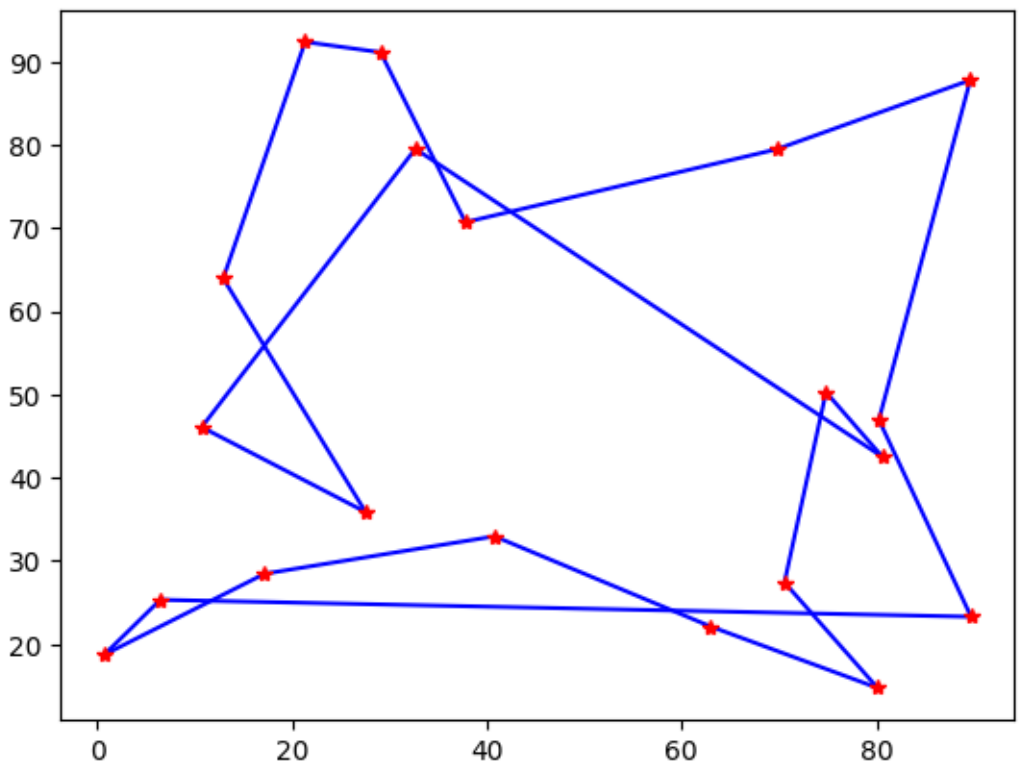

```
589.5283399005345 [17  4  2  1  8 10  5 18 11  0 19 14  7 12  3 15  9 16  6 13]
```



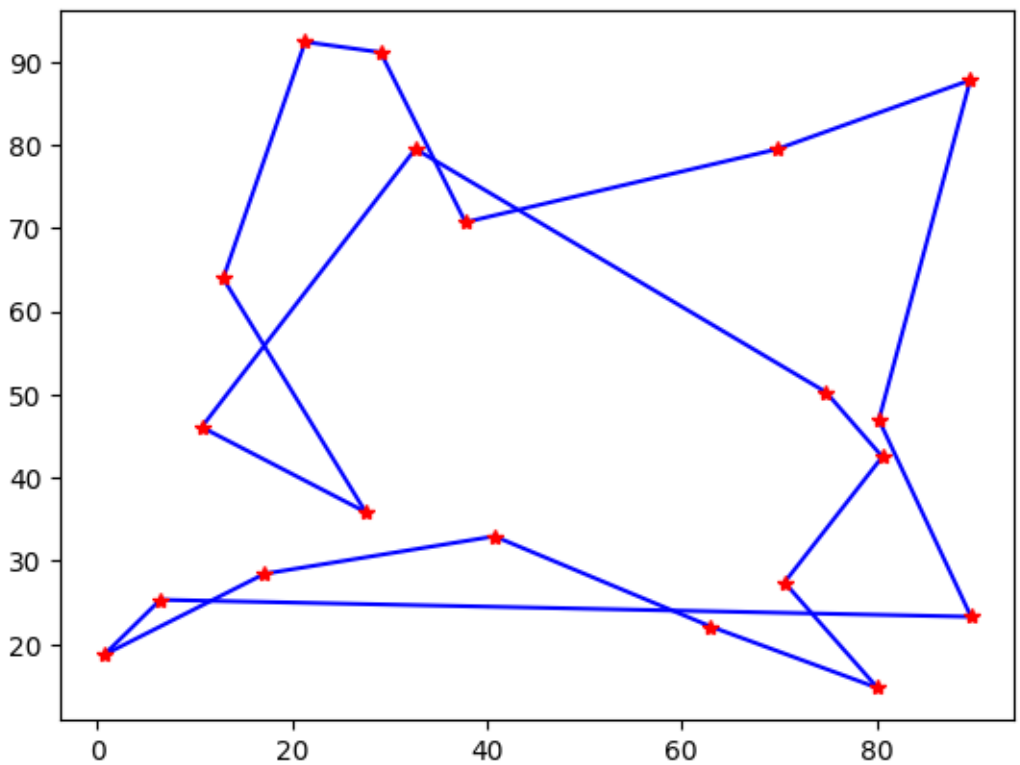

```
571.8543824008989 [17  2  4  1  8 10  5 18 11  0 19 14  7 12 15  3 16  9  6 13]
```



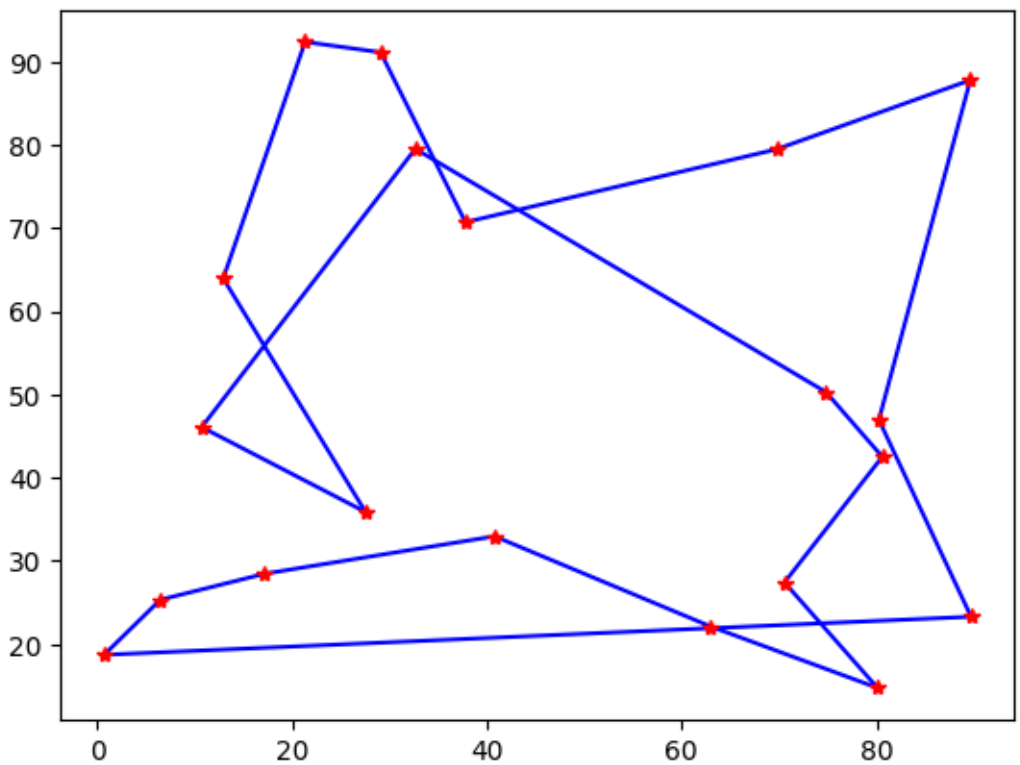

```
560.9386898536318 [17  1  4  2  8 10  5 18 11 19  0 14  7 12  3 15  9 16  6 13]
```




```
546.5576630539643 [17  1  4  2  8 10  5 18 11 19  0 14  7 12  3 15 16  9  6 13]
```

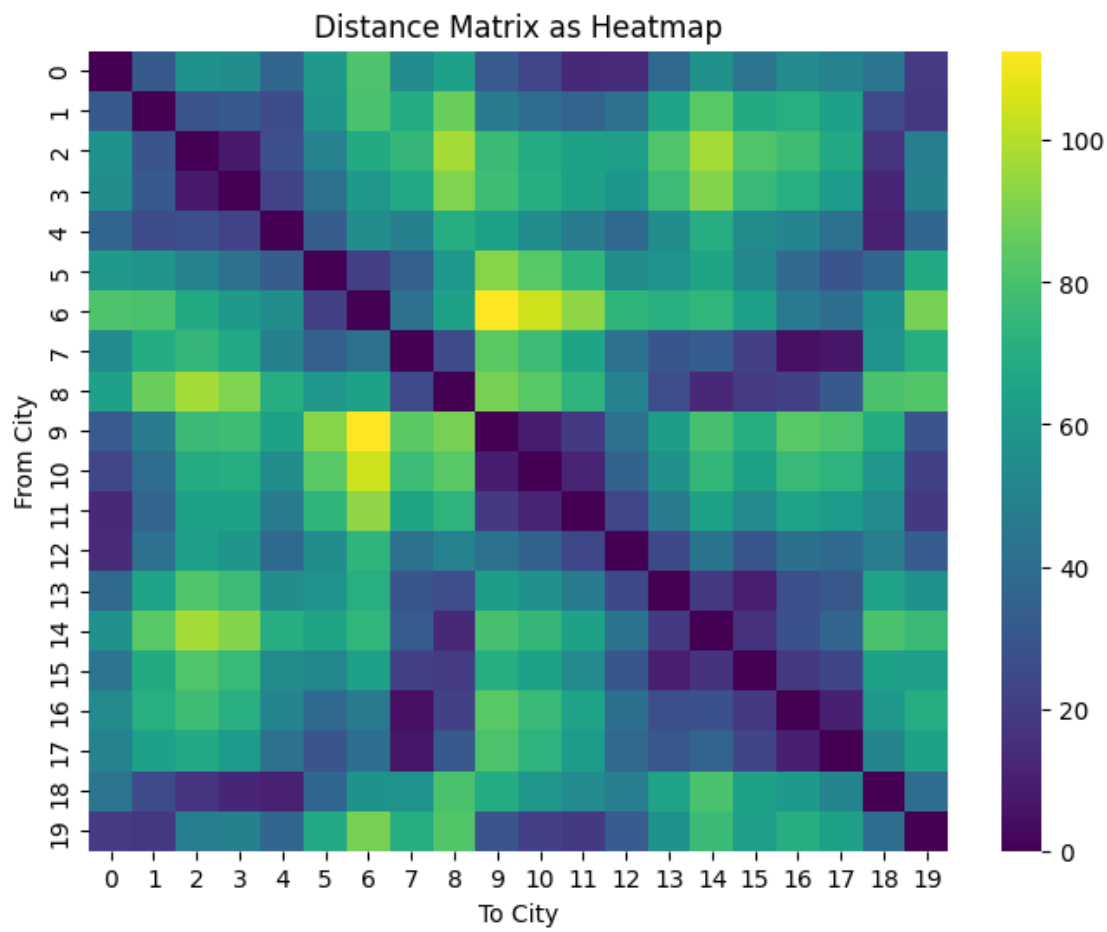



```
544.4317772046725 [17  1  4  2  8 10  5 18 11  0 19 14  7 12  3 15 16  9  6 13]
```



	0	1	2	3	4	5	6	7	8	9	10	11	\
0	0.0	31.7	56.9	55.3	36.3	60.6	80.9	53.8	63.4	31.8	23.6	12.8	
1	31.7	0.0	29.6	31.6	25.8	58.9	80.3	69.4	86.9	46.9	39.2	35.8	
2	56.9	29.6	0.0	7.8	27.2	50.1	68.4	74.4	97.2	76.5	68.7	64.1	
3	55.3	31.6	7.8	0.0	22.2	42.3	60.6	67.6	91.0	77.8	69.6	63.8	
4	36.3	25.8	27.2	22.2	0.0	33.1	54.6	48.6	70.3	63.9	55.1	47.0	
5	60.6	58.9	50.1	42.3	33.1	0.0	21.6	34.2	59.6	91.9	83.2	73.2	
6	80.9	80.3	68.4	60.6	54.6	21.6	0.0	41.9	64.5	112.6	104.0	93.6	
7	53.8	69.4	74.4	67.6	48.6	34.2	41.9	0.0	25.5	84.4	76.9	65.7	
8	63.4	86.9	97.2	91.0	70.3	59.6	64.5	25.5	0.0	89.1	83.2	72.7	
9	31.8	46.9	76.5	77.8	63.9	91.9	112.6	84.4	89.1	0.0	8.8	19.1	
10	23.6	39.2	68.7	69.6	55.1	83.2	104.0	76.9	83.2	8.8	0.0	11.1	
11	12.8	35.8	64.1	63.8	47.0	73.2	93.6	65.7	72.7	19.1	11.1	0.0	
12	13.6	41.7	62.5	59.3	37.9	54.8	73.4	41.9	49.9	42.5	35.1	24.0	
13	38.0	65.2	81.7	76.9	54.7	57.8	70.9	30.3	26.8	62.3	56.6	46.2	
14	56.6	83.3	97.4	91.9	70.2	65.6	73.7	32.2	12.9	79.5	74.4	64.4	
15	43.8	68.2	81.5	76.0	54.3	52.1	63.3	21.8	19.6	70.4	64.1	53.4	
16	53.4	71.0	77.4	70.8	51.2	38.5	46.2	4.5	21.3	83.3	76.1	64.9	
17	49.4	63.4	68.0	61.3	42.3	29.6	40.3	6.4	30.8	80.5	72.7	61.6	
18	44.0	25.1	17.1	12.1	10.2	37.1	57.6	57.7	80.1	68.7	60.2	53.4	
19	19.7	18.0	47.5	48.7	36.6	67.8	89.2	69.5	82.2	29.1	21.2	18.8	

	12	13	14	15	16	17	18	19
0	13.6	38.0	56.6	43.8	53.4	49.4	44.0	19.7
1	41.7	65.2	83.3	68.2	71.0	63.4	25.1	18.0
2	62.5	81.7	97.4	81.5	77.4	68.0	17.1	47.5
3	59.3	76.9	91.9	76.0	70.8	61.3	12.1	48.7
4	37.9	54.7	70.2	54.3	51.2	42.3	10.2	36.6
5	54.8	57.8	65.6	52.1	38.5	29.6	37.1	67.8
6	73.4	70.9	73.7	63.3	46.2	40.3	57.6	89.2
7	41.9	30.3	32.2	21.8	4.5	6.4	57.7	69.5
8	49.9	26.8	12.9	19.6	21.3	30.8	80.1	82.2
9	42.5	62.3	79.5	70.4	83.3	80.5	68.7	29.1
10	35.1	56.6	74.4	64.1	76.1	72.7	60.2	21.2
11	24.0	46.2	64.4	53.4	64.9	61.6	53.4	18.8
12	0.0	24.7	43.3	30.3	40.9	38.2	47.3	32.8
13	24.7	0.0	18.7	9.2	27.0	30.6	64.9	57.5
14	43.3	18.7	0.0	15.9	27.8	35.9	80.3	76.1
15	30.3	9.2	15.9	0.0	18.1	23.3	64.4	62.6
16	40.9	27.0	27.8	18.1	0.0	9.7	60.5	69.9
17	38.2	30.6	35.9	23.3	9.7	0.0	51.3	64.2
18	47.3	64.9	80.3	64.4	60.5	51.3	0.0	40.0
19	32.8	57.5	76.1	62.6	69.9	64.2	40.0	0.0



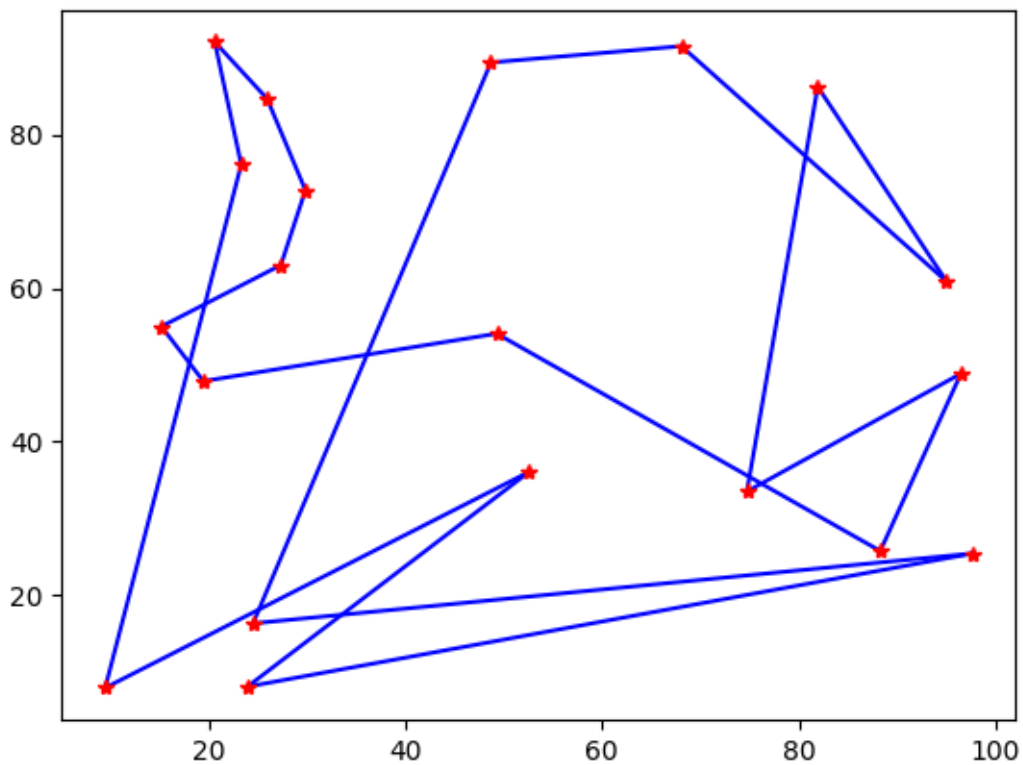
1.9 Task 10

[10]:
```
# ADDED INVERSION MUTATION
from random import randint
from random import sample
from random import random

# Creating the cities
N = 20
xy = 100 * np.random.rand(N,2)

# Computing distance matrix
d = np.empty((N,N))
for i in range(N):
    for j in range(N):
        # Calculate Euclidean distance
        dx = xy[i,0] - xy[j,0]
        dy = xy[i,1] - xy[j,1]
```

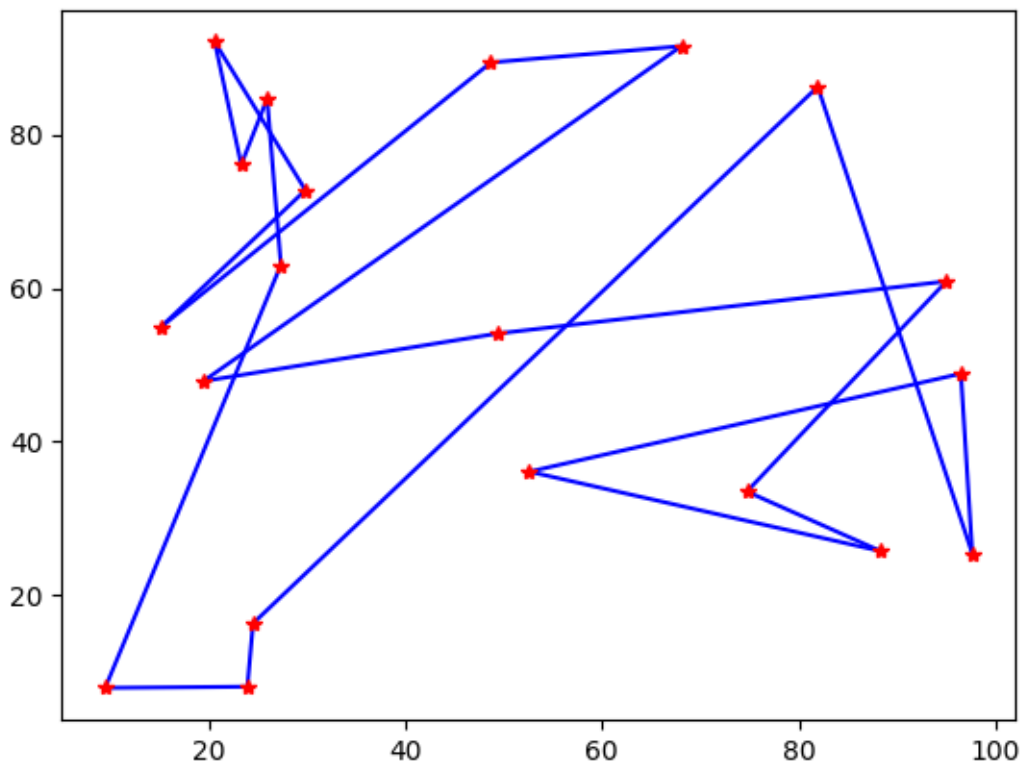
```
        d[i,j] = np.sqrt(dx**2 + dy**2)

# Initialize Global Parameters
iterations = 1_000_000
# N = Genome length is determined by cities
M = 1000         # Population size
crossover_prob = 0.5
mutation_rate = 1 / N

def fitness_function(priorities):
    permutation = np.argsort(priorities)[::-1]
    distance = tsp_simulator(permutation)
    fitness = 1.0 / distance # So the higher the distance the worse the fitness
    return fitness

def random_individual(N):
    return np.random.rand(N)

def random_population(M, N):
    return [random_individual(N) for _ in range(M)]

def eval_fitness(population):
    return [fitness_function(ind) for ind in population]

def tournament():
    i, j = sample(range(M), 2)
    return i if fitnesses[i] >= fitnesses[j] else j

def negative_tournament():
    i, j = sample(range(M), 2)
    # Returns the index of the weaker individual to be replaced
    return i if fitnesses[i] <= fitnesses[j] else j

def mutate():
    idx = tournament()
    parent = population[idx]
    offset = 0.05 * (np.random.rand(N) - 0.5)
    offspring = parent + offset
    return offspring

def invert():
    idx = tournament()
    parent = population[idx]
    r1, r2 = sample(range(N), 2)
    i_start_invert = min(r1,r2)
    i_end_invert = max(r1,r2)
    offspring = parent.copy()
```

```
    offspring[i_start_invert:i_end_invert] = offspring[i_start_invert:
 ↪i_end_invert][::-1]
    return offspring

def crossover():
    idx1 = tournament()
    idx2 = tournament()
    parent1, parent2 = population[idx1], population[idx2]

    # line crossover
    alpha = np.random.rand()
    offspring = alpha * parent1 + (1 - alpha) * parent2
    return offspring

population = random_population(M, N)
fitnesses = eval_fitness(population)

# Track the best individual
best_idx = fitnesses.index(max(fitnesses))
best_genome = np.argsort(population[best_idx])[::-1]
best_fit = fitnesses[best_idx]
for i in range(iterations):
    if random() < crossover_prob:
        offspring = crossover()
    else:
        if random() < 0.5: # old mutation
            offspring = mutate()
        else: # Inversion mutation
            offspring = invert()

    f = fitness_function(offspring)

    k = negative_tournament()
    population[k] = offspring
    fitnesses[k] = f

    if f > best_fit:
        best_fit = f
        best_genome = np.argsort(offspring)[::-1]
        print(1/best_fit, best_genome)
        plot_tour(best_genome)

permuted_d = d[best_genome][:,best_genome]
visualise(permuted_d)
```

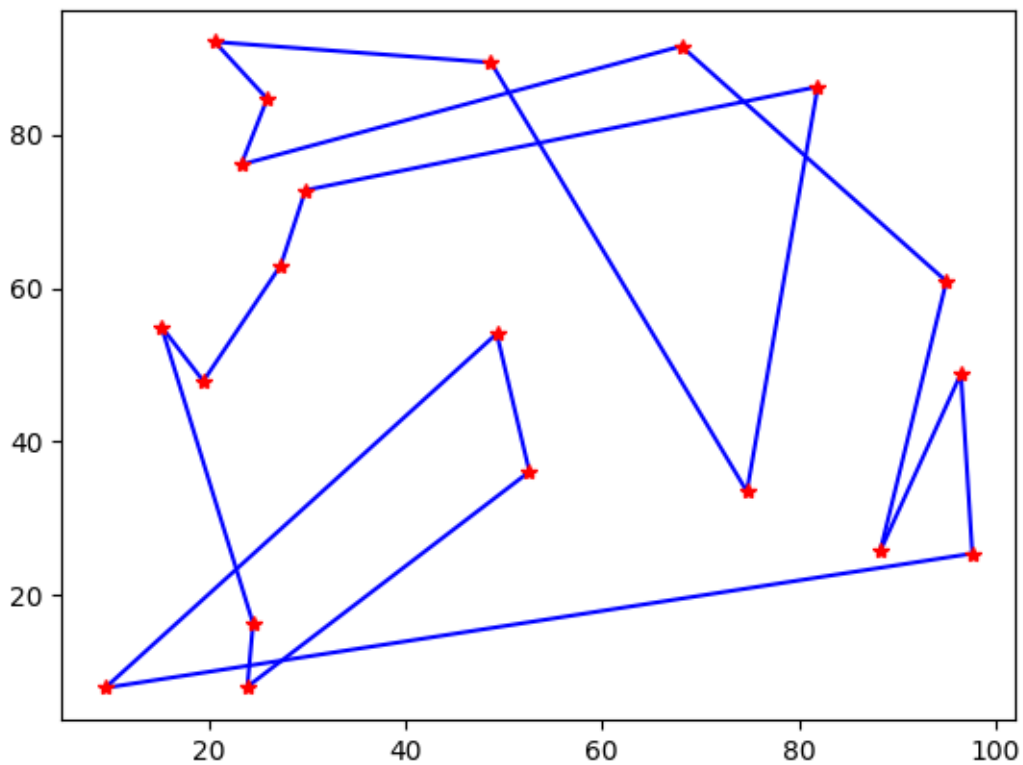
```
729.8353877905879 [ 0 14 19  3 10  2  9  7 15  1 16  6  8 17 12 18 13  5 11  4]
```



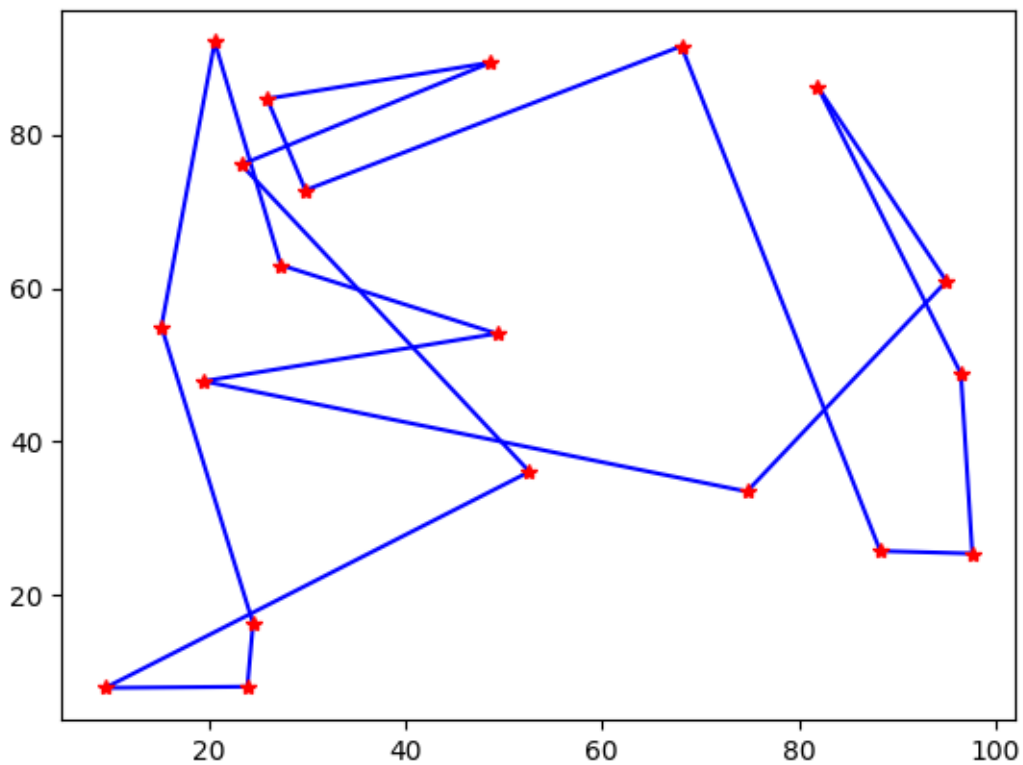

```
690.4372636742248 [ 7  2 13  5 10  3 11  0 19  8 14 12  4 18 17  6  9 15 16  1]
```



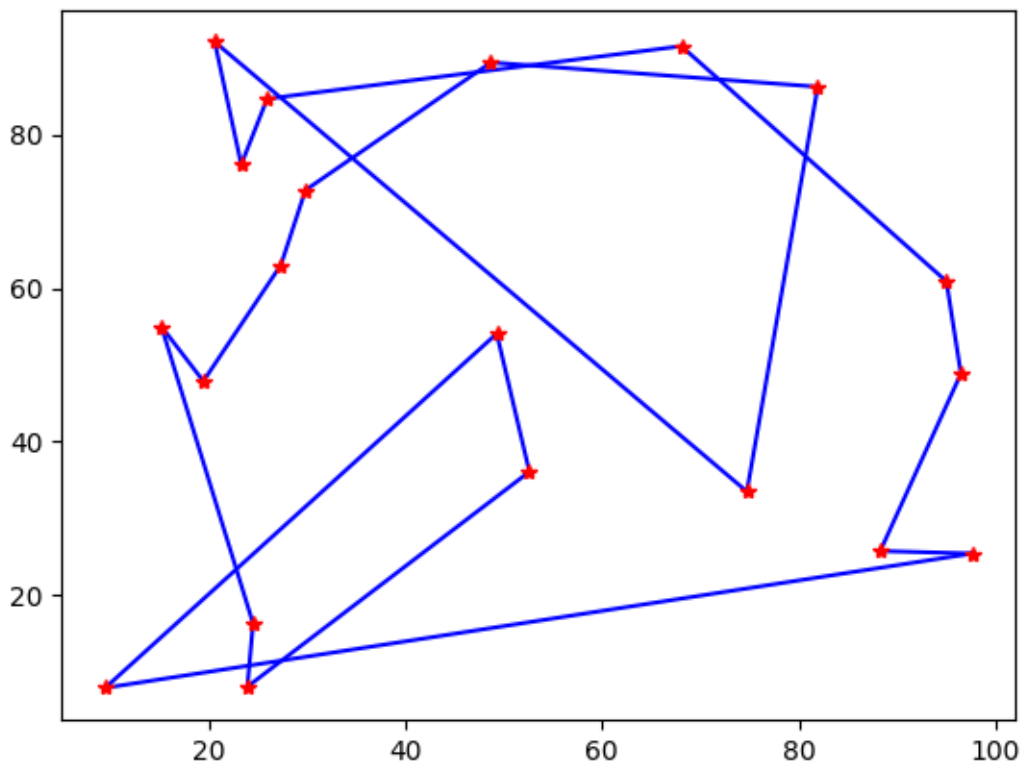

```
679.2466427192192 [ 5 11 19 14 12  6  3  8 17 18  2 10  9  7  4  0 13  1 15 16]
```



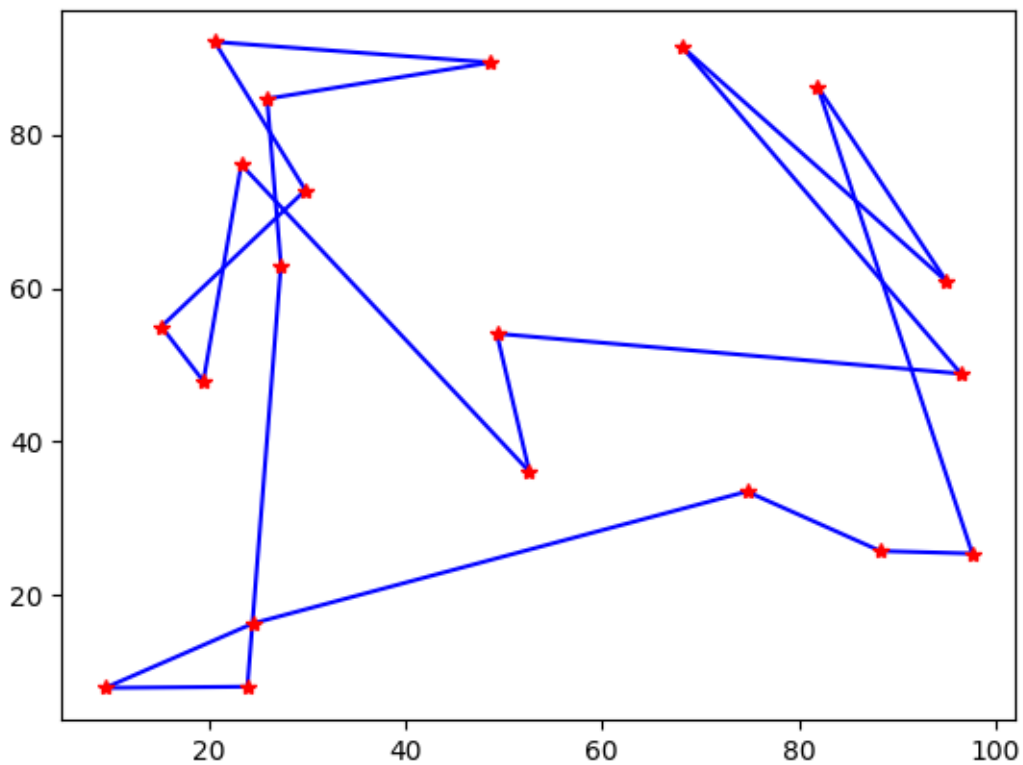

```
653.2730593870059 [ 0 11  4 14 12 19  5  7 15 13 16  8  6 17 18  2  1  9  3 10]
```



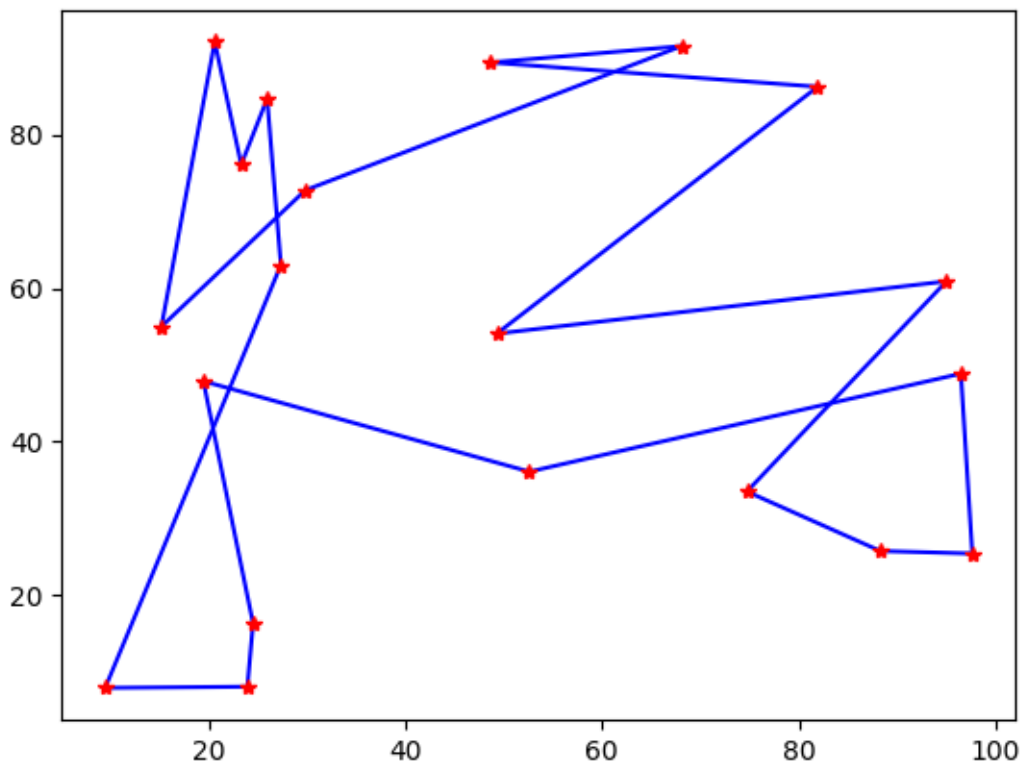

```
638.5435044480951 [ 5 11 14 19 12  6  3  8 17 18  2 10  9  7 13  4  0  1 16 15]
```



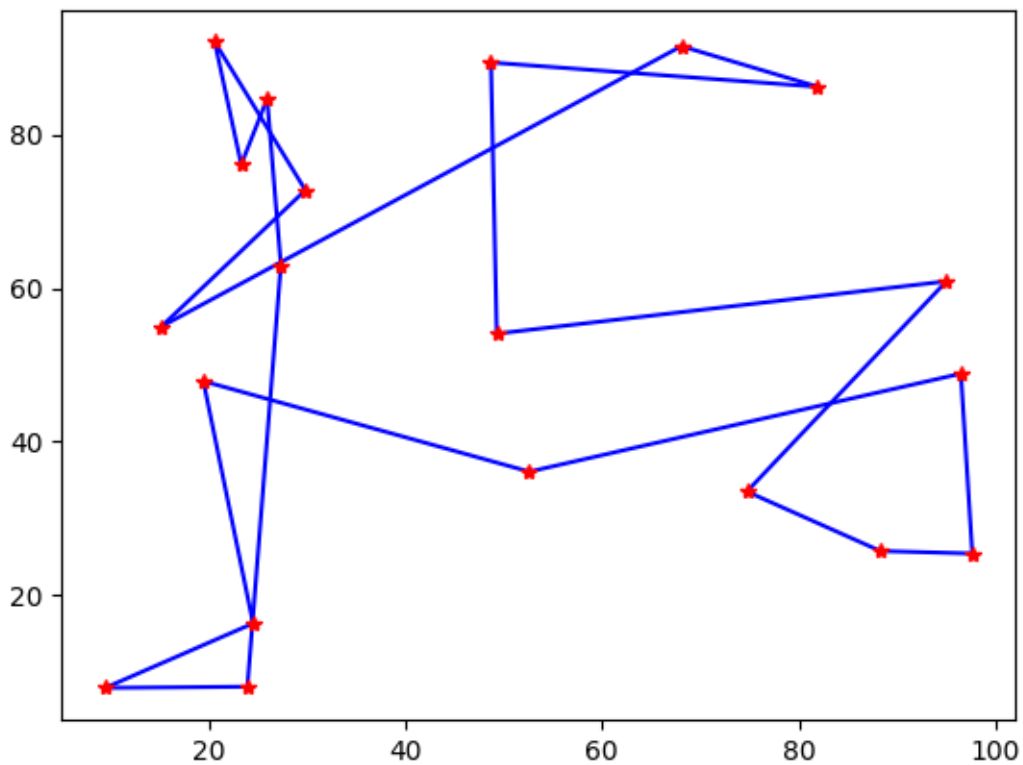

```
617.599955868463 [ 4 12 19  0 18  6 17  9 15 13  1  7  2 10 16  8  3 14  5 11]
```



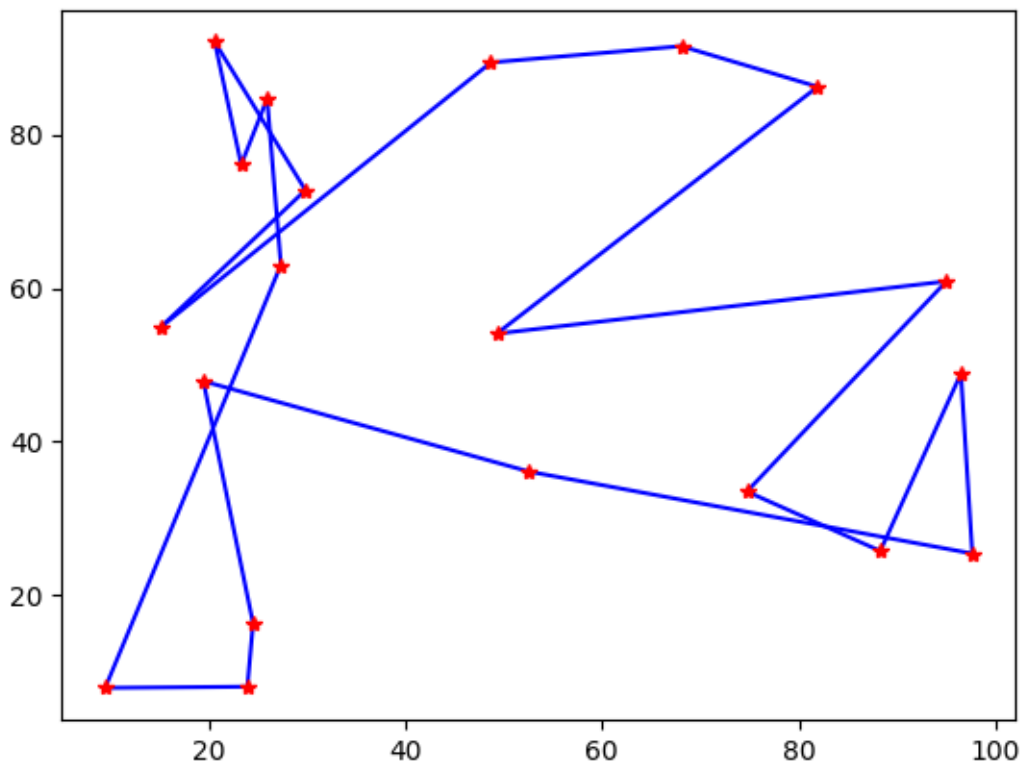

```
570.6346130946554 [ 2  7  5 13  4  3 11  0 19 12 14  8 10 18 17  6  9 15 16  1]
```



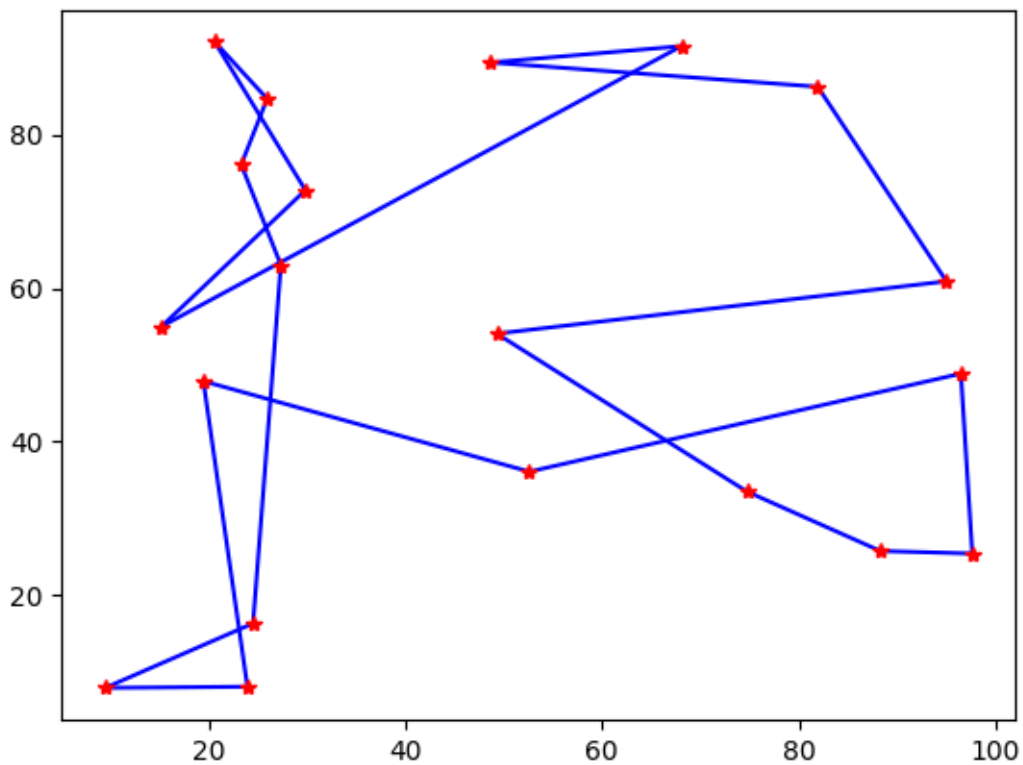

```
567.1933349714343 [ 7  2  5  4 13  3 11  0 19 12 14  8 10 18  6 17  9 15 16  1]
```



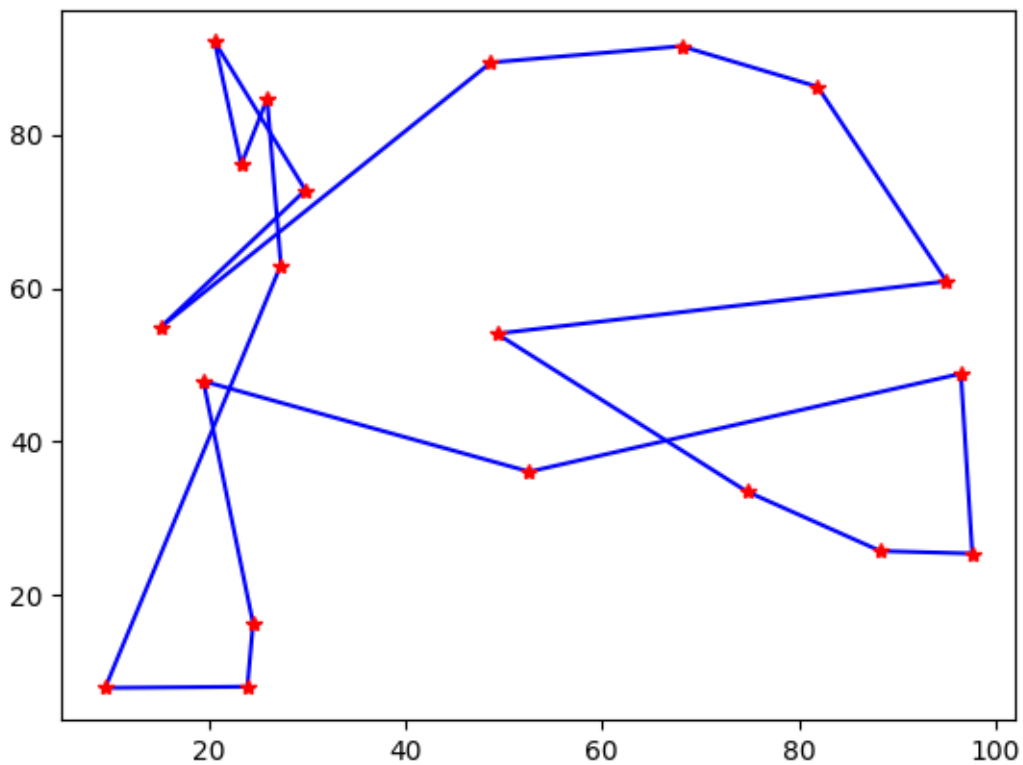

```
557.1918423280584 [ 7  2 13  5  4  3 11  0 19 14 12  8 10 18 17  6  9 15 16  1]
```



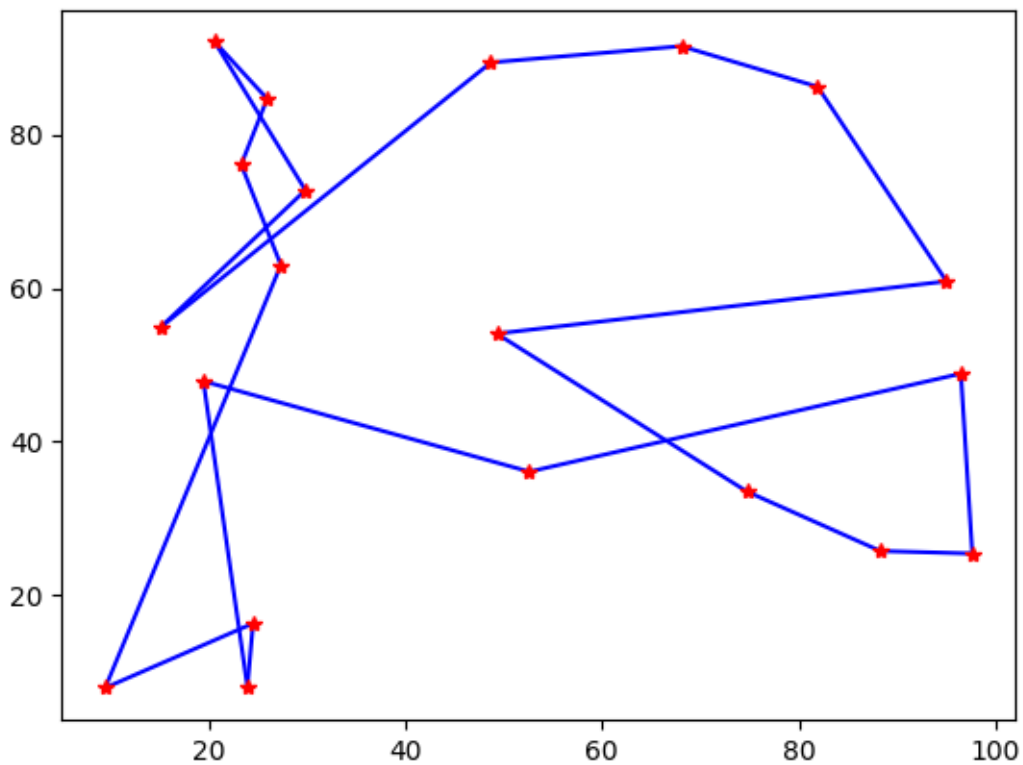

```
548.406040914478 [ 7  2  5 13  4 11  3  0 19 12 14  8 10 17  6 18  9 16 15  1]
```



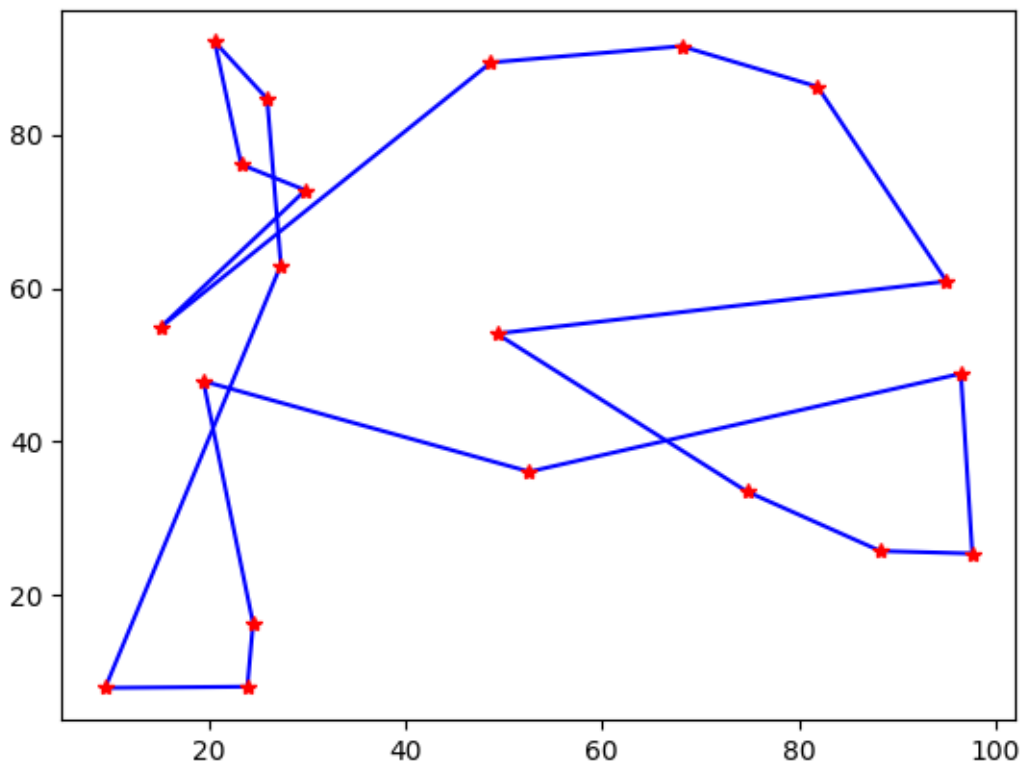

```
522.8047675490659 [ 7  2 13  5  4 11  3  0 19 12 14  8 10 18 17  6  9 15 16  1]
```



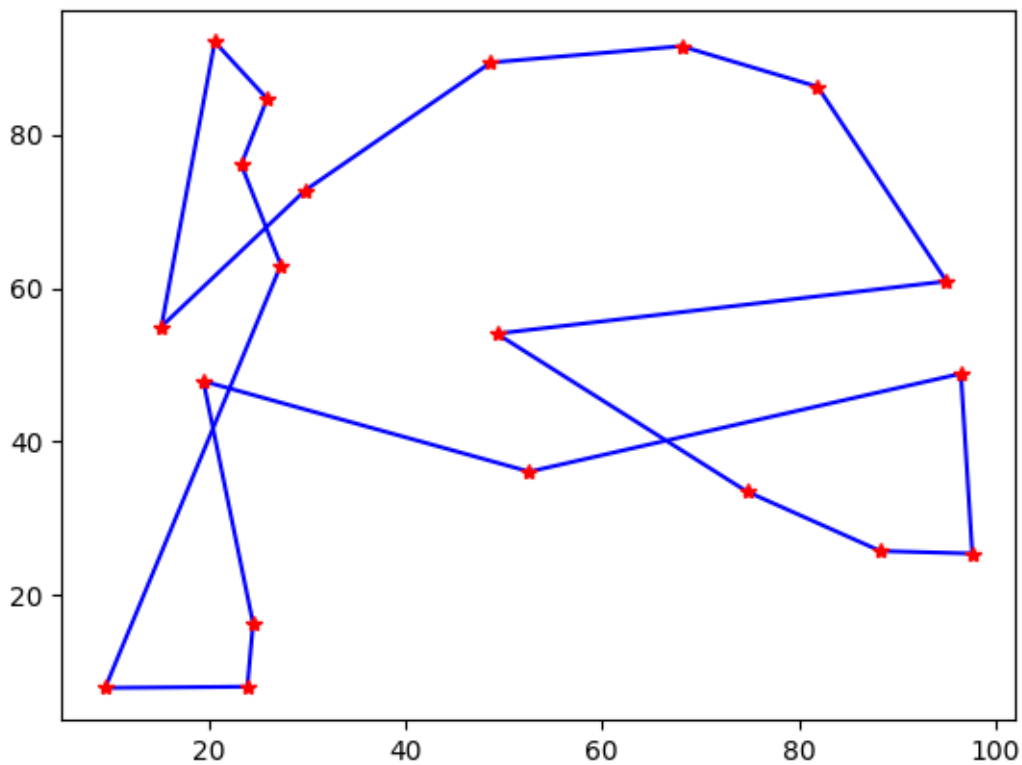

```
518.5709177360602 [ 7  2 13  5  4 11  3  0 19 12 14  8 10 17 18  6  9 16 15  1]
```



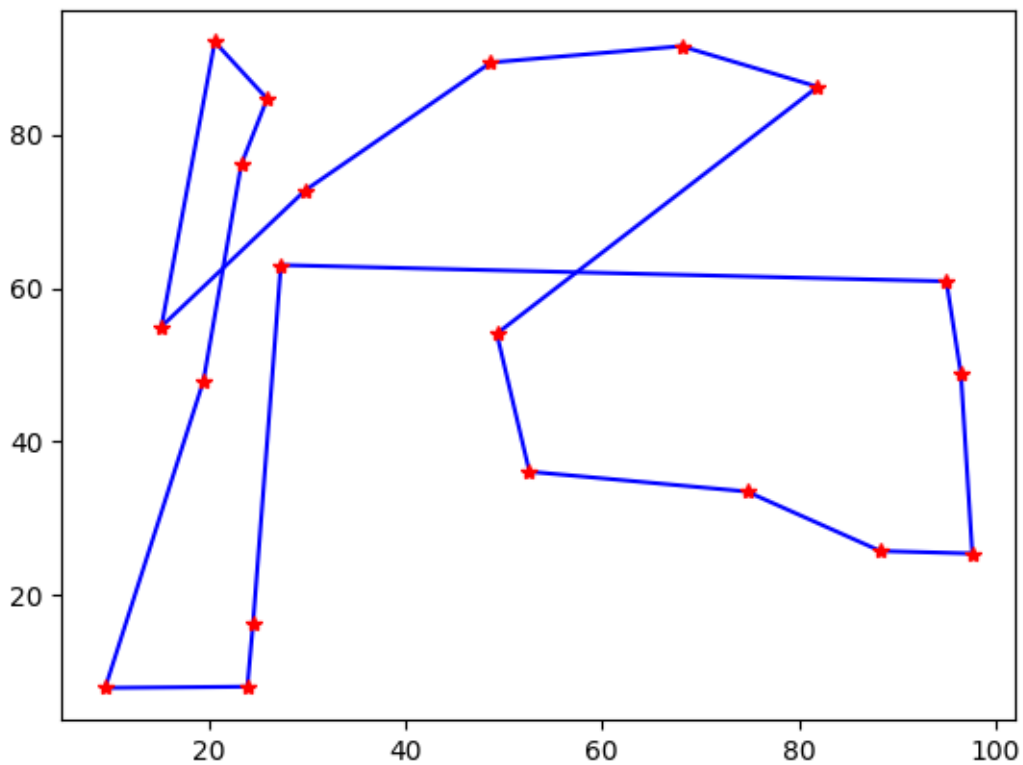

```
508.89384854263915 [ 7  2 13  5  4 11  3  0 19 12 14  8 10 18 17  6  9 15  1 16]
```



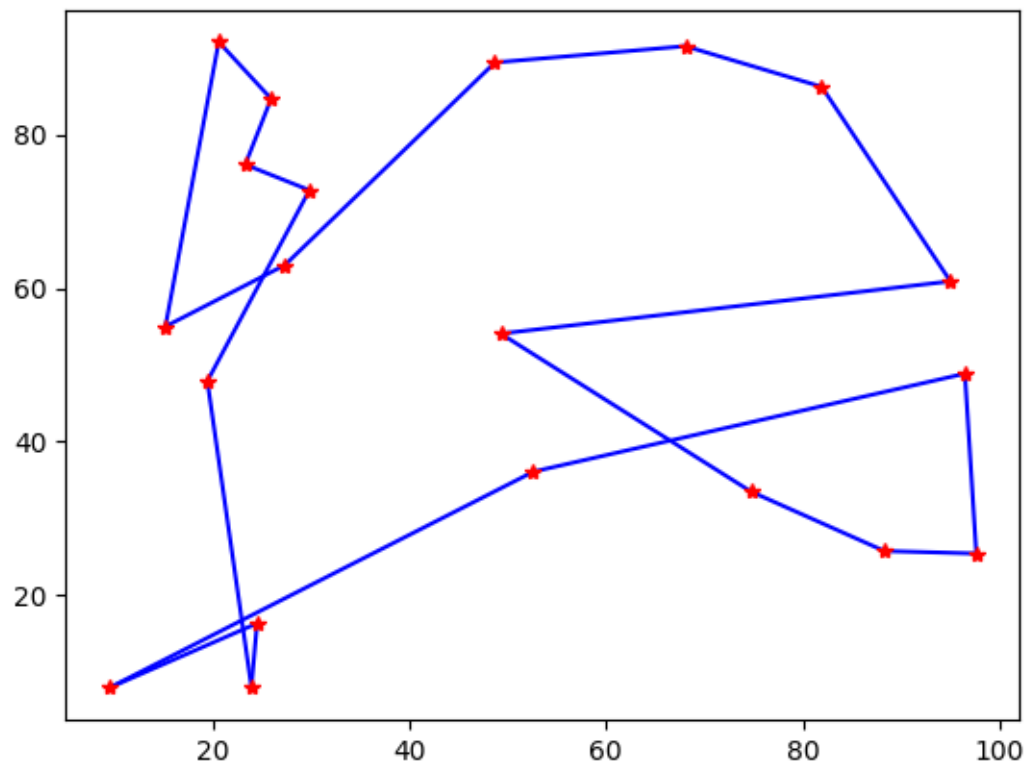

```
500.9179592153351 [ 2  7 13  5  4 11  3  0 19 12 14  8 10 18 17  6  9 16 15  1]
```



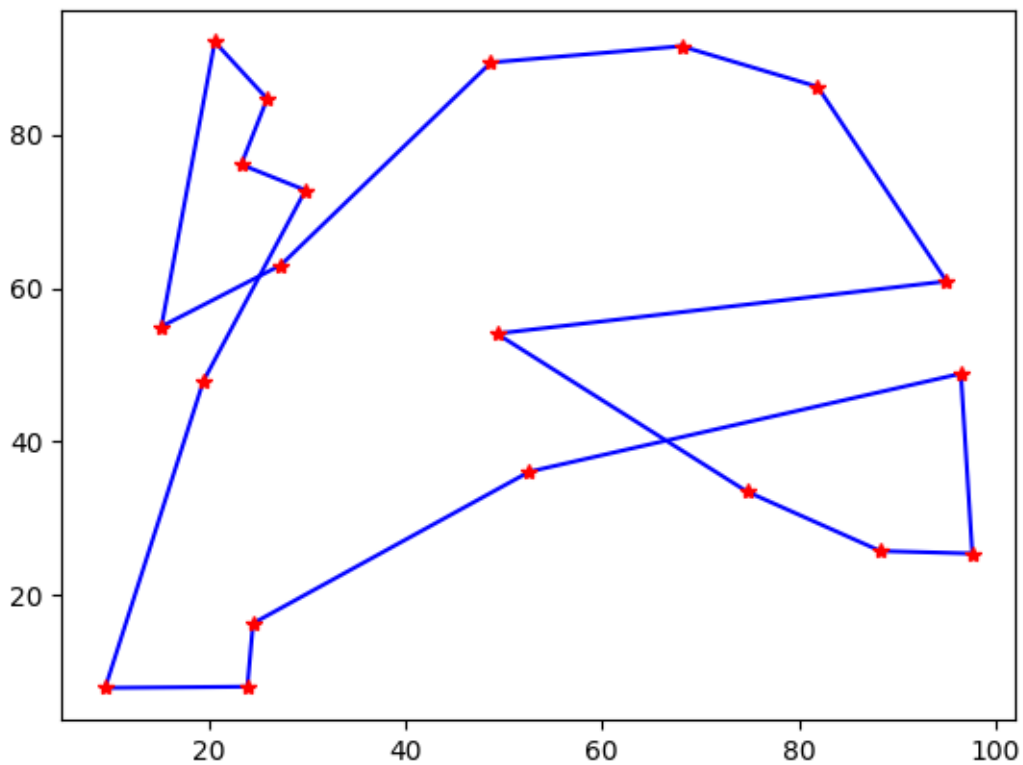

```
491.6344182163741 [ 2  7 13  5  4  3  8  0 19 12 14 11  9 18 17  6 10 16 15  1]
```



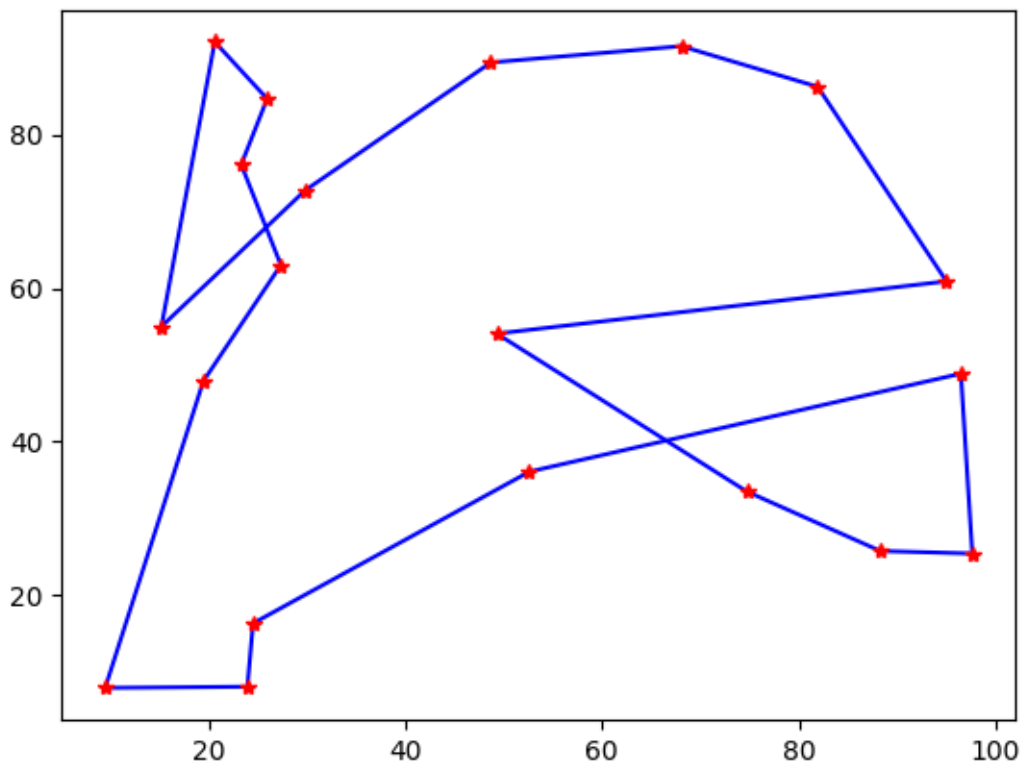

```
491.1302555080038 [ 2  9 13  5  4 11  3  0 19 12 14  8  6 18 17 10  7 16 15  1]
```



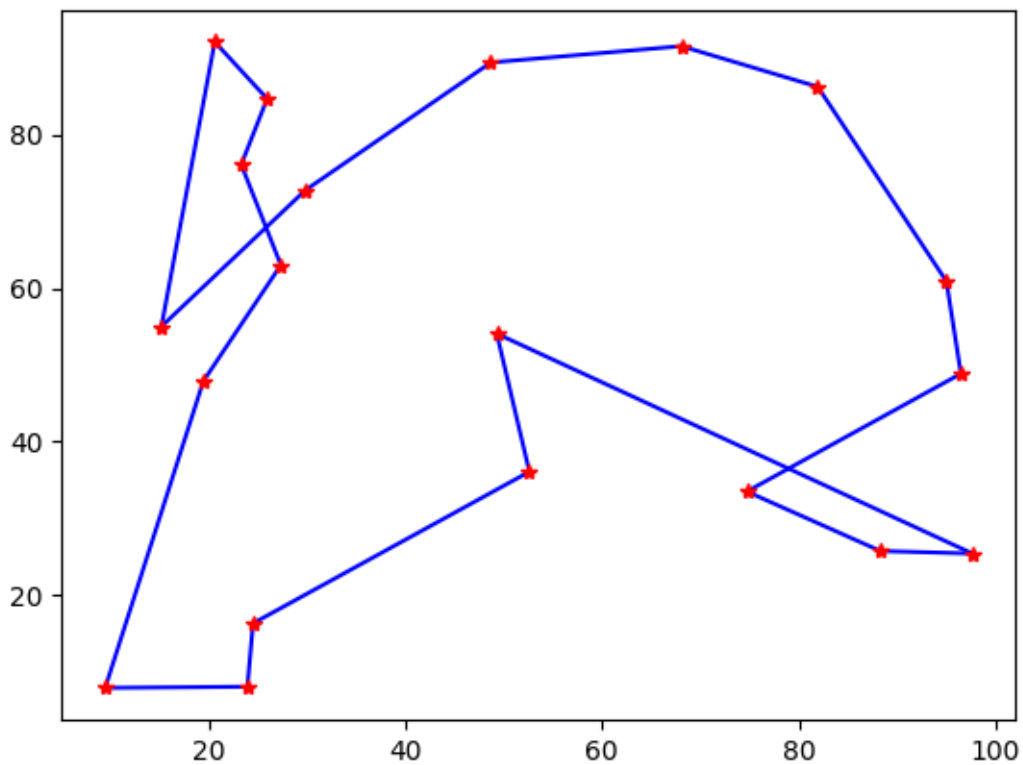

```
472.34260346662114 [ 2  9 13  5  4 11  3  0 19 12 14  8 18 17  6 10  7 16 15  1]
```



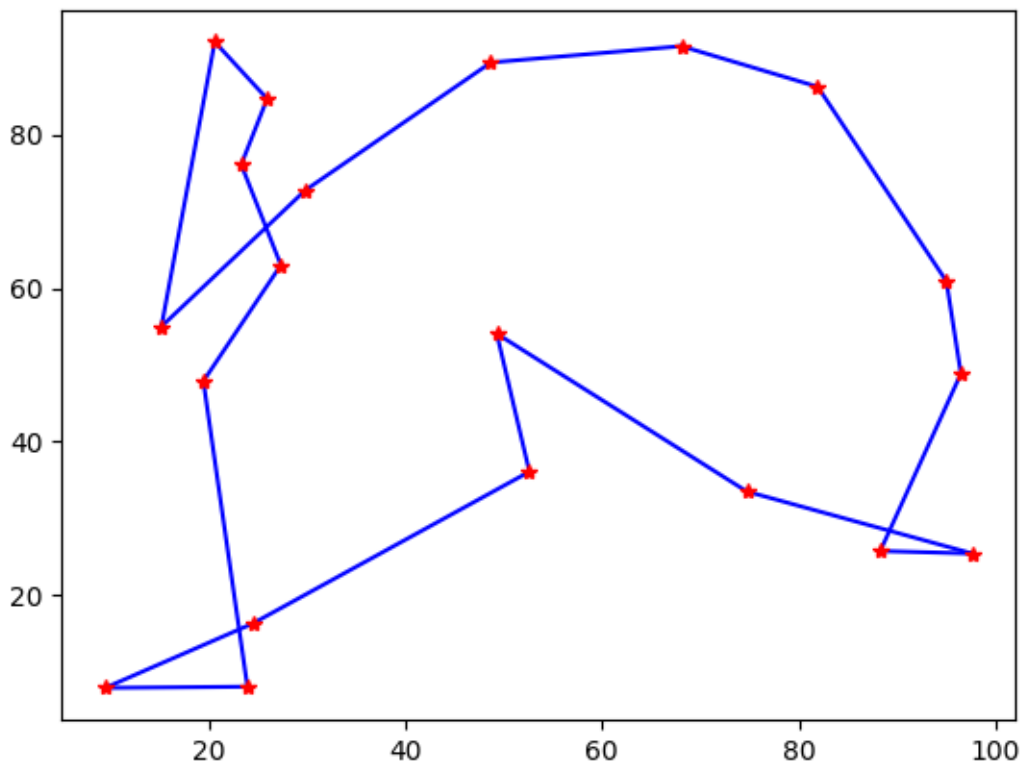

```
468.47959033632026 [ 2  7 13  5  4 11  3  0 19 12 14  8 18 17  6 10  9 16 15  1]
```



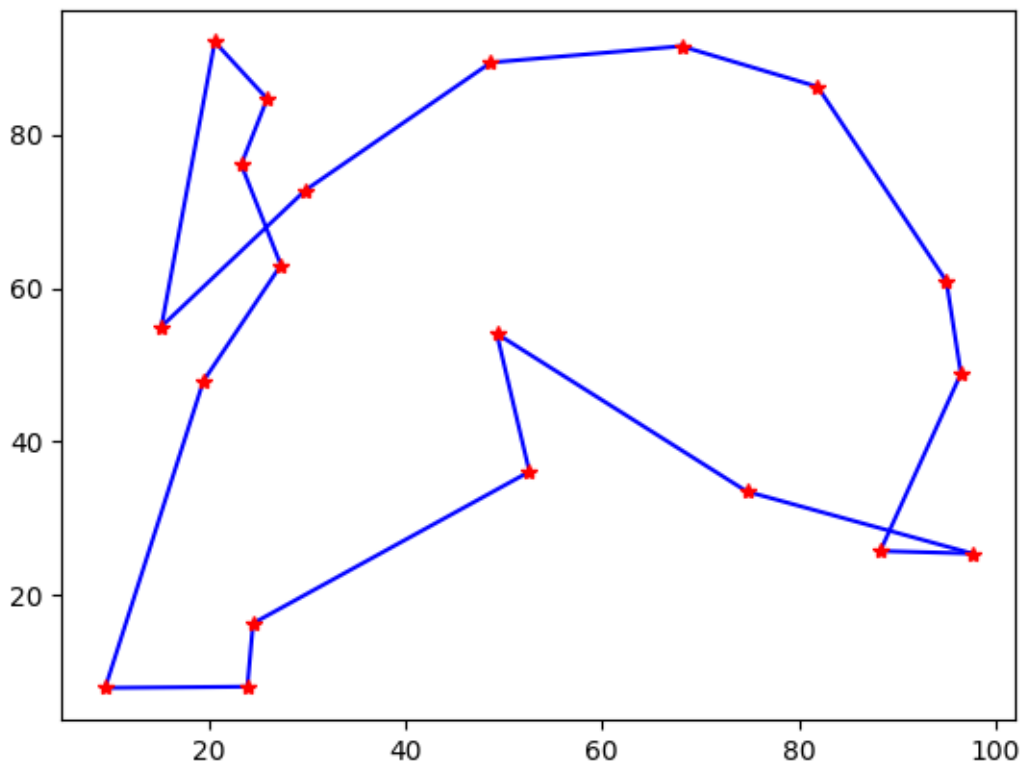

```
433.6249258297317 [ 2  7 13  5  4 11 14  0 19 12  3  8 18 17  6 10  9 16 15  1]
```



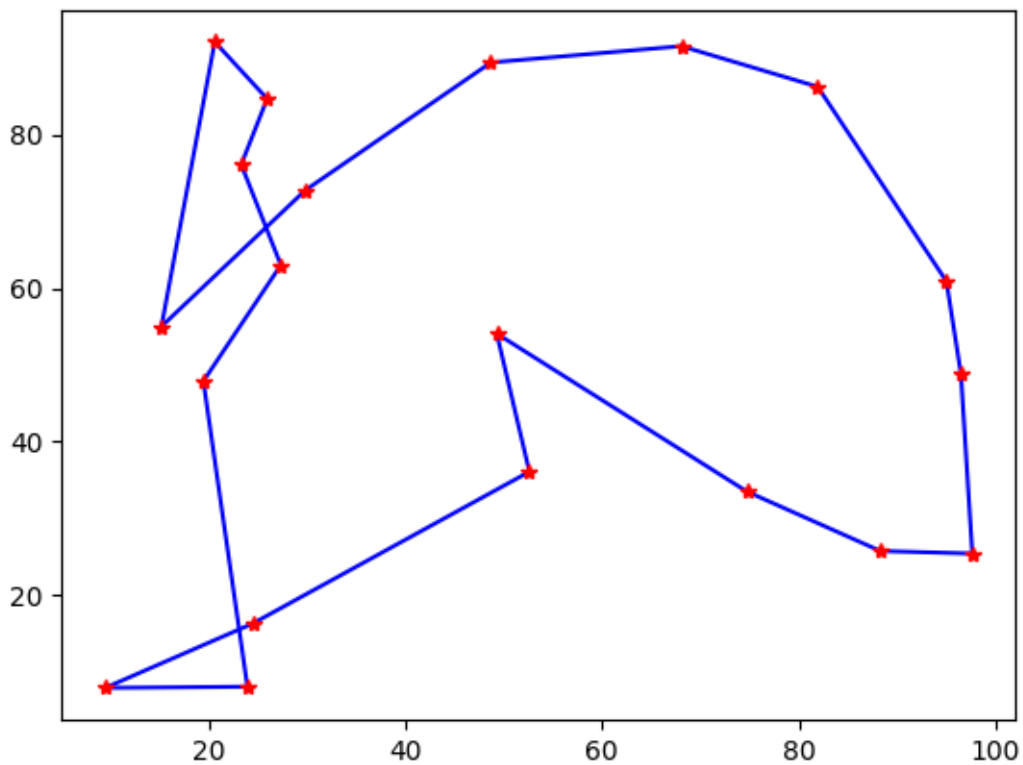

```
424.5206645401821 [ 2  7 13  5  4 11 14 19 12  0  3  8 18  6 17 10  9 16 15  1]
```



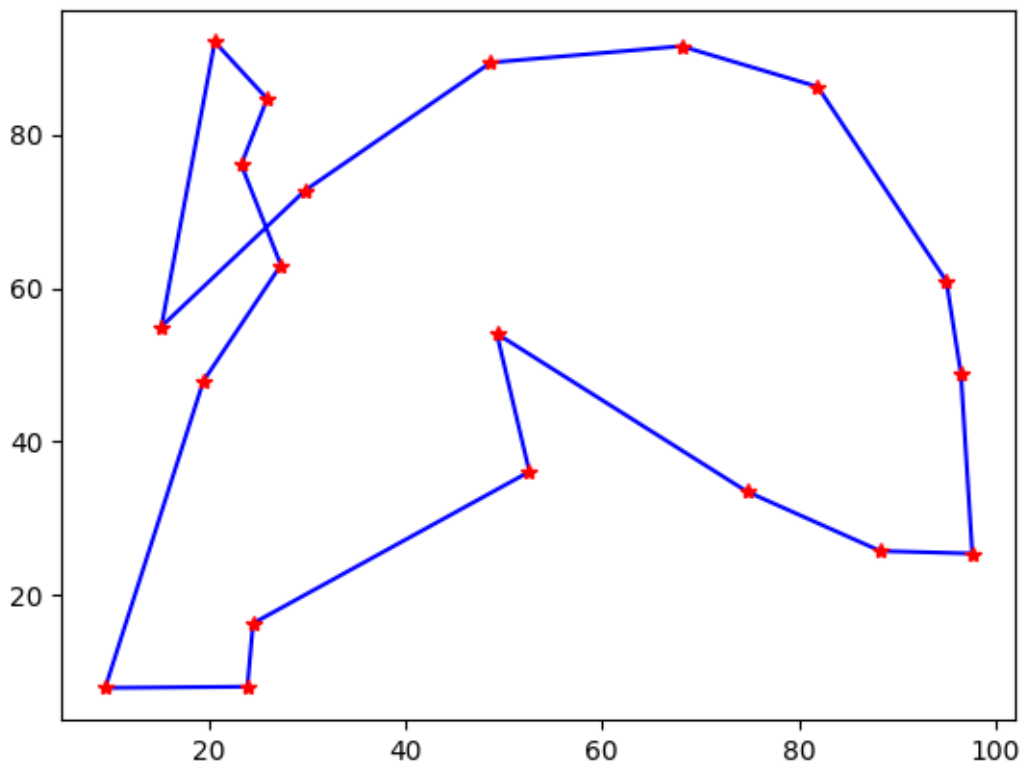

```
416.6939918612172 [ 2  7 13  5  4 11 14 19 12  0  3  8 18 17  6 10  9 16 15  1]
```



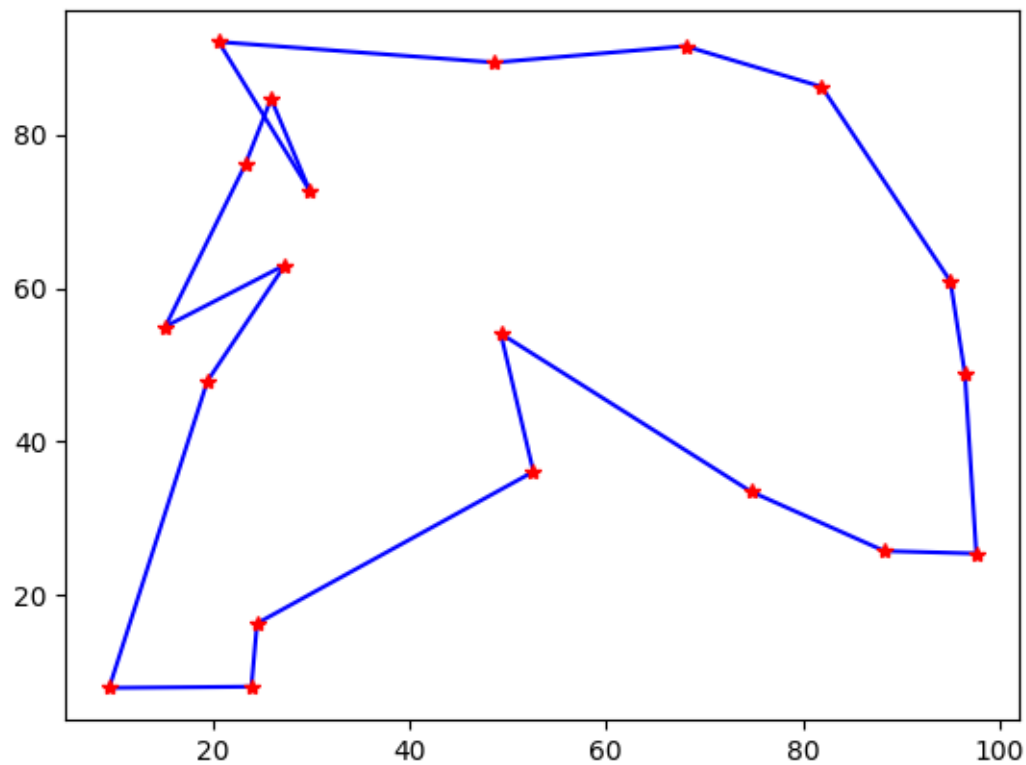

```
414.78477949938144 [ 2  7 13  5  4 11 14 12 19  0  3  8 18  6 17 10  9 16 15  1]
```



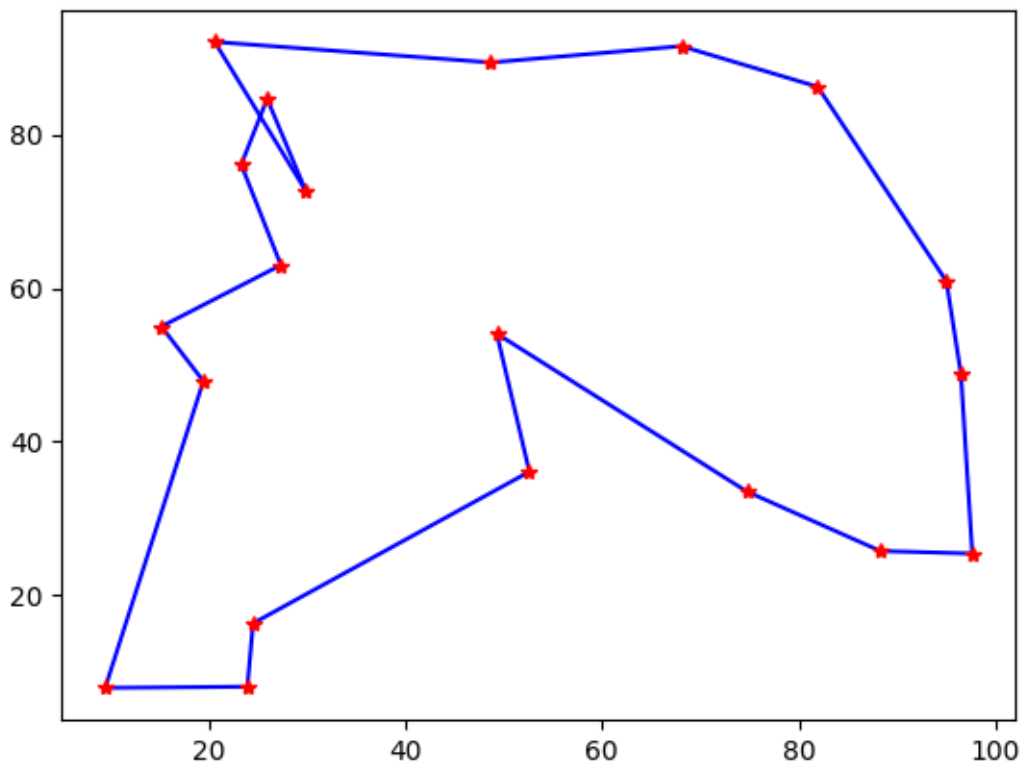

```
406.9581068204166 [ 2  7 13  5  4 11 14 12 19  0  3  8 18 17  6 10  9 16 15  1]
```



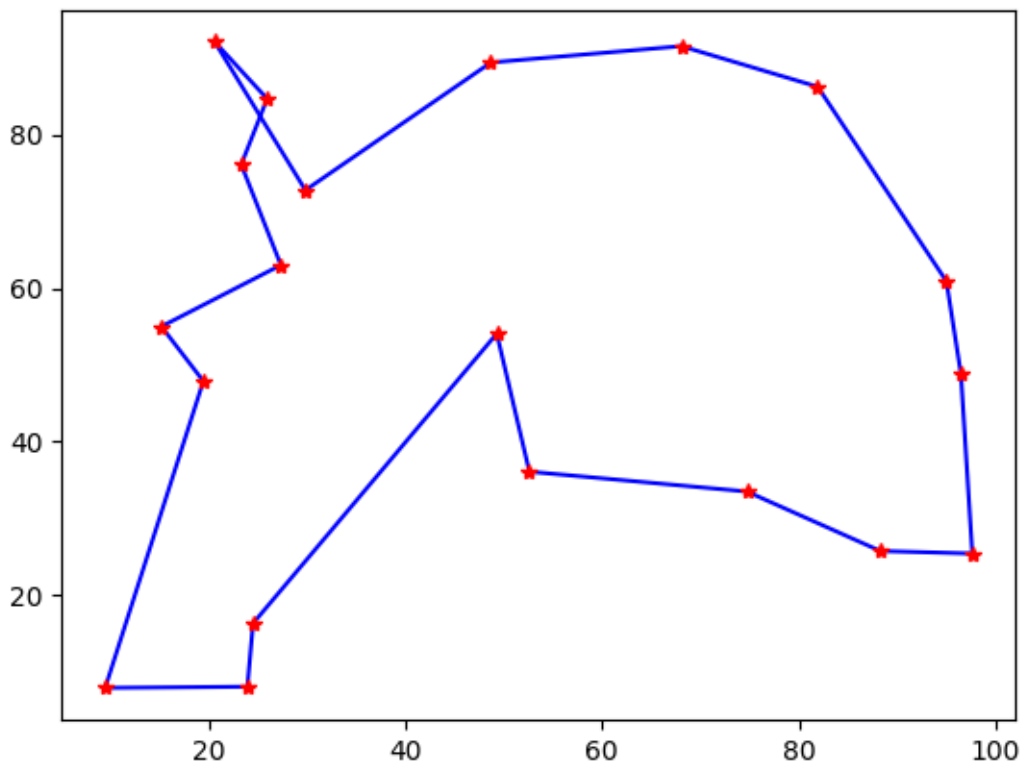

```
397.81467640465246 [ 7  1 13  5  4 11 14 12 19  0  3  8 18 17  6 10  9  2 16 15]
```



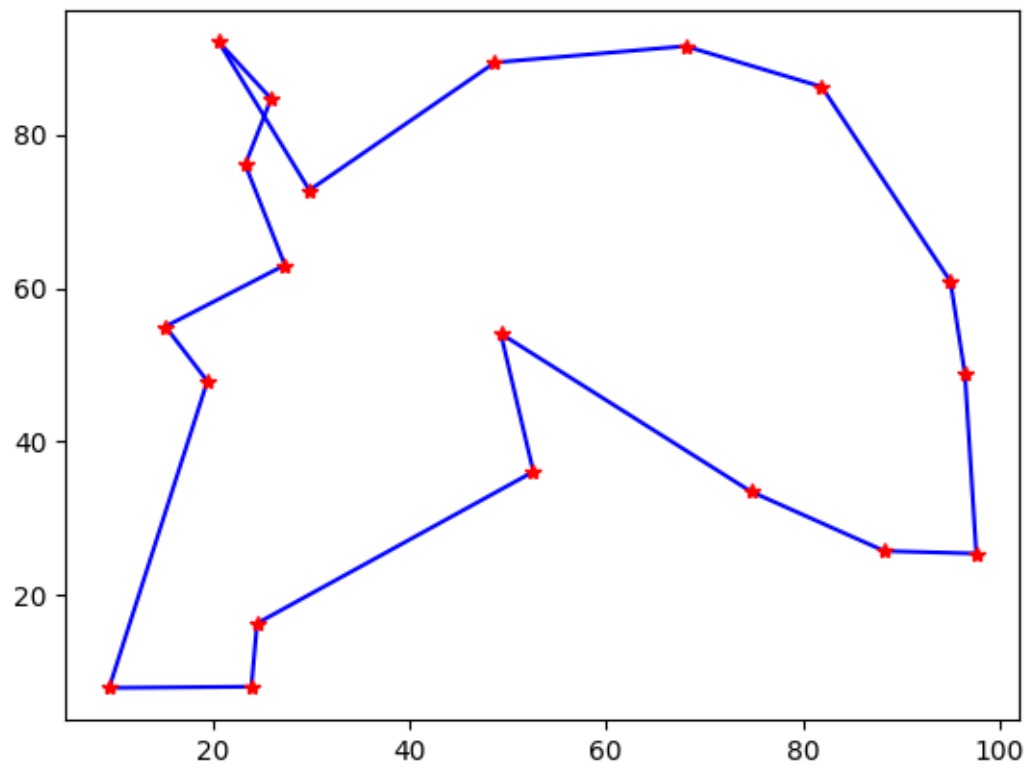

```
380.155495181076 [ 7  1 13  5  4 11 14 12 19  0  3  8 18 17  6 10  2  9 16 15]
```




```
374.03892549194217 [ 1  7 13  5  4 11 14 12 19  0  8  3 18 17  6 10  2  9 16 15]
```

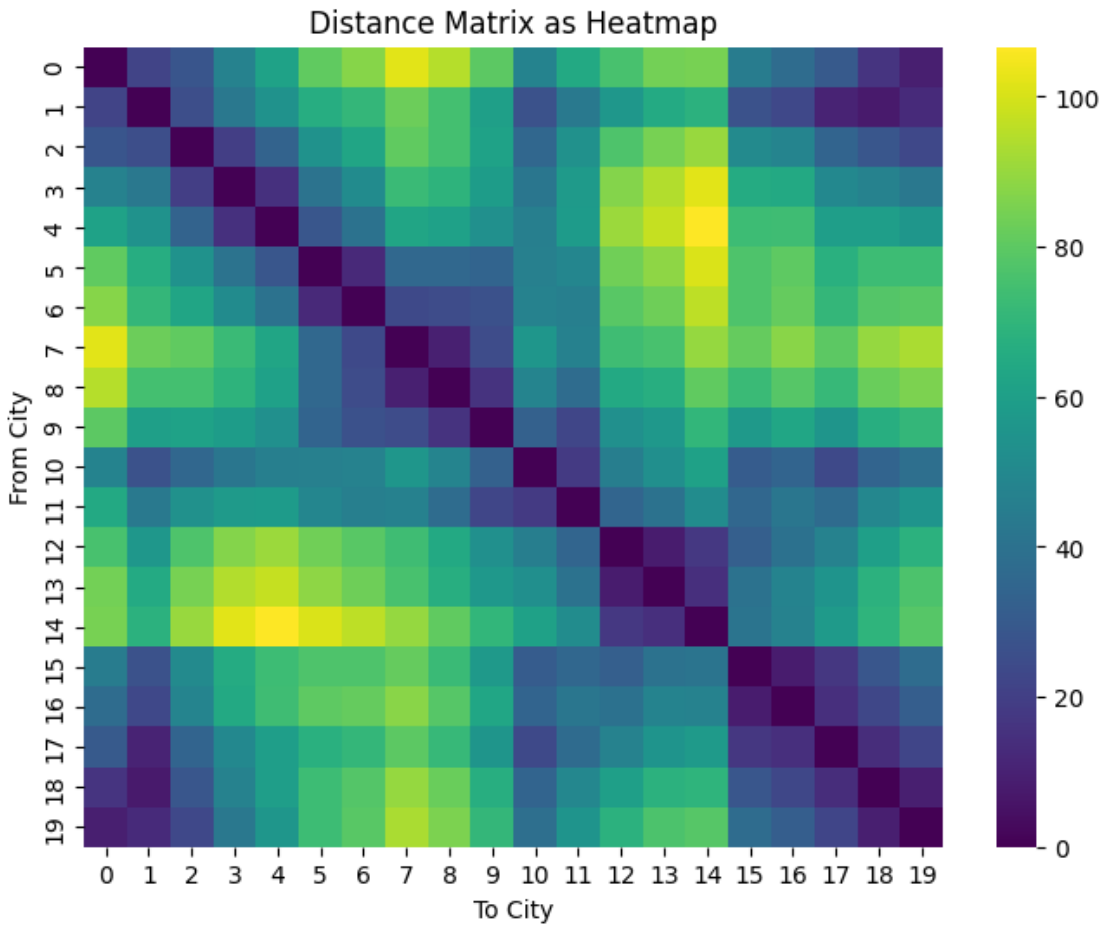



```
373.6861884185065 [ 1  7 13  5  4 11 14 12 19  0  3  8 18 17  6 10  2  9 16 15]
```



	0	1	2	3	4	5	6	7	8	9	10	\
0	0.0	21.4	28.2	47.4	61.6	80.7	87.3	101.8	94.7	79.7	47.6	
1	21.4	0.0	25.1	42.6	53.8	66.2	70.8	82.7	75.0	59.6	27.0	
2	28.2	25.1	0.0	19.5	33.4	54.4	62.6	80.5	74.9	61.6	35.3	
3	47.4	42.6	19.5	0.0	14.8	40.8	51.2	72.4	68.8	58.4	41.9	
4	61.6	53.8	33.4	14.8	0.0	28.5	40.1	62.8	60.8	53.2	45.8	
5	80.7	66.2	54.4	40.8	28.5	0.0	12.1	35.5	35.7	34.1	46.2	
6	87.3	70.8	62.6	51.2	40.1	12.1	0.0	23.4	24.5	26.6	47.5	
7	101.8	82.7	80.5	72.4	62.8	35.5	23.4	0.0	9.4	24.3	56.2	
8	94.7	75.0	74.9	68.8	60.8	35.7	24.5	9.4	0.0	15.6	48.2	
9	79.7	59.6	61.6	58.4	53.2	34.1	26.6	24.3	15.6	0.0	32.7	
10	47.6	27.0	35.3	41.9	45.8	46.2	47.5	56.2	48.2	32.7	0.0	
11	64.4	43.1	53.4	57.5	58.1	49.1	45.7	46.3	37.1	22.3	18.3	
12	75.8	56.6	76.9	86.8	90.4	83.4	79.0	73.7	64.5	53.1	45.1	
13	84.0	64.9	85.0	94.3	97.3	88.5	83.2	75.7	66.7	56.8	52.5	
14	84.8	67.9	90.4	102.0	106.6	100.6	96.1	89.9	80.8	70.1	60.9	
15	44.2	26.9	50.8	65.3	73.3	76.7	77.0	81.3	72.3	57.1	30.5	
16	37.4	23.0	48.0	64.2	73.7	80.0	81.5	87.6	78.7	63.3	34.1	
17	29.8	10.1	33.9	49.7	59.3	67.7	70.6	79.7	71.4	55.8	23.7	
18	16.2	7.3	28.7	47.3	59.5	73.3	78.1	89.9	82.2	66.7	34.0	
19	9.1	12.5	23.2	42.7	56.0	73.0	79.1	93.0	85.7	70.7	38.5	

	11	12	13	14	15	16	17	18	19
0	64.4	75.8	84.0	84.8	44.2	37.4	29.8	16.2	9.1
1	43.1	56.6	64.9	67.9	26.9	23.0	10.1	7.3	12.5
2	53.4	76.9	85.0	90.4	50.8	48.0	33.9	28.7	23.2
3	57.5	86.8	94.3	102.0	65.3	64.2	49.7	47.3	42.7
4	58.1	90.4	97.3	106.6	73.3	73.7	59.3	59.5	56.0
5	49.1	83.4	88.5	100.6	76.7	80.0	67.7	73.3	73.0
6	45.7	79.0	83.2	96.1	77.0	81.5	70.6	78.1	79.1
7	46.3	73.7	75.7	89.9	81.3	87.6	79.7	89.9	93.0
8	37.1	64.5	66.7	80.8	72.3	78.7	71.4	82.2	85.7
9	22.3	53.1	56.8	70.1	57.1	63.3	55.8	66.7	70.7
10	18.3	45.1	52.5	60.9	30.5	34.1	23.7	34.0	38.5
11	0.0	34.3	40.1	51.5	35.2	41.9	36.9	49.6	55.4
12	34.3	0.0	8.3	17.2	31.9	39.8	46.7	59.7	68.3
13	40.1	8.3	0.0	14.5	40.0	47.7	55.0	68.0	76.6
14	51.5	17.2	14.5	0.0	41.2	47.4	57.9	69.5	78.4
15	35.2	31.9	40.0	41.2	0.0	8.3	17.0	28.5	37.3
16	41.9	39.8	47.7	47.4	8.3	0.0	14.5	22.6	31.5
17	36.9	46.7	55.0	57.9	17.0	14.5	0.0	13.7	21.7
18	49.6	59.7	68.0	69.5	28.5	22.6	13.7	0.0	9.0
19	55.4	68.3	76.6	78.4	37.3	31.5	21.7	9.0	0.0


Distance Matrix as Heatmap
From City
To City
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
0
20
40
60
80
100